DISASTER RESPONSE INTELLIGENCE WAREHOUSE (DRIW)

Design, Implementation, and Validation of an Enterprise Data Warehouse for Disaster Impact, Evacuation, Relief, and Emergency-Response Analytics in the Philippines, 2020–2025

A Project Report Submitted in Partial Fulfillment of the Requirements for

IT271 – Enterprise Data Warehousing

Submitted by

Ivan Christian Salinas

October 2026

Repository: github.com/sardinexszc/NEUSTIT271_EDW_Project

Data notice: event names, dates and PSGC geography are real. All damage, evacuation, relief and response figures are synthetic, generated for demonstration.

CONTENTS

Executive Summary 9
Problem 9
Proposed solution 9
What is real and what is synthetic 10
Key evidence 10
Limitations 10
How to read this report 11
Requirements Coverage 12
List of Abbreviations 14
1. Business Case and Requirements Analysis 15
1.1 Enterprise domain 15
1.2 Stakeholders and BI objectives 16
1.3 Key business questions 16
1.4 Functional requirements 17
1.5 Non-functional requirements 18
1.6 Assumptions and constraints 18
2. Data Warehouse Design 19
2.1 Conceptual model 19
2.2 Logical model 20
2.3 Physical model 22
2.4 DRIW Dimensional Model 29
3. Data Dictionary and Metadata 35
3.1 Schemas (layers) 35
3.2 Lineage 35
3.3 dw · Star schema 36
3.4 mart · Data marts 44

3.5 Supporting layers 49
4. ETL Design 53
4.1 Overview 53
4.2 Data sources 53
4.3 Extract (raw layer) 54
4.4 Transform (staging and data-quality layers, dbt) 55
4.5 Load (warehouse and marts) 55
4.6 Orchestration, scheduling and error handling 56
4.7 Tools 56
5. Data Quality Framework 57
5.1 Where data quality happens 57
5.2 Severity policy 57
5.3 Key techniques 58
5.4 Proof: reconciliation against planted defects 59
5.5 Running 59
6. Data Governance, Security and Privacy 60
6.1 Roles 60
6.2 Row-level security (regional users) 61
6.3 Surviving rebuilds 62
6.4 Auditing access 62
6.5 Privacy 62
6.6 Scope of the security evidence 63
6.7 Supabase 64
6.8 Standards alignment 64
7. Analytics: OLAP and Dashboards 65
7.1 OLAP approach (ROLAP) 65
7.2 Dashboards and web application walkthrough 65

7.3 Sample findings (synthetic data) 91
8. Testing and Validation 93
8.1 Summary 93
8.2 What is validated 93
8.3 Defects found by testing (and fixed) 95
8.4 Known limitations 96
8.5 Query Performance Test Report 96
9. Implementation and Live Demo 99
9.1 Technology stack 99
9.2 Repository layout 99
9.3 Build and run 100
9.4 Live demo script 100
10. Enterprise Positioning and Data Exchange 102
10.1 Why an enterprise data warehouse has no content management system 102
10.2 Data exchange with LGUs and national agencies 103
10.3 Other ways in and out 111
10.4 Next steps 111
11. Conclusion and Recommendations 112
11.1 Summary 112
11.2 What the analysis demonstrates 112
11.3 Limitations 113
11.4 Recommendations 113
References 115
Appendix A. Sample Analytical Queries 117

LIST OF FIGURES

Figure 1: Conceptual model of DRIW 19
Figure 2: ER diagram of the damage assessment star 24
Figure 3: ER diagram of the evacuation snapshot star 25
Figure 4: ER diagram of the resource distribution star 26
Figure 5: ER diagram of the response action star 27
Figure 6: ER diagram of the displacement outside evacuation centers star 28
Figure 7: ER diagram of the lifeline disruption star 29
Figure 8: ETL workflow 53
Figure 9: Executive dashboard 68
Figure 10: Close-up of the Executive headline tiles 69
Figure 11: Close-up of deaths per 100,000 people 70
Figure 12: Operations dashboard for Typhoon Kristine (October 2024) 73
Figure 13: Close-up of the Operations headline tiles for Kristine 74
Figure 14: Close-up of daily displacement for Kristine 75
Figure 15: Close-up of lifeline restoration for Kristine 76
Figure 16: Logistics dashboard for all events 79
Figure 17: Close-up of the cost of assistance by provider 80
Figure 18: Logistics dashboard filtered to Typhoon Kristine 83
Figure 19: About page 84
Figure 20: API health check 85
Figure 21: API response for the event summary mart 85
Figure 22: Operations page on a narrow screen 91
Figure 23: Data exchange page 108
Figure 24: Check of the Albay PDRRMO evacuation report 109

LIST OF TABLES

Table 1: What is real and what is synthetic in DRIW 10
Table 2: Headline results of the project 10
Table 3: Requirements traceability matrix 12
Table 4: Abbreviations and acronyms used in this report 14
Table 5: Agencies in a Philippine disaster response and their systems of record 15
Table 6: Stakeholders, their needs and the BI objectives that serve them 16
Table 7: Key business questions (BQ1–BQ12) 16
Table 8: Functional requirements and how they are met 17
Table 9: Non-functional requirements, targets and evidence 18
Table 10: Business processes, the agency reporting each, and the entities involved 20
Table 11: Fact tables of the logical model 21
Table 12: Dimensions of the logical model 21
Table 13: Design and technology choices and the reasons for them 22
Table 14: Physical layers as PostgreSQL schemas 23
Table 15: Fact tables with grain and row counts 30
Table 16: Dimension tables with natural keys and row counts 31
Table 17: SCD Type 2 versions of Parang, Maguindanao 32
Table 18: Bus matrix of facts and conformed dimensions 33
Table 19: Data marts and the dashboards they feed 34
Table 20: Database schemas and the layer each one holds 35
Table 21: Lineage from each source file to its warehouse table 35
Table 22: Columns of dw.dim_damage_category 36
Table 23: Columns of dw.dim_date 36
Table 24: Columns of dw.dim_evacuation_center 37
Table 25: Columns of dw.dim_event 37
Table 26: Columns of dw.dim_lifeline 38
Table 27: Columns of dw.dim_location 38
Table 28: Columns of dw.dim_organization 39
Table 29: Columns of dw.dim_resource 39
Table 30: Columns of dw.dim_response_type 39
Table 31: Columns of dw.dim_source_system 40
Table 32: Columns of dw.fact_damage_assessment 40

Table 33: Columns of dw.fact_displacement_outside 41
Table 34: Columns of dw.fact_evacuation_snapshot 41
Table 35: Columns of dw.fact_lifeline_disruption 42
Table 36: Columns of dw.fact_resource_distribution 42
Table 37: Columns of dw.fact_response_action 43
Table 38: Columns of mart.mart_displacement_daily 44
Table 39: Columns of mart.mart_evacuation_daily 44
Table 40: Columns of mart.mart_event_summary 45
Table 41: Columns of mart.mart_impact_cube 46
Table 42: Columns of mart.mart_lifeline_restoration 47
Table 43: Columns of mart.mart_province_event_impact 47
Table 44: Columns of mart.mart_relief_by_org_category 48
Table 45: Columns of mart.mart_response_performance 48
Table 46: Supporting objects in the raw, reference, staging, data-quality and governance layers 49
Table 47: Data sources with format, volume and planted defects 53
Table 48: Orchestration, scheduling and error handling 56
Table 49: Tools used by the ETL pipeline 56
Table 50: Data-quality columns added to every staging row 57
Table 51: Severity policy of the data-quality rules 57
Table 52: Reconciliation of planted and detected defects 59
Table 53: Database roles and what each may read 60
Table 54: Region mapping used by row-level security 61
Table 55: Data classification under the Data Privacy Act 62
Table 56: OLAP operations and how DRIW performs them in SQL 65
Table 57: Elements common to every dashboard page 66
Table 58: Parts of the Executive dashboard and how to read them 69
Table 59: Parts of the Operations dashboard and how to read them 74
Table 60: Lifeline disruption and restoration during Kristine 76
Table 61: Parts of the Logistics dashboard and how to read them 79
Table 62: Test suites and their results 93
Table 63: Defects found by testing, and their fixes 95
Table 64: Timing of the 14 sample analytical queries 97
Table 65: Technology stack 99

Table 66: Repository layout 99
Table 67: Source systems, warehouse and business intelligence as a chain 102
Table 68: Inmon's characteristics of a data warehouse, and the evidence in DRIW 103
Table 69: Enterprise capabilities and where DRIW provides them 103
Table 70: Capabilities of the Data exchange page 105
Table 71: Verdict on each row of the Albay report 110
Table 72: Other ways data enters and leaves DRIW 111
Table 73: Next steps for data exchange 111
Table 74: Project objectives and what was achieved 112

Executive Summary

Problem

A Philippine typhoon or monsoon flood is reported by several agencies at once: NDRRMC (damage and lifelines), DSWD (people displaced inside and outside evacuation centers), NGOs, UN agencies and LGUs (relief), PAGASA (the hazard) and LGUs (response requests). Their systems share no place codes, event names or units. Leadership therefore cannot answer cross-agency questions such as "which provinces got the least relief per affected person?" without weeks of manual reconciliation.

Proposed solution

DRIW, the Disaster Response Intelligence Warehouse, is an enterprise data warehouse that integrates the five sources into one dimensional model. A batch ETL pipeline cleans the source files, conforms them to official place codes and event names, and checks them against data-quality rules. The cleaned data is stored as star schemas: fact tables of measures that share conformed dimensions such as date, place and event. Pre-aggregated marts serve dashboards for three audiences: executives, operations managers and logistics officers. Besides damage, casualties, evacuation, relief and response, the dashboards report measures that disaster-risk practice asks for: deaths and affected people per 100,000 (Sendai Framework targets A and B; UNDRR, 2015), people displaced outside evacuation centers, the restoration of roads, bridges, power, water and communications, and the cost of assistance by provider.

The system is built with free, widely used tools: PostgreSQL 17 as the warehouse, Python and dbt for the ETL and its tests, and a PHP back-end on XAMPP that serves the dashboards through a read-only database login. Chapter 2 explains why each was chosen.

What is real and what is synthetic

Agency operational data is not public, so the operational figures are generated (seed 271), scaled to population and storm strength. Everything else is real or built exactly as it would be on agency data. Table 1 separates the two.

Table 1: What is real and what is synthetic in DRIW

Everything that shows the warehouse works is real; only the operational figures, and the source files that carry them, are generated.

Real	Simulated (synthetic)
The architecture, ETL pipeline, data-quality rules, star schema, marts, security and dashboards: built and run exactly as they would be on agency data	Every damage, casualty, evacuation, relief and response **figure** in this report and on the dashboards
The 31 events (PAGASA names and dates, 2020–2025)	The source files themselves (CSV, Excel, JSON) and their data-entry defects
All geography: PSA PSGC, 17 regions, 81 provinces, 1,634 cities and municipalities, 42,046 barangays	

The numbers demonstrate what the warehouse can do; they are not findings about real disasters. Synthetic data has one advantage for testing: 9,995 defects of 19 known types were planted on purpose, so the ETL can be shown to be correct by counting what it finds.

Key evidence

Each claim in Table 2 is measured by an automated test or a repeatable report described in Chapter 8, run on the submitted version of the project.

Table 2: Headline results of the project

The evidence column names how each result was measured.

Measure	Result	Evidence
Source rows integrated	113,191 from 5 systems in 4 formats	Row counts per raw table (Chapter 4)
Planted defects detected	19 of 19 types, each at the exact count (9,995 defects)	`dq.dq_reconciliation` (Chapter 5)
Place names matched to official codes	63,347 rows checked against known true codes, 0 false matches	dbt test on every LGU and DROMIC row that carries a true code (Chapter 5)
Automated checks	117 dbt checks and 91 end-to-end, governance, API and data-exchange tests, all passing	Test reports (Chapter 8)
Query performance	14 of 14 sample queries within 1 s (slowest about 34 ms); marts about 60–80× faster than the facts	Benchmark on a laptop, 5 timed runs per query (Chapter 8)
Access control	4 roles; row-level security by region	Per-role access tests (Chapter 6)

Limitations

- **Synthetic figures.** The dashboard findings show the analysis the warehouse supports, not real outcomes. Replacing the generator with agency extracts changes the input files only.

- **Estimated population.** Per-100,000 rates use a population estimate (barangay count × 2,600) until the PSA census file is loaded, so they compare events with each other.
- **No live agency connection.** Data exchange is file-based: Excel templates and checked uploads, tested with sample files. No agency system is connected to DRIW.
- **Tested, not audited.** The security tests show the designed access rules hold; they are not a security audit or penetration test.
- **Batch loads.** The warehouse is reloaded in full on each run; incremental, scheduled loads are future work.

How to read this report

The Requirements Coverage page that follows maps each item of the project brief to the chapter that meets it, and the List of Abbreviations explains the agency and technical acronyms. Chapters 1–2 cover the business case and the design, 3–5 the metadata, ETL and data quality, 6 governance and privacy, 7 the analytics, 8 testing, 9 how to run the live demo, 10 the warehouse's place in the enterprise and its data exchange with LGUs, and 11 the conclusions and recommendations. The References list the standards and sources cited, and Appendix A lists the sample queries.

Requirements Coverage

Each requirement of the IT271 semester project brief, and where this submission meets it.

Table 3 maps each requirement of the project brief to the chapter and section that meets it.

Table 3: Requirements traceability matrix

Every requirement of the IT271 project brief is listed with the chapter or section that meets it, so coverage can be checked item by item.

Brief section	Requirement	Where it is met
1. Overview	Requirements gathering and BI objectives	Chapter 1 (Business Case), §2–§5
1. Overview	Data modeling: conceptual, logical, physical	Chapter 2 (Warehouse Design), §1–§3
1. Overview	ETL processes	Chapter 4 (ETL Design)
1. Overview	OLAP cube or star/snowflake schema	Chapter 2 (six star schemas); Chapter 7 (ROLAP cube)
1. Overview	Data governance and security	Chapter 6 (Governance, Security and Privacy)
1. Overview	Analytics, dashboards, reports	Chapter 7 (Analytics)
2A	Description of the enterprise domain	Chapter 1 §1
2A	Key business questions	Chapter 1 §3 (BQ1–BQ12)
2A	Functional and non-functional requirements	Chapter 1 §4 (FR1–FR12), §5 (NFR1–NFR9)
2A	Schema design: star or snowflake (fact + dimension tables)	Chapter 2 §2; Chapter 2 (dimensional model detail)
2A	ER/UML diagrams of the warehouse design	Chapter 2: conceptual diagram and six ER diagrams
2A	Data dictionary and metadata documentation	Chapter 3 (generated from the catalog; lineage, row counts, load times)
2A	ETL: sources of data (mock or real)	Chapter 4 §2
2A	ETL: transformation logic (cleaning, integration, aggregation)	Chapter 4 §4; Chapter 5
2A	ETL workflow diagram	Chapter 4 §1
2A	Governance: security, privacy, access controls	Chapter 6 (roles, row-level security, web login, Privacy section)
2A	Governance: data quality management strategy	Chapter 5 (Data Quality Framework)
2B	Warehouse on SQL Server, Oracle, PostgreSQL, MySQL or cloud DW	PostgreSQL 17 (Chapter 2 §3, Chapter 9)
2B	ETL pipelines (tools or custom Python/SQL)	Python + dbt, `ingestion/pipeline.py` (Chapter 4)
2B	OLAP cube / analytical queries	`mart_impact_cube` (CUBE), GROUPING SETS, 14 sample queries (Chapter 7, Appendix A)
2B	Reports/dashboards (Power BI, Tableau or similar)	Executive, Operations, Logistics dashboards on XAMPP, with a screenshot and discussion of every page (Chapter 7 §2)
2C	Validation of ETL: completeness, correctness, accuracy	Chapter 8 (117 dbt checks, 91 automated tests, 19/19 reconciliation, 0 false matches)
2C	Test reports for data consistency and query performance	Chapter 8 (consistency); Query Performance Test Report
2D	Live demo	Chapter 9 §4 (demo script)
2D	Showcase of sample queries, OLAP analysis, dashboards	Appendix A, Chapter 7

Brief section	Requirement	Where it is met
2D	Defense of design choices	Chapter 2 (star vs snowflake, ROLAP), dimensional-model design decisions, Chapter 4 (full reload, quarantine); Chapter 11 (conclusions and recommendations)
2D	Enterprise positioning	Why an EDW needs no CMS; Excel data exchange with LGUs and agencies: templates, layout profiles, checked uploads, Excel export (Chapter 10)

List of Abbreviations

Table 4 explains the abbreviations used in this report.

Table 4: Abbreviations and acronyms used in this report

Agency names and technical terms are listed together in alphabetical order. BQ, FR, NFR and Q (sample query, Appendix A) are numbering prefixes used within this report.

Abbreviation	Meaning
API	Application programming interface
BI	Business intelligence
BQ	Business question (BQ1–BQ12, Chapter 1)
CDRRMO	City Disaster Risk Reduction and Management Office
CI	Continuous integration (automated tests on every change)
CMS	Content management system
CSV	Comma-separated values (a plain-text table file)
DQ	Data quality
DRIW	Disaster Response Intelligence Warehouse (this project)
DRRM	Disaster risk reduction and management
DROMIC	Disaster Response Operations Monitoring and Information Center (DSWD)
DSWD	Department of Social Welfare and Development
EC	Evacuation center
EDW	Enterprise data warehouse
ER, ERD	Entity-relationship (diagram)
ETL	Extract, transform, load
FK	Foreign key
FR, NFR	Functional requirement, non-functional requirement
JSON	JavaScript Object Notation (a structured text format)
LGU	Local government unit (province, city or municipality)
MDRRMO	Municipal Disaster Risk Reduction and Management Office
MOLAP, ROLAP	Multidimensional OLAP, relational OLAP
NDRRMC	National Disaster Risk Reduction and Management Council
NGO	Non-governmental organization
OCD	Office of Civil Defense
OLAP	Online analytical processing
PAGASA	Philippine Atmospheric, Geophysical and Astronomical Services Administration
PDRRMO	Provincial Disaster Risk Reduction and Management Office
PHP	Philippine peso (for amounts); also the PHP language of the web back-end
PK	Primary key
PRC	Philippine Red Cross
PSA	Philippine Statistics Authority
PSGC	Philippine Standard Geographic Code
RA	Republic Act
RLS	Row-level security
SCD	Slowly changing dimension (Type 0, 1 or 2)
SQL	Structured Query Language
UN	United Nations
UNDRR	United Nations Office for Disaster Risk Reduction
UNICEF	United Nations Children's Fund
WFP	World Food Programme
XAMPP	Apache web server bundle with PHP (the local web host)

1. Business Case and Requirements Analysis

1.1 Enterprise domain

Government: disaster risk reduction and management (DRRM) in the Philippines.

The Philippines is struck by around twenty tropical cyclones a year, plus monsoon floods. Responding to one event involves many agencies, each with its own system.

Table 5 lists the agencies involved and the system each one keeps.

Table 5: Agencies in a Philippine disaster response and their systems of record
Five organizations report the same event in five unconnected systems; this fragmentation is the problem DRIW solves.

Organization	Role in a disaster	System of record (simulated)
NDRRMC / Office of Civil Defense	Consolidates damage and casualty figures	Situation reports (CSV), cumulative, re-issued per event
DSWD (DROMIC)	Runs and reports evacuation centers	Daily Excel workbooks per field office
NGOs and UN agencies (PRC, Caritas, WFP, UNICEF) and DSWD	Distribute relief goods	Partner reports (nested JSON) with their own item codes and units
PAGASA	Names and tracks hazard events	Weather bulletins (CSV)
LGUs / MDRRMOs	Rescue, evacuation, road clearing, relief delivery	Response request logs (CSV)

The NDRRMC coordinates this response under the Philippine Disaster Risk Reduction and Management Act (Republic Act No. 10121, 2010); DROMIC and NDRRMC publish their reports online (DSWD, n.d.; NDRRMC, n.d.), and PAGASA names the events (PAGASA, n.d.). These systems share no place codes, event names or units. Leadership therefore cannot answer cross-agency questions without manual reconciliation in spreadsheets.

Scope. Tropical cyclones and southwest-monsoon floods, 2020–2025: 31 events. Event names and dates (PAGASA) and all geography (PSA PSGC: 17 regions, 81 provinces, 1,634 cities and municipalities, 42,046 barangays) are real. Operational figures are synthetic, generated with counted, planted defects (113,191 source rows; 9,995 defects of 19 types), because agency data is not public and known defects let the ETL be proven correct.

1.2 Stakeholders and BI objectives

Five groups of stakeholders use the warehouse; Table 6 sets out what each needs and which part of DRIW serves it.

Table 6: Stakeholders, their needs and the BI objectives that serve them

Each stakeholder group is mapped to the dashboard, mart or schema built for it.

Stakeholder	Needs	BI objective	Served by
Executive leadership (NDRRMC council, agency heads)	Overall impact, trends, where to invest	Single view of damage, casualties and relief across all agencies and years	Executive dashboard, `mart_event_summary`, `mart_impact_cube`
Operations managers (regional DRRM offices)	During and after an event: evacuation load, response speed	Per-event evacuation and response performance	Operations dashboard, `secure.*` views for their region
Logistics officers	Whether relief matched need	Relief by organization and resource, coverage per province	Logistics dashboard, `mart_province_event_impact`
Data analysts	Ad-hoc analysis, forecasting inputs	Clean, conformed star schema with history	`dw.*`, sample queries
Data stewards / IT	Data quality, reconciliation, operations	Measurable data quality with traceability to source	`dq.*`, quarantine, lineage

1.3 Key business questions

The design starts from the questions decision makers ask. Table 7 lists the twelve key business questions and where each is answered.

Table 7: Key business questions (BQ1–BQ12)

Each question is answered by a sample query (Q-number) or a dashboard, so every part of the design traces back to a business need.

#	Business question	Answered by
BQ1	Which events caused the most damage, deaths and displacement?	Q01, Q13; Executive dashboard
BQ2	How does damage trend by year and category, and how did each year compare with the last?	Q02, Q05, Q12; Executive dashboard
BQ3	Within an event, which regions, provinces and towns were hit hardest?	Q03 (drill-down)
BQ4	Which regions suffer most from typhoons versus monsoon floods?	Q04, Q05 (cube); Executive dashboard
BQ5	How many people were displaced at the peak, in evacuation centers and outside them, and for how long?	Q06; Operations dashboard
BQ6	Which evacuation centers are repeatedly overcrowded?	Q07; Operations dashboard
BQ7	How quickly are response teams dispatched, by request type and priority, and where is it slowest?	Q08, Q09; Operations dashboard
BQ8	Did relief reach provinces in proportion to impact? Which were under-served?	Q10; Logistics dashboard
BQ9	Which organizations delivered which kinds of relief, and who paid for it (DSWD, LGUs, NGOs, international)?	Q11; Logistics dashboard

#	Business question	Answered by
BQ10	How reliable is the data: what was rejected, and why?	Q14; `dq.dq_reconciliation`
BQ11	Which events hit hardest relative to the population (deaths and affected per 100,000, Sendai targets A and B)?	`mart_event_summary`; Executive dashboard
BQ12	How long were roads, bridges, power, water and communications down, and which take longest to restore?	`mart_lifeline_restoration`; Operations dashboard

The Q-numbers refer to Appendix A.

1.4 Functional requirements

Table 8 states what the system must do and how each requirement is met.

Table 8: Functional requirements and how they are met

All twelve are met; the third column names the component that delivers each one.

#	Requirement	How it is met	Status
FR1	Ingest data from the five agency systems in their native formats (CSV, Excel, JSON).	`ingestion/load_raw.py` lands every file in `raw.*` unchanged.	Met
FR2	Keep lineage from every warehouse row to its source file and row.	`meta.load_batch`, `_batch_id/_source_file/_row_number`, `dim_source_system`.	Met
FR3	Conform place names to the official PSGC codes.	`stg_location_resolution`: exact, normalized, fuzzy; 0 false matches on ground truth.	Met
FR4	Conform event names, organizations and relief units across sources.	Event aliases with time windows; organization and item alias maps; unit conversion.	Met
FR5	Detect, correct, flag or quarantine data-quality defects, with reasons.	45 rules in `dq_rules`; `dq_quarantine`; 19/19 planted defect types reconciled.	Met
FR6	Store integrated data in a dimensional model with conformed dimensions.	Star schema: 6 facts, 10 dimensions (Chapter 2).	Met
FR7	Preserve history when administrative boundaries change.	SCD Type 2 `dim_location` (2022 Maguindanao split).	Met
FR8	Support OLAP analysis: roll-up, drill-down, slice, dice, pivot.	`GROUP BY CUBE / ROLLUP / GROUPING SETS`, marts, sample queries Q01–Q14.	Met
FR9	Provide dashboards for executive, operational and logistics users.	Next.js dashboards on XAMPP: Executive, Operations, Logistics.	Met
FR10	Let users filter by event, share views and export data.	`?event=` links, CSV download on every chart table.	Met
FR11	Restrict data by role, including region-level restriction.	4 roles, row-level security, `driw_web` least-privilege login.	Met
FR12	Rebuild the warehouse end to end with one command.	`python ingestion/pipeline.py`.	Met

1.5 Non-functional requirements

Table 9 sets a measurable target for each quality attribute and gives the evidence that it was met.

Table 9: Non-functional requirements, targets and evidence

Each quality attribute has a measurable target and the evidence that it was reached.

#	Requirement	Target	Evidence	Status
NFR1	Data accuracy	Every planted defect type detected at the exact planted count	`dq.dq_reconciliation`: 19/19 match	Met
NFR2	Query performance	Analytical queries under 1 second	Query Performance Test Report (Chapter 8): 14/14, slowest about 34 ms	Met
NFR3	Security	Least privilege; regional row-level restriction; no credentials in code	Chapter 6; 27 governance tests (access per role) and the web-role tests	Met
NFR4	Privacy	No personal data in the warehouse; casualty data aggregated	Chapter 6, Privacy section	Met
NFR5	Reliability	A failed source row never stops the load; rejected rows are kept with reasons	Quarantine design; staging keeps every row	Met
NFR6	Reproducibility	Same inputs produce the same warehouse	Deterministic generator (seed 271); e2e test rebuilds from nothing	Met
NFR7	Maintainability	Version-controlled, tested, documented	GitHub repo, dbt tests, CI on every push, generated data dictionary	Met
NFR8	Usability and accessibility	Readable on desktop and phone; color-blind-safe charts; table view for every chart	Validated palette, legends, "Show table", responsive layout	Met
NFR9	Portability	Run on the agency's own infrastructure without cloud services	Local PostgreSQL 17 + XAMPP (Apache, PHP); migrations also run on Supabase	Met

1.6 Assumptions and constraints

- Operational figures are synthetic; they are scaled to population and storm strength to be plausible, not to match official NDRRMC totals.
- City population is estimated from barangay counts, because the PSA workbook with census population could not be downloaded automatically.
- Loads are batch (full reload), not real time; near-real-time feeds are future work.
- Standards followed: the Disaster-related Statistics Framework (ESCAP, 2018), the Sendai Framework 2015–2030 and its monitoring guidance (UNDRR, 2015, 2017), the UNDRR Data Strategy and Roadmap 2023–2027 (UNDRR, 2023), the PSGC (Philippine Statistics Authority, n.d.) and the Data Privacy Act of 2012 (Republic Act No. 10173, 2012).

2. Data Warehouse Design

The design moves from business concepts to tables in three levels. The logical level is detailed in the Dimensional Model section at the end of this chapter (grains, bus matrix, SCD Type 2, design decisions); the full column list is in Chapter 3.

2.1 Conceptual model

The DRIW answers questions about six **business processes**, each recorded by an agency. Every process involves the same core **entities**: a hazard event, a place, a date, the organizations involved and the source that reported it. These shared entities are what allow the processes to be analyzed together.

Figure 1 shows the conceptual model: the business processes and the entities they share.

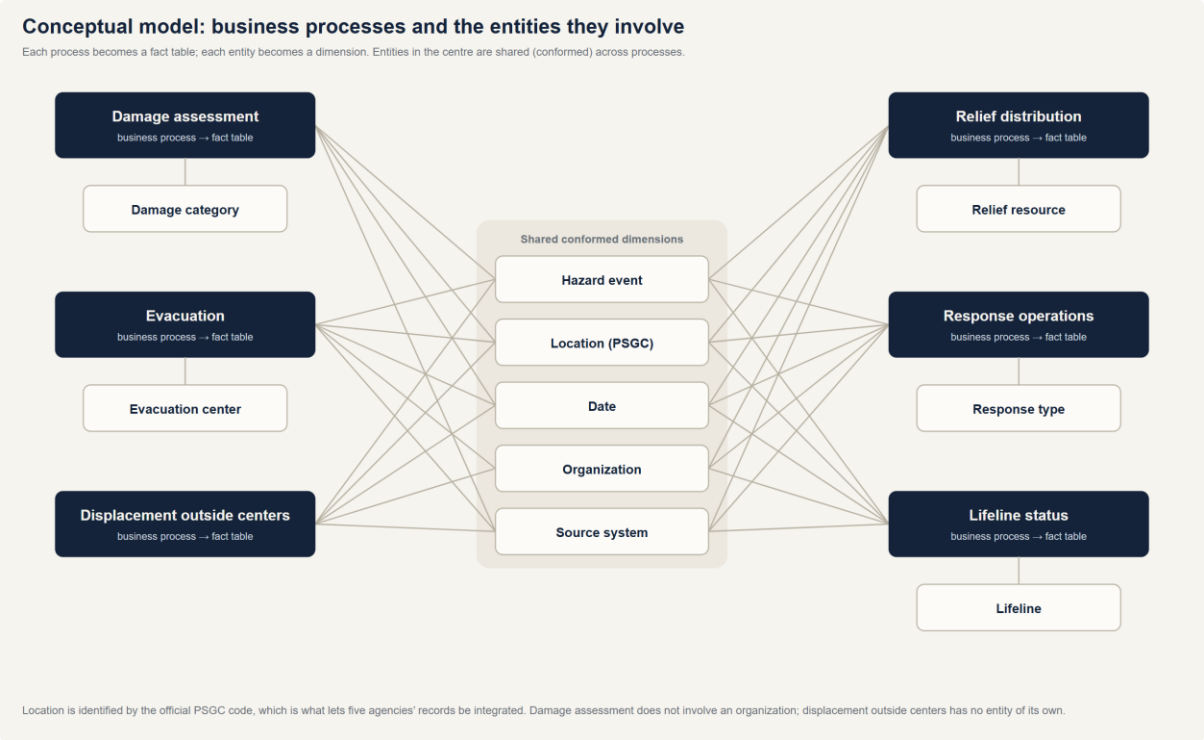


Figure 1: Conceptual model of DRIW

The six business processes (dark boxes) become fact tables; the shared entities in the centre become conformed dimensions, which is what lets the processes be analyzed together.

Table 10 lists each process with the agency that reports it and the question it answers.

Table 10: Business processes, the agency reporting each, and the entities involved
The entities of each process become the dimensions of its star.

Business process	Reported by	Key question it answers	Entities involved
Damage assessment	NDRRMC	How bad was it, and where?	Event, Location, Date, Damage category, Source
Evacuation	DSWD DROMIC	How many were displaced, and were centers adequate?	Event, Location, Date, Organization, Evacuation center, Source
Relief distribution	NGOs, UN agencies, DSWD	Did relief reach the people affected?	Event, Location, Date, Organization, Relief resource, Source
Response operations	LGUs / MDRRMOs	How fast did help arrive?	Event, Location, Date, Organization, Response type, Source
Displacement outside centers	DSWD DROMIC	How many families stayed with relatives or friends instead of in a center?	Event, Location, Date, Organization, Source
Lifeline status	NDRRMC	Which roads, bridges, power, water and communication lines went down, and for how long?	Event, Location, Date, Organization, Lifeline, Source

Why a dimensional (Kimball) design. The users ask aggregate, multi-perspective questions ("damage by region by year by hazard"), not transactional ones. A dimensional model (Kimball & Ross, 2013) makes those questions simple to write and fast to run, and conformed dimensions let the six processes be compared directly. The enterprise integration happens in the staging layer before data reaches the stars.

2.2 Logical model

Each business process becomes a **fact table** at a stated grain; each entity becomes a **dimension**. The model is a set of six **star schemas** sharing conformed dimensions (a "constellation").

Table 11 lists the six fact tables with their type, grain and measures.

Table 11: Fact tables of the logical model

Each fact is declared at one grain and typed by how it records the business: transactions, periodic snapshots or accumulating snapshots.

Fact table	Type	Grain (one row per…)	Measures
`fact_damage_assessment`	Transaction	event × city/municipality × damage category	dead, injured, missing, affected families/persons, houses damaged, cost (PHP)
`fact_evacuation_snapshot`	Periodic snapshot	evacuation center × day × event	families inside, persons inside, capacity, over-capacity flag
`fact_resource_distribution`	Transaction	relief item in one distribution	quantity (standard unit), value (PHP), beneficiary families
`fact_response_action`	Accumulating snapshot	response request	hours to dispatch / arrive / on site / complete, personnel, completed flag
`fact_displacement_outside`	Periodic snapshot	city/municipality × day × event	families and persons displaced outside evacuation centers
`fact_lifeline_disruption`	Accumulating snapshot	event × city/municipality × lifeline	units affected, hours to restore, restored flag

Table 12 lists the ten dimensions with their keys, history handling and hierarchies.

Table 12: Dimensions of the logical model

History says how changes are handled (Type 0 never changes, Type 1 overwrites, Type 2 keeps versions); the last column gives the drill-down paths.

Dimension	Natural key	History	Hierarchy / notable attributes
`dim_date`	calendar date	Type 0	year → quarter → month → day; typhoon season; -1 "Not yet occurred"
`dim_location`	PSGC code	**Type 2**	region → province → city/municipality; population (Type 1, denominator for per-100,000 rates)
`dim_event`	event code	Type 1	hazard type → category → event
`dim_organization`	org code	Type 1	org type → organization
`dim_resource`	item code	Type 1	category → item
`dim_evacuation_center`	EC code	Type 1	type, barangay, capacity
`dim_damage_category`	category	Type 0	—
`dim_response_type`	type code	Type 0	cluster → type
`dim_lifeline`	lifeline code	Type 0	sector (transport, utilities) → lifeline; unit of measure
`dim_source_system`	batch id	—	lineage: source system → file

Star, not snowflake. Dimensions are kept denormalized (region and province names sit in `dim_location` rather than in separate region and province tables). This costs a little storage but keeps every query to one join per dimension, which suits the BI users, and the hierarchies are small. The PSGC master data itself stays normalized in `ref.psgc`, so the star is derived from a single authoritative source.

The bus matrix, measure additivity rules and the SCD Type 2 treatment of the 2022 Maguindanao split are in the Dimensional Model section at the end of this chapter.

2.3 Physical model

Platform. PostgreSQL 17 (local; the same migrations run unchanged on Supabase). Database size about 80 MB.

Design and technology choices. Each major choice was made against the requirements in Chapter 1, not for convenience. Table 13 lists the alternatives considered and the reason for each decision.

Table 13: Design and technology choices and the reasons for them

Each decision traces to a requirement (FR or NFR, Chapter 1) or to the nature of the business questions.

Decision	Alternatives considered	Reason
Dimensional (Kimball) model	Normalized enterprise model (Inmon)	The business questions are aggregate and multi-perspective ("damage by region by year"); a dimensional model answers them with simple, fast queries, and conformed dimensions still integrate the agencies (Kimball & Ross, 2013).
Star schema	Snowflake schema	One join per dimension keeps queries simple for analysts; hierarchies are small, so the extra storage is negligible.
PostgreSQL 17	SQL Server, Oracle, MySQL, a cloud warehouse	Free and runs on an agency's own server (NFR9); supports `CUBE`, `ROLLUP`, `GROUPING SETS`, materialized views and row-level security natively (FR8, FR11), which MySQL lacks; the same migrations run on a cloud database.
ROLAP with materialized views	MOLAP cube server	No second server or copy of the data to manage; marts give dashboard speed (NFR2) while the facts keep full detail.
Python and dbt for ETL	Graphical ETL tools (SSIS, Talend)	Python reads CSV, Excel and JSON natively; dbt keeps every transformation as version-controlled SQL with built-in tests and documentation (NFR7); both are free.
Full reload each run	Incremental loads	At this volume a full reload takes about a minute and makes every run reproducible (NFR6); incremental loads are future work.
Web dashboards on XAMPP with a PHP back-end	Power BI, Tableau	No licences; runs on the course's XAMPP stack; reads through a read-only login (NFR3). Power BI or Excel can still connect to the same marts.

Schemas as layers.

Table 14 shows how the layers of the architecture map to PostgreSQL schemas.

Table 14: Physical layers as PostgreSQL schemas

Data moves down through the layers; business users only read the last ones.

Schema	Contents	Built by	Materialization
`raw`	source data as received (text)	Python	tables, truncated and reloaded
`ref`, `meta`	master data, lineage, defect manifest	Python, dbt seeds	tables
`staging`	typed, conformed, quality-tagged data	dbt	tables
`dq`	rules, issue log, scorecard, quarantine, reconciliation	dbt	tables
`dw`	dimensions and facts	dbt	tables with PK/FK constraints and row-level security
`mart`	dashboard aggregates, CUBE, GROUPING SETS	dbt	materialized views
`secure`, `governance`	region-scoped views, role mapping	dbt, migration	views

Keys and data types.

- Surrogate keys are `integer`s generated in dbt (`row_number()` over the natural key), so facts join on small integers. `dim_date` uses a smart key, `YYYYMMDD`.
- Natural keys are kept as attributes (`psgc_code`, `event_code`, `ec_code`, `org_code`, `item_code`) for traceability.
- Every dimension has a **primary key**; every fact has **foreign keys** to its dimensions (38 in total), declared as real constraints so ERD tools draw the schema and the database enforces referential integrity.
- Measures use `numeric` (exact money and counts) or `integer`; elapsed times are `numeric(…, 2)` hours; flags are `boolean`.
- Degenerate dimensions stay on the fact (`sitrep_no`, `distribution_id` + `line_no`, `request_id`, `priority`).

Performance. Dashboards read the marts (materialized views), not the facts: about 60–80× faster for the event summary. At the current volume every analytical query runs in well under 50 ms with sequential scans (see the Query Performance Test Report (Chapter 8)). The scale-up path is B-tree indexes on the most-filtered fact foreign keys and partitioning facts by event year.

Security objects. Row-level security policies on all six facts, grants declared in dbt (re-applied on every build), and the `governance.my_regions` view (see Chapter 6).

ER diagrams

Generated from the database catalog by `docs/build/diagrams.py`, so they always match the deployed schema.

The six stars follow one by one, starting with Figure 2, the damage assessment star.

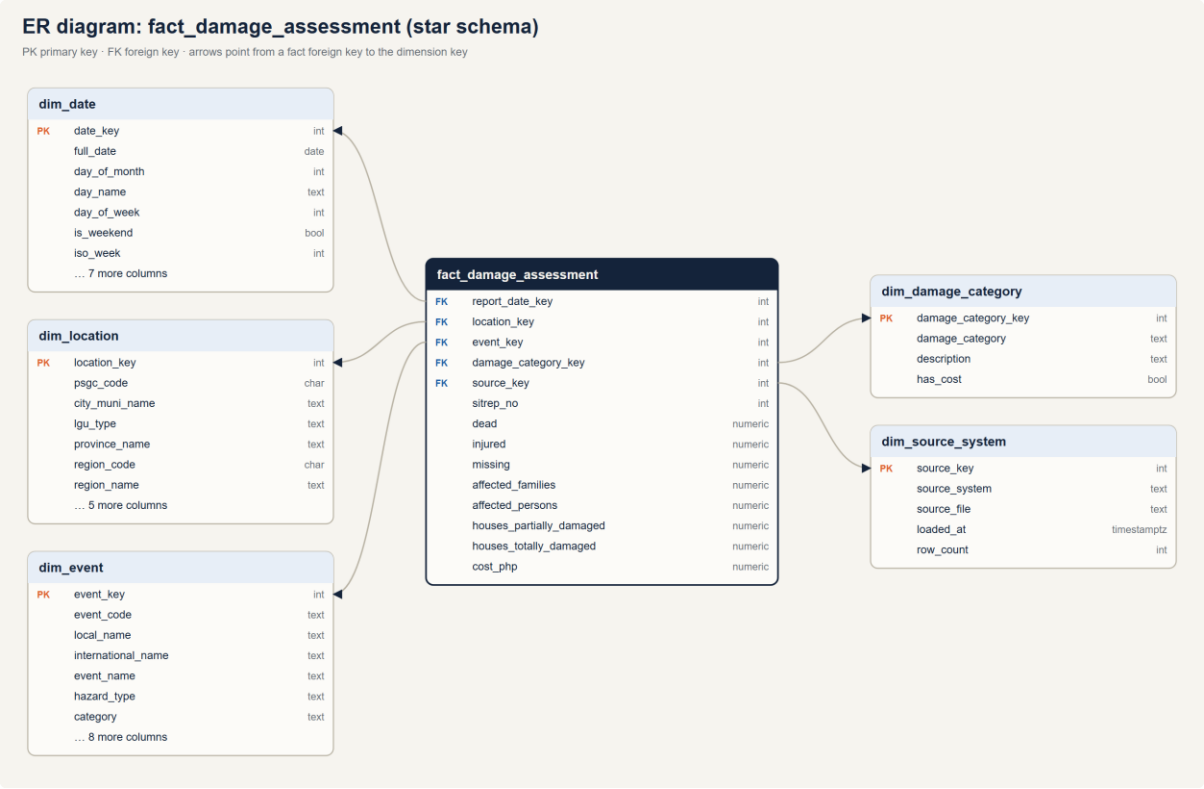


Figure 2: ER diagram of the damage assessment star

The transaction fact in the centre holds casualties, houses damaged and cost per event, city/municipality and damage category; each line is a foreign key to a dimension.

Figure 3 is the evacuation snapshot star.

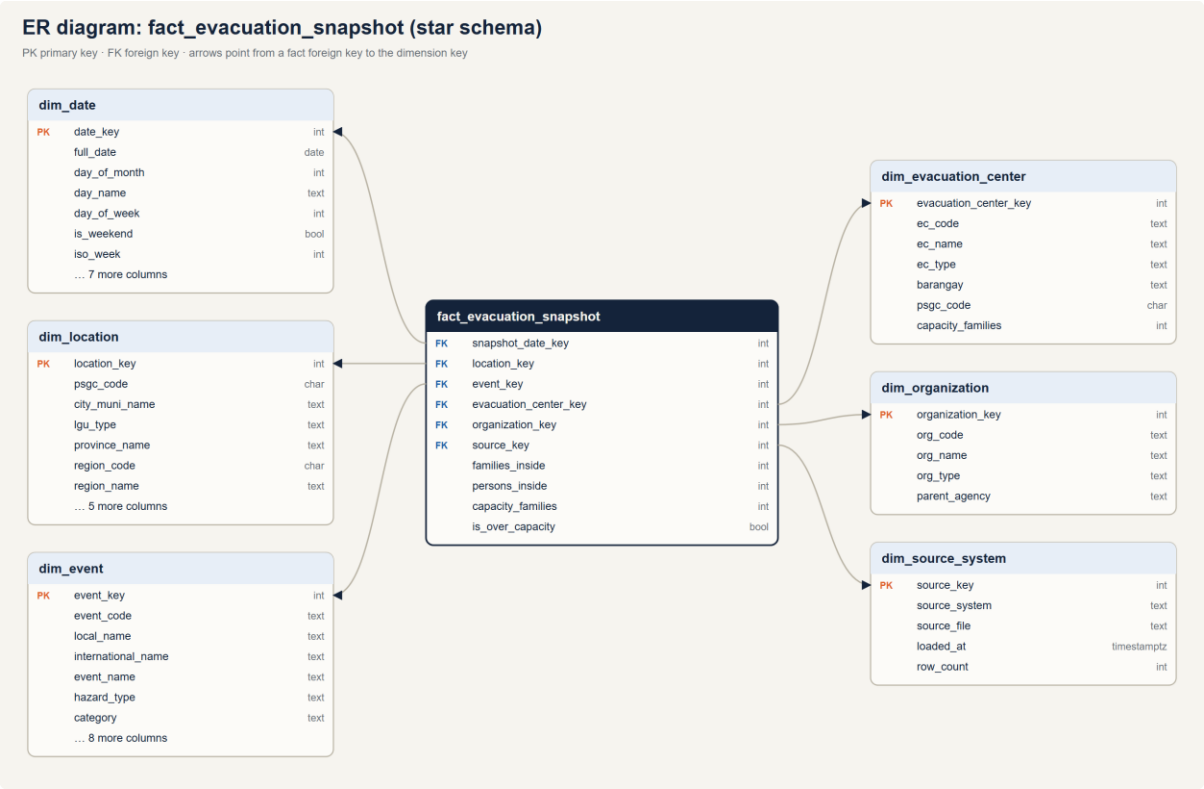


Figure 3: ER diagram of the evacuation snapshot star

One row per evacuation center per day per event; its measures add up across centers but not across days.

Figure 4 is the resource distribution star.

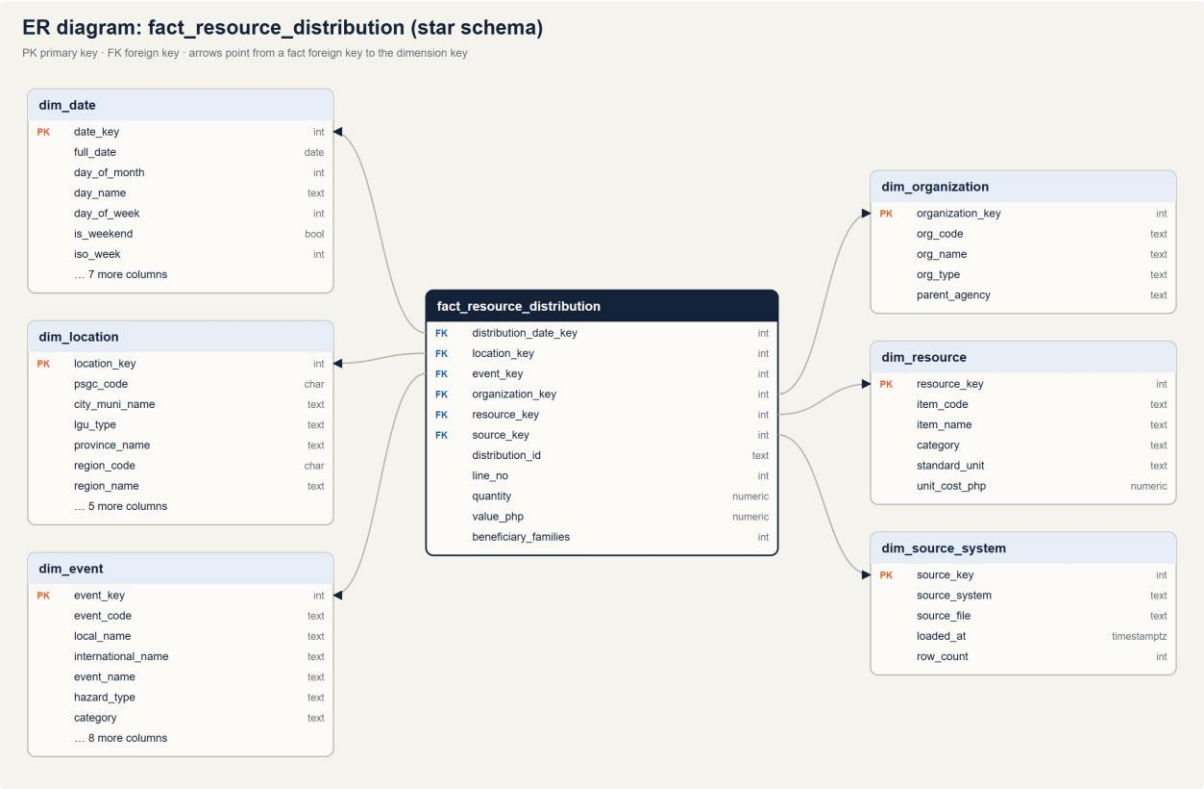


Figure 4: ER diagram of the resource distribution star

One row per relief item line; the organization and resource dimensions let relief be analyzed by provider and category.

Figure 5 is the response action star.

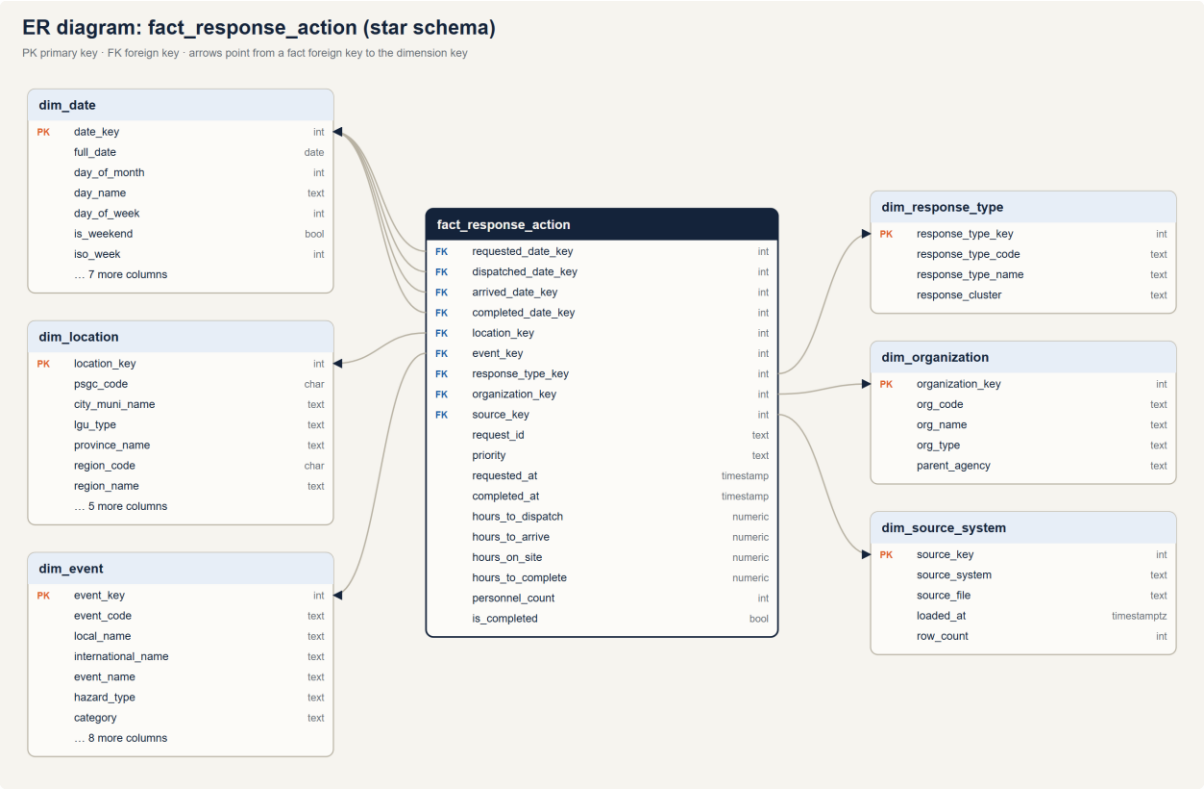


Figure 5: ER diagram of the response action star

An accumulating snapshot: its four date keys (request, dispatch, arrival, completion) each play the role of the date dimension.

Figure 6 is the star for families displaced outside evacuation centers.

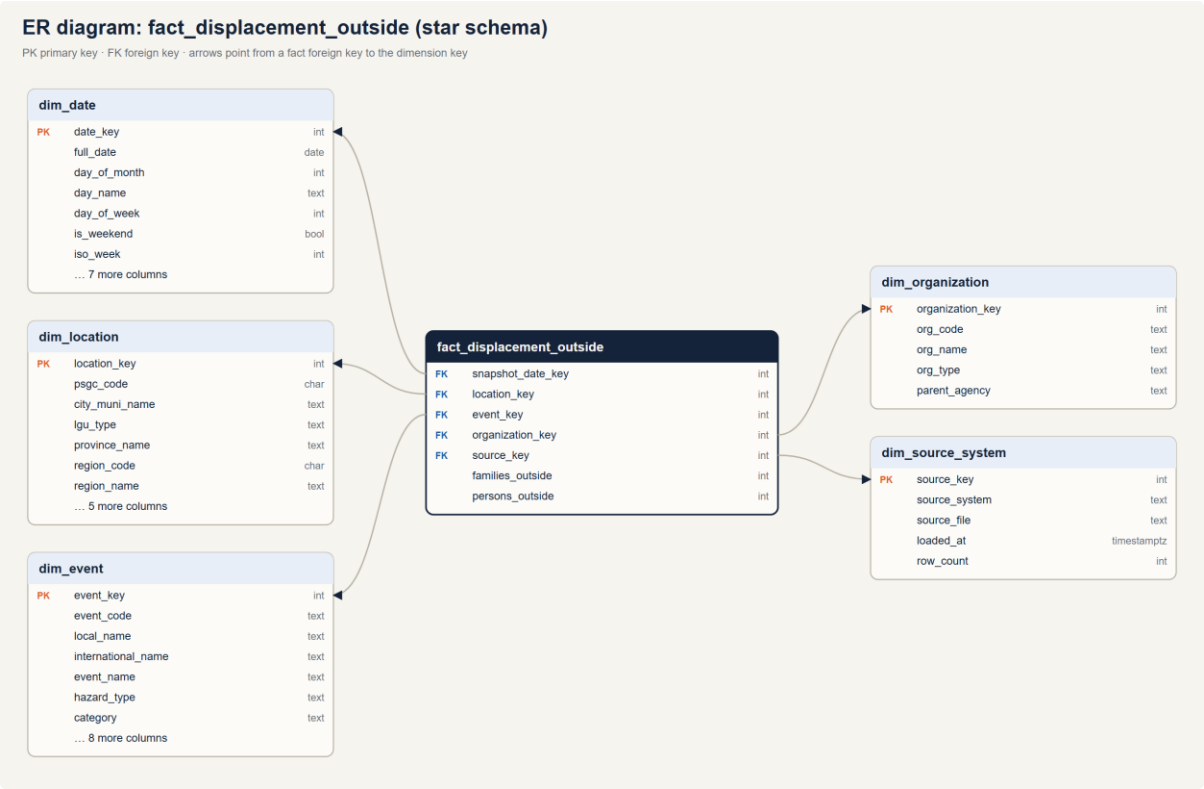


Figure 6: ER diagram of the displacement outside evacuation centers star

Families staying with relatives or friends, per city/municipality and day; added to the evacuation snapshot for the same day it gives total displacement.

Figure 7 is the lifeline disruption star.

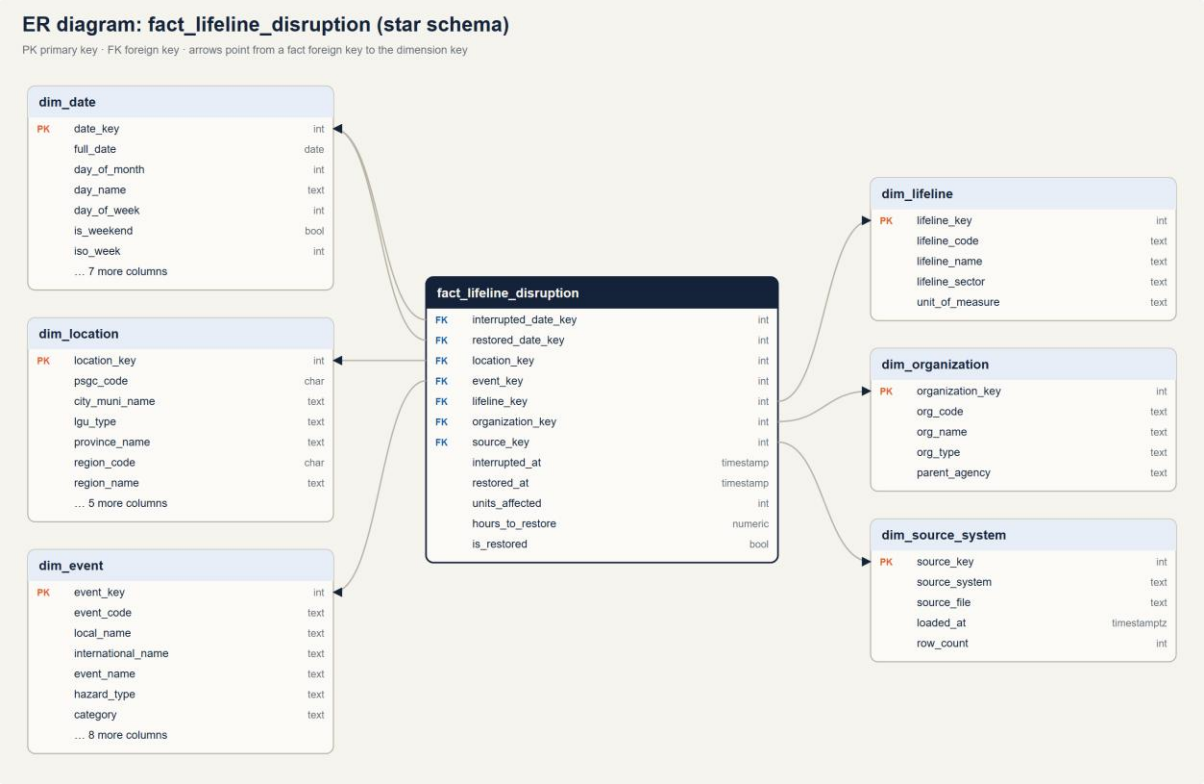


Figure 7: ER diagram of the lifeline disruption star

One row per lifeline that went down in a city/municipality during an event; the interrupted and restored dates both play the role of the date dimension.

2.4 DRIW Dimensional Model

Star schema in the `dw` schema, built by dbt from the cleansed staging layer (only rows with `dq_status` valid or warning). All facts share conformed dimensions, so damage, evacuation, relief and response can be analyzed side by side for the same event and place.

Fact tables

Table 15 gives the grain and current row count of each fact table.

Table 15: Fact tables with grain and row counts

The row counts are the volume each star holds after data-quality filtering (seed 271).

Fact	Type	Grain (one row per…)	Rows*	Measures
`fact_damage_assessment`	Transaction	event × city/municipality × damage category	8,806	dead, injured, missing, affected families/persons, houses partially/totally damaged, cost (PHP)
`fact_evacuation_snapshot`	Periodic snapshot	evacuation center × day × event	31,110	families inside, persons inside, capacity, over-capacity flag
`fact_resource_distribution`	Transaction	distribution line item	14,927	quantity (standard unit), value (PHP), beneficiary families
`fact_response_action`	Accumulating snapshot	response request	32,347	hours to dispatch / arrive / on site / complete, personnel, completed flag
`fact_displacement_outside`	Periodic snapshot	city/municipality × day × event	21,896	families and persons displaced outside evacuation centers (with relatives or friends)
`fact_lifeline_disruption`	Accumulating snapshot	event × city/municipality × lifeline	2,677	units affected, hours to restore, restored flag

* Seed 271.

Design decisions

- **Damage uses the latest sitrep only.** NDRRMC situation reports are cumulative, so summing every sitrep would double-count. The fact keeps the latest *valid* sitrep per event × place × category.
- **Evacuation grain includes the event.** Overlapping events (Nika, Ofel and Pepito, Nov 2024) can fill the same center on the same day.
- **Evacuation measures are semi-additive.** Sum across centers or places for one day; across days use the peak (MAX of daily totals) or the average, never the sum. The same holds for displacement outside centers; total displacement = inside + outside on the same day.
- **Displacement outside centers is its own fact.** DSWD DROMIC reports families staying with relatives or friends per city/municipality, not per center, so it cannot share the evacuation-center grain. Usually more people are displaced outside centers than inside.
- **Lifelines are an accumulating snapshot.** A disruption opens when a road, bridge, power, water or communication line goes down and closes when service is restored (`restored_date_key = -1` while still down).

- **Rates per 100,000 (Sendai Framework targets A and B).** `dim_location.population` is the denominator. The PSGC API has no census figures, so it is an estimate (barangay count × 2,600, labelled in `population_source`); rates compare events with each other rather than give official figures. Loading the PSA workbook (`load_psgc.py`) swaps in census population.
- **Cost of assistance by provider.** LGU relief (MDRRMO/CDRRMO/PDRRMO) arrives through the same distribution feed, so relief splits by `dim_organization.org_type`: national government (DSWD), LGU, NGO and international, as DROMIC reports it.
- **Header measures stay additive.** `beneficiary_families` belongs to a whole distribution, so it is stored on the record's first valid line only.
- **Open milestones.** A response request not yet completed has `completed_date_key = -1` ("Not yet occurred"), so foreign keys are never null; a later load fills it in.
- **Degenerate dimensions:** `sitrep_no`, `distribution_id` + `line_no`, `request_id`, `priority`.

Dimensions

Table 16 describes each dimension and its size.

Table 16: Dimension tables with natural keys and row counts

Location is the only Type 2 dimension; the date dimension includes the member −1 for milestones not yet reached.

Dimension	Natural key	Rows	Notes
`dim_date`	calendar date	2,923	2019–2026 + member −1. Month, quarter, year, weekend, typhoon-season flag. Role-plays four times in `fact_response_action`.
`dim_location`	PSGC code	1,671	Region → province → city/municipality, population (type 1). **SCD Type 2** (see the next subsection).
`dim_event`	event code (`TC2024-KRISTINE`)	31	Local + international name, hazard type, category and rank, max wind, max signal, dates, duration.
`dim_organization`	org code	13	Government, NGO, international, LGU; source name variants resolved via aliases.
`dim_resource`	item code	10	Category (Food, Water, Non-Food, Medical, Shelter), standard unit, unit cost.
`dim_evacuation_center`	EC code	1,751	Type, barangay, capacity, PSGC of its city/municipality. SCD Type 1.
`dim_damage_category`	category	4	Population, Housing, Infrastructure, Agriculture.
`dim_response_type`	type code	5	Response type and cluster (life-saving, logistics, access).

Dimension	Natural key	Rows	Notes
dim_lifeline	lifeline code	5	Roads, bridges, power, water, communications; sector and unit of measure.
dim_source_system	batch id	per load	Lineage: source system and file each fact row came from.

SCD Type 2: dim_location

Provinces change. On 17 September 2022 a plebiscite ratified **RA 11550** (Republic Act No. 11550, 2021), splitting Maguindanao into *Maguindanao del Norte* and *Maguindanao del Sur*. The change log `dbt/seeds/psgc_changes.csv` records each affected city/municipality with its new province and effective date (2022-09-18). `dim_location` turns it into versions.

Table 17 shows the two versions of one affected town.

Table 17: SCD Type 2 versions of Parang, Maguindanao

The same PSGC code has two versions; a fact joins to the one valid on its own date, so history is reported under the provinces of the time.

psgc_code	city_muni_name	province_name	valid_from	valid_to	is_current
1903811000	Parang	Maguindanao	1900-01-01	2022-09-17	false
1903811000	Parang	Maguindanao del Norte	2022-09-18	9999-12-31	true

Facts join to the version **valid on the fact's own date**, so Typhoon Paeng (Oct 2022) reports under the new provinces, while any earlier event would still roll up to "Maguindanao". New PSGC releases are handled by appending rows to the change log.

Bus matrix

Table 18 is the bus matrix: which dimensions each fact uses.

Table 18: Bus matrix of facts and conformed dimensions

A tick means the fact uses the dimension. Dimensions ticked for several facts (date, location, event, organization, source) are the conformed dimensions that integrate the stars.

Fact \ Dimension	Date	Location	Event	Organization	Resource	Evac Center	Damage Cat	Response Type	Lifeline	Source
Damage Assessment	✓	✓	✓				✓			✓
Evacuation Snapshot	✓	✓	✓	✓		✓				✓
Displacement Outside	✓	✓	✓	✓						✓
Resource Distribution	✓	✓	✓	✓	✓					✓
Response Action	✓ ×4	✓	✓	✓				✓		✓
Lifeline Disruption	✓ ×2	✓	✓	✓					✓	✓

Physical design and tests

Every dimension has a primary key and every fact has foreign keys to its dimensions (38 in total), so ERD tools such as pgAdmin or DBeaver draw the star directly. dbt tests check that:

- keys are unique and not null, and each fact has one row per declared grain;
- every row that passed data quality reaches its fact, so inner joins drop nothing silently;
- SCD2 versions never overlap or leave gaps, and exactly one version per place is current;
- Paeng's damage in the former Maguindanao rolls up to the new provinces.

Data marts (mart schema)

Materialized views refreshed by every `dbt build`, each shaped for a dashboard. A dbt test checks that every mart adds back up to the facts it summarizes.

Table 19 lists the eight marts, their grain and the dashboards they serve.

Table 19: Data marts and the dashboards they feed

Each materialized view pre-aggregates one view of the facts, so the dashboards never scan the facts directly.

Mart	Grain	Used by
`mart_event_summary`	event	Executive: headline figures from every fact, deaths and affected per 100,000, peak displaced, relief by provider type, lifeline restoration
`mart_impact_cube`	`GROUP BY CUBE (year, region, hazard)`: every subtotal, labelled by `grouping_level`	Executive: region totals
`mart_province_event_impact`	event × province (SCD2 province at the time)	Logistics: relief vs impact; deaths and affected per 100,000 of the province
`mart_evacuation_daily`	event × day (sum across centers only)	Operations
`mart_response_performance`	`GROUPING SETS` over event, response type, priority (`All` = rolled up)	Operations
`mart_relief_by_org_category`	event × organization × resource category	Logistics (incl. cost of assistance by provider)
`mart_displacement_daily`	event × day: inside + outside evacuation centers	Operations
`mart_lifeline_restoration`	event × lifeline: places disrupted, restored within 72 h, median / p90 hours	Operations

The dashboards read these marts live through the PHP API on XAMPP (`/driw/api/marts.php`), as the read-only role `driw_web`. A JSON snapshot of the marts (`ingestion/export_marts.py` → `web/src/data/`) is bundled as a fallback for when the warehouse is unreachable.

Example: OLAP with GROUP BY CUBE

```
select d.year_number, r.category, sum(f.value_php) as relief_value_php
from dw.fact_resource_distribution f
join dw.dim_date d on d.date_key = f.distribution_date_key
join dw.dim_resource r using (resource_key)
group by cube (d.year_number, r.category);      -- every subtotal plus the grand total
```

3. Data Dictionary and Metadata

Generated from the live PostgreSQL catalog by `docs/build/data_dictionary.py` on 2026-10-25 03:30. Names, types, keys and nullability come from the database; descriptions are curated. Row counts are from the current build (seed 271).

Last successful load: 2026-10-24 03:37.

3.1 Schemas (layers)

Table 20 lists the schemas, one per layer.

Table 20: Database schemas and the layer each one holds

Each schema is one layer of the architecture, from raw extracts to the marts the dashboards read; Objects counts its tables and views.

Schema	Purpose	Objects
`raw`	Landing zone. Source extracts exactly as received, every business column as text, plus lineage columns `_batch_id`, `_source_file`, `_row_number`. Loaded by `ingestion/load_raw.py` (full reload).	7
`ref`	Master and reference data: PSGC geography, canonical organizations and relief items, and the alias maps that conform source spellings to them.	7
`meta`	Pipeline metadata: one row per loaded file (`load_batch`), applied migrations, and the defect manifest written by the synthetic generator.	3
`staging`	Cleansed and conformed source data (dbt). Keeps every raw row and adds `dq_issues` (rules that fired) and `dq_status` (valid, warning, rejected, duplicate).	9
`dq`	Data-quality layer (dbt): rule catalog, issue log, scorecard, quarantine and reconciliation.	5
`dw`	Enterprise star schema (dbt): conformed dimensions and fact tables. Only valid/warning rows load.	16
`mart`	Data marts (dbt materialized views) shaped for the dashboards, including ROLAP aggregates.	8
`secure`	Region-scoped views for regional users; they run with the caller's rights (row-level security).	2

3.2 Lineage

Table 21 traces each source to the warehouse.

Table 21: Lineage from each source file to its warehouse table

Read a row left to right to follow one source through the raw, staging and warehouse layers; the same path can be followed backwards from any figure on a dashboard.

Source system	Source file(s)	Raw table	Staging model	Warehouse fact
NDRRMC situation reports	`ndrrmc/sitreps_<event>.csv`	`raw.ndrrmc_damage_report`	`staging.stg_ndrrmc_damage`	`dw.fact_damage_assessment`
NDRRMC lifeline status	`ndrrmc/lifelines/lifelines_<event>.csv`	`raw.ndrrmc_lifeline_status`	`staging.stg_ndrrmc_lifeline`	`dw.fact_lifeline_disruption`
DSWD DROMIC	`dromic/dromic_<event>.xlsx`	`raw.dromic_evacuation`	`staging.stg_dromic_evacuation`	`dw.fact_evacuation_snapshot`

Source system	Source file(s)	Raw table	Staging model	Warehouse fact
DSWD DROMIC, sheet *Outside ECs*	dromic/dromic_<event>.xlsx	raw.dromic_displaced_outside	staging.stg_dromic_displaced_outside	dw.fact_displacement_outside
NGO / agency partners	ngo/distributions_<year>.json	raw.ngo_distribution	staging.stg_ngo_distribution_line	dw.fact_resource_distribution
PAGASA	pagasa/tc_bulletins.csv	raw.pagasa_event	staging.stg_pagasa_event	dw.dim_event
LGU / MDRRMO	lgu/response_logs.csv	raw.lgu_response_log	staging.stg_lgu_response	dw.fact_response_action
PSA PSGC (community API)	psgc_api/*.json	ref.psgc	staging.stg_location_resolution	dw.dim_location

Every fact row carries `source_key` → `dw.dim_source_system` (the file and batch it came from), and every raw row keeps its `_batch_id`, `_source_file` and `_row_number`.

3.3 dw · Star schema

dw.dim_damage_category

Table 22 lists its columns.

Table 22: Columns of dw.dim_damage_category

Population, Housing, Infrastructure, Agriculture. Table, 4 rows. PK and FK mark the primary and foreign keys; Null says whether a value may be missing.

Column	Type	Key	Null	Description
damage_category_key	integer	PK	no	Surrogate key of dim_damage_category.
damage_category	text		yes	Population, Housing, Infrastructure or Agriculture.
description	text		yes	Description of the category.
has_cost	boolean		yes	Whether the category carries a PHP cost.

dw.dim_date

Table 23 lists its columns.

Table 23: Columns of dw.dim_date

Calendar, 2019–2026, plus member -1 ("Not yet occurred") for open milestones. Type 0. Table, 2,923 rows. PK and FK mark the primary and foreign keys; Null says whether a value may be missing.

Column	Type	Key	Null	Description
date_key	integer	PK	no	Surrogate key of dim_date, YYYYMMDD (-1 = not yet occurred).
full_date	date		yes	Calendar date.
day_of_month	integer		yes	Day of the month, 1–31.
day_name	text		yes	Weekday name.
day_of_week	integer		yes	ISO weekday, 1 (Monday) to 7 (Sunday).
is_weekend	boolean		yes	Saturday or Sunday.
iso_week	integer		yes	ISO week number.
month_number	integer		yes	Month, 1–12.
month_name	text		yes	Month name.
year_month	text		yes	YYYY-MM.
quarter_number	integer		yes	Quarter, 1–4.

Column	Type	Key	Null	Description
quarter_name	text		yes	Q1–Q4.
year_number	integer		yes	Calendar year.
is_typhoon_season	boolean		yes	June–November.

dw.dim_evacuation_center

Table 24 lists its columns.

Table 24: Columns of dw.dim_evacuation_center

Evacuation centers from DROMIC reports. SCD Type 1 (latest attributes). Table, 1,751 rows. PK and FK mark the primary and foreign keys; Null says whether a value may be missing.

Column	Type	Key	Null	Description
evacuation_center_key	integer	PK	no	Surrogate key of dim_evacuation_center.
ec_code	text		yes	Evacuation-center code from DROMIC (natural key).
ec_name	text		yes	Evacuation-center name.
ec_type	text		yes	School, covered court, gymnasium, barangay hall or church.
barangay	text		yes	Barangay where the center is located.
psgc_code	character		yes	Philippine Standard Geographic Code, 10 digits (natural key).
capacity_families	integer		yes	Families the evacuation center can hold.

dw.dim_event

Table 25 lists its columns.

Table 25: Columns of dw.dim_event

Hazard events (tropical cyclones and monsoon floods) from PAGASA bulletins. Table, 31 rows. PK and FK mark the primary and foreign keys; Null says whether a value may be missing.

Column	Type	Key	Null	Description
event_key	integer	PK	no	Surrogate key of dim_event.
event_code	text		yes	Event natural key, TC<year>-<LOCAL NAME> (e.g. TC2024-KRISTINE); 'All' in roll-up rows.
local_name	text		yes	PAGASA local storm name.
international_name	text		yes	International (JMA) storm name.
event_name	text		yes	Local name with international name, e.g. Kristine (Trami).
hazard_type	text		yes	Tropical Cyclone or Flood (southwest monsoon).
category	text		yes	Category: storm category for events, resource category for relief items.
category_rank	integer		yes	Storm category as a number, 1 (tropical depression) to 5 (super typhoon).
max_wind_kph	integer		yes	Maximum sustained winds, km/h.
max_signal	integer		yes	Highest tropical cyclone wind signal raised.
start_date	date		yes	First day of the event.
end_date	date		yes	Last day of the event.
duration_days	integer		yes	Event length in days, inclusive.
event_year	integer		yes	Year the event started.
affected_areas	text		yes	Provinces named in the PAGASA bulletin, as published.

dw.dim_lifeline

Table 26 lists its columns.

Table 26: Columns of dw.dim_lifeline

Lifelines tracked in NDRRMC sitreps: roads, bridges, power, water and communications. Table, 5 rows. PK and FK mark the primary and foreign keys; Null says whether a value may be missing.

Column	Type	Key	Null	Description
lifeline_key	integer	PK	no	Surrogate key of dim_lifeline.
lifeline_code	text		yes	Lifeline code (natural key): ROAD, BRIDGE, POWER, WATER or COMMS.
lifeline_name	text		yes	Lifeline name, e.g. Road sections not passable.
lifeline_sector	text		yes	Transport or Utilities.
unit_of_measure	text		yes	What units_affected counts: road sections, bridges or barangays.

dw.dim_location

Table 27 lists its columns.

Table 27: Columns of dw.dim_location

City/municipality, keyed on PSGC code. SCD Type 2: one row per period of unchanged attributes (valid_from–valid_to). Table, 1,671 rows. PK and FK mark the primary and foreign keys; Null says whether a value may be missing.

Column	Type	Key	Null	Description
location_key	integer	PK	no	Surrogate key of dim_location (the SCD2 version valid on the fact's date).
psgc_code	character		yes	Philippine Standard Geographic Code, 10 digits (natural key).
city_muni_name	text		yes	Official PSGC name of the city or municipality.
lgu_type	text		yes	City or Municipality.
province_name	text		yes	Province in force on the fact's date (e.g. Maguindanao del Sur after Sep 2022).
region_code	character(10)		yes	PSGC region code.
region_name	text		yes	Region name.
population	integer		yes	Residents (estimate: barangay count × 2,600 unless a census figure is loaded).
population_source	text		yes	Source of the population figure (census name or 'Estimate').
valid_from	date		yes	SCD2: first day this version applies.
valid_to	date		yes	SCD2: last day this version applies (9999-12-31 = current).
is_current	boolean		yes	Whether this SCD2 version is the current one.

dw.dim_organization

Table 28 lists its columns.

Table 28: Columns of dw.dim_organization

Responding and reporting organizations (government, NGO, international, LGU). Table, 13 rows. PK and FK mark the primary and foreign keys; Null says whether a value may be missing.

Column	Type	Key	Null	Description
organization_key	integer	PK	no	Surrogate key of dim_organization.
org_code	text		yes	Organization code (natural key).
org_name	text		yes	Organization name.
org_type	text		yes	National Government, NGO, International or LGU.
parent_agency	text		yes	Department the organization belongs to.

dw.dim_resource

Table 29 lists its columns.

Table 29: Columns of dw.dim_resource

Relief items with category, standard unit and unit cost. Table, 10 rows. PK and FK mark the primary and foreign keys; Null says whether a value may be missing.

Column	Type	Key	Null	Description
resource_key	integer	PK	no	Surrogate key of dim_resource.
item_code	text		yes	Canonical relief item code.
item_name	text		yes	Relief item name.
category	text		yes	Category: storm category for events, resource category for relief items.
standard_unit	text		yes	Unit all quantities are converted to (pack, kg, liter, kit, piece).
unit_cost_php	numeric(12,2)		yes	Indicative unit cost, PHP.

dw.dim_response_type

Table 30 lists its columns.

Table 30: Columns of dw.dim_response_type

Response request types and their cluster. Table, 5 rows. PK and FK mark the primary and foreign keys; Null says whether a value may be missing.

Column	Type	Key	Null	Description
response_type_key	integer	PK	no	Surrogate key of dim_response_type.
response_type_code	text		yes	Response type code, e.g. RESCUE.
response_type_name	text		yes	Response type name.
response_cluster	text		yes	Life-saving, Logistics, or Access and infrastructure.

dw.dim_source_system

Table 31 lists its columns.

Table 31: Columns of dw.dim_source_system

Lineage: one row per loaded source file (batch). Table, 1,099 rows. PK and FK mark the primary and foreign keys; Null says whether a value may be missing.

Column	Type	Key	Null	Description
source_key	integer	PK	no	Surrogate key of dim_source_system (lineage to the loaded file).
source_system	text		yes	NDRRMC, DSWD_DROMIC, NGO, PAGASA or LGU.
source_file	text		yes	Source file name.
loaded_at	timestamp with time zone		yes	When the batch was loaded.
row_count	integer		yes	Rows in the source file.

dw.fact_damage_assessment

Table 32 lists its columns.

Table 32: Columns of dw.fact_damage_assessment

Grain: event × city/municipality × damage category (latest valid sitrep). Transaction fact. Table, 8,806 rows. PK and FK mark the primary and foreign keys; Null says whether a value may be missing.

Column	Type	Key	Null	Description
report_date_key	integer	FK → dim_date	yes	Date of the sitrep the figures come from (dim_date).
location_key	integer	FK → dim_location	yes	Surrogate key of dim_location (the SCD2 version valid on the fact's date).
event_key	integer	FK → dim_event	yes	Surrogate key of dim_event.
damage_category_key	integer	FK → dim_damage_category	yes	Surrogate key of dim_damage_category.
source_key	integer	FK → dim_source_system	yes	Surrogate key of dim_source_system (lineage to the loaded file).
sitrep_no	integer		yes	NDRRMC situation-report number (degenerate dimension).
dead	numeric		yes	Deaths.
injured	numeric		yes	Injured persons.
missing	numeric		yes	Missing persons.
affected_families	numeric		yes	Families affected (NDRRMC).
affected_persons	numeric		yes	Persons affected (NDRRMC).
houses_partially_damaged	numeric		yes	Houses partially damaged.
houses_totally_damaged	numeric		yes	Houses totally destroyed.
cost_php	numeric		yes	Damage cost, PHP (Housing, Infrastructure, Agriculture rows).

dw.fact_displacement_outside

Table 33 lists its columns.

Table 33: Columns of dw.fact_displacement_outside

Grain: city/municipality × day × event. Periodic snapshot of families displaced outside evacuation centers (with relatives or friends); semi-additive. Table, 21,896 rows. PK and FK mark the primary and foreign keys; Null says whether a value may be missing.

Column	Type	Key	Null	Description
snapshot_date_key	integer	FK → dim_date	yes	Date of the snapshot (dim_date).
location_key	integer	FK → dim_location	yes	Surrogate key of dim_location (the SCD2 version valid on the fact's date).
event_key	integer	FK → dim_event	yes	Surrogate key of dim_event.
organization_key	integer	FK → dim_organization	yes	Surrogate key of dim_organization.
source_key	integer	FK → dim_source_system	yes	Surrogate key of dim_source_system (lineage to the loaded file).
families_outside	integer		yes	Families displaced outside evacuation centers (semi-additive).
persons_outside	integer		yes	Persons displaced outside evacuation centers (semi-additive).

dw.fact_evacuation_snapshot

Table 34 lists its columns.

Table 34: Columns of dw.fact_evacuation_snapshot

Grain: evacuation center × day × event. Periodic snapshot; occupancy measures are semi-additive (never sum across days). Table, 31,110 rows. PK and FK mark the primary and foreign keys; Null says whether a value may be missing.

Column	Type	Key	Null	Description
snapshot_date_key	integer	FK → dim_date	yes	Date of the snapshot (dim_date).
location_key	integer	FK → dim_location	yes	Surrogate key of dim_location (the SCD2 version valid on the fact's date).
event_key	integer	FK → dim_event	yes	Surrogate key of dim_event.
evacuation_center_key	integer	FK → dim_evacuation_center	yes	Surrogate key of dim_evacuation_center.
organization_key	integer	FK → dim_organization	yes	Surrogate key of dim_organization.
source_key	integer	FK → dim_source_system	yes	Surrogate key of dim_source_system (lineage to the loaded file).
families_inside	integer		yes	Families inside the center (semi-additive).
persons_inside	integer		yes	Persons inside the center (semi-additive).
capacity_families	integer		yes	Families the evacuation center can hold.
is_over_capacity	boolean		yes	Families inside exceeded the center's capacity.

dw.fact_lifeline_disruption

Table 35 lists its columns.

Table 35: Columns of dw.fact_lifeline_disruption

Grain: event × city/municipality × lifeline. Accumulating snapshot: opens when the lifeline goes down, closes when service is restored. Table, 2,677 rows. PK and FK mark the primary and foreign keys; Null says whether a value may be missing.

Column	Type	Key	Null	Description
interrupted_date_key	integer	FK → dim_date	yes	Date the lifeline went down (role-playing dim_date).
restored_date_key	integer	FK → dim_date	yes	Date service was restored (role-playing dim_date; -1 = not yet).
location_key	integer	FK → dim_location	yes	Surrogate key of dim_location (the SCD2 version valid on the fact's date).
event_key	integer	FK → dim_event	yes	Surrogate key of dim_event.
lifeline_key	integer	FK → dim_lifeline	yes	Surrogate key of dim_lifeline.
organization_key	integer	FK → dim_organization	yes	Surrogate key of dim_organization.
source_key	integer	FK → dim_source_system	yes	Surrogate key of dim_source_system (lineage to the loaded file).
interrupted_at	timestamp without time zone		yes	Timestamp the lifeline went down.
restored_at	timestamp without time zone		yes	Timestamp service was restored (null while still down).
units_affected	integer		yes	Road sections, bridges or barangays affected.
hours_to_restore	numeric		yes	Hours from interruption to restoration (null while still down).
is_restored	boolean		yes	Whether service has been restored.

dw.fact_resource_distribution

Table 36 lists its columns.

Table 36: Columns of dw.fact_resource_distribution

Grain: one relief item in one distribution. Transaction fact; quantities in standard units. Table, 14,927 rows. PK and FK mark the primary and foreign keys; Null says whether a value may be missing.

Column	Type	Key	Null	Description
distribution_date_key	integer	FK → dim_date	yes	Date of the distribution (dim_date).
location_key	integer	FK → dim_location	yes	Surrogate key of dim_location (the SCD2 version valid on the fact's date).
event_key	integer	FK → dim_event	yes	Surrogate key of dim_event.
organization_key	integer	FK → dim_organization	yes	Surrogate key of dim_organization.
resource_key	integer	FK → dim_resource	yes	Surrogate key of dim_resource.

Column	Type	Key	Null	Description
source_key	integer	FK → dim_source_system	yes	Surrogate key of dim_source_system (lineage to the loaded file).
distribution_id	text		yes	Source distribution identifier (degenerate dimension).
line_no	integer		yes	Line number within the distribution.
quantity	numeric		yes	Quantity in the item's standard unit.
value_php	numeric		yes	Value of the line, PHP.
beneficiary_families	integer		yes	Families served by the whole distribution; stored on its first line only so it stays additive.

dw.fact_response_action

Table 37 lists its columns.

Table 37: Columns of dw.fact_response_action

Grain: one response request. Accumulating snapshot with four milestone dates. Table, 32,347 rows. PK and FK mark the primary and foreign keys; Null says whether a value may be missing.

Column	Type	Key	Null	Description
requested_date_key	integer	FK → dim_date	yes	Date of the request (role-playing dim_date).
dispatched_date_key	integer	FK → dim_date	yes	Date a unit was dispatched (role-playing dim_date).
arrived_date_key	integer	FK → dim_date	yes	Date the responding unit arrived (role-playing dim_date; -1 = not yet).
completed_date_key	integer	FK → dim_date	yes	Date the request was completed (role-playing dim_date; -1 = not yet).
location_key	integer	FK → dim_location	yes	Surrogate key of dim_location (the SCD2 version valid on the fact's date).
event_key	integer	FK → dim_event	yes	Surrogate key of dim_event.
response_type_key	integer	FK → dim_response_type	yes	Surrogate key of dim_response_type.
organization_key	integer	FK → dim_organization	yes	Surrogate key of dim_organization.
source_key	integer	FK → dim_source_system	yes	Surrogate key of dim_source_system (lineage to the loaded file).
request_id	text		yes	Source request identifier (degenerate dimension).
priority	text		yes	P1 critical, P2 urgent, P3 routine ('All' in roll-up rows).
requested_at	timestamp without time zone		yes	Timestamp of the request.
completed_at	timestamp without time zone		yes	Timestamp the request was completed (null while open).
hours_to_dispatch	numeric		yes	Hours from request to dispatch.
hours_to_arrive	numeric		yes	Hours from dispatch to arrival.
hours_on_site	numeric		yes	Hours from arrival to completion.

Column	Type	Key	Null	Description
hours_to_complete	numeric		yes	Hours from request to completion.
personnel_count	integer		yes	Personnel sent on the request.
is_completed	boolean		yes	Whether the request has been completed.

3.4 mart · Data marts

mart.mart_displacement_daily

Table 38 lists its columns.

Table 38: Columns of mart.mart_displacement_daily

Everyone displaced per event per day: inside evacuation centers plus outside. Materialized view, 509 rows. PK and FK mark the primary and foreign keys; Null says whether a value may be missing.

Column	Type	Key	Null	Description
event_code	text		yes	Event natural key, TC<year>-<LOCAL NAME> (e.g. TC2024-KRISTINE); 'All' in roll-up rows.
snapshot_date	date		yes	Date of the daily snapshot.
day_of_event	integer		yes	Days since the event started (negative = pre-emptive evacuation).
families_inside	bigint		yes	Families inside the center (semi-additive).
persons_inside	bigint		yes	Persons inside the center (semi-additive).
families_outside	bigint		yes	Families displaced outside evacuation centers (semi-additive).
persons_outside	bigint		yes	Persons displaced outside evacuation centers (semi-additive).
persons_displaced	bigint		yes	Persons inside plus outside evacuation centers that day.
share_outside_centers	numeric		yes	Share of displaced persons staying outside centers, 0–1.

mart.mart_evacuation_daily

Table 39 lists its columns.

Table 39: Columns of mart.mart_evacuation_daily

Evacuation-center load per event per day (summed across centers only). Materialized view, 372 rows. PK and FK mark the primary and foreign keys; Null says whether a value may be missing.

Column	Type	Key	Null	Description
event_code	text		yes	Event natural key, TC<year>-<LOCAL NAME> (e.g. TC2024-KRISTINE); 'All' in roll-up rows.
snapshot_date	date		yes	Date of the daily snapshot.
day_of_event	integer		yes	Days since the event started (negative = pre-emptive evacuation).
centers_open	bigint		yes	Evacuation centers with evacuees that day.
families_inside	bigint		yes	Families inside the center (semi-additive).
persons_inside	bigint		yes	Persons inside the center (semi-additive).
capacity_families	bigint		yes	Families the evacuation center can hold.
centers_over_capacity	bigint		yes	Centers where families inside exceeded capacity that day.

Column	Type	Key	Null	Description
occupancy_rate	numeric		yes	Families inside divided by capacity, across open centers.

mart.mart_event_summary

Table 40 lists its columns.

Table 40: Columns of mart.mart_event_summary

One row per event: headline damage, evacuation, relief and response figures. Materialized view, 31 rows. PK and FK mark the primary and foreign keys; Null says whether a value may be missing.

Column	Type	Key	Null	Description
event_code	text		yes	Event natural key, TC<year>-<LOCAL NAME> (e.g. TC2024-KRISTINE); 'All' in roll-up rows.
event_name	text		yes	Local name with international name, e.g. Kristine (Trami).
local_name	text		yes	PAGASA local storm name.
international_name	text		yes	International (JMA) storm name.
hazard_type	text		yes	Tropical Cyclone or Flood (southwest monsoon).
category	text		yes	Category: storm category for events, resource category for relief items.
category_rank	integer		yes	Storm category as a number, 1 (tropical depression) to 5 (super typhoon).
max_signal	integer		yes	Highest tropical cyclone wind signal raised.
start_date	date		yes	First day of the event.
end_date	date		yes	Last day of the event.
event_year	integer		yes	Year the event started.
dead	numeric		yes	Deaths.
injured	numeric		yes	Injured persons.
missing	numeric		yes	Missing persons.
affected_families	numeric		yes	Families affected (NDRRMC).
affected_persons	numeric		yes	Persons affected (NDRRMC).
houses_partially_damaged	numeric		yes	Houses partially damaged.
houses_totally_damaged	numeric		yes	Houses totally destroyed.
damage_cost_php	numeric		yes	Total damage cost, PHP.
housing_cost_php	numeric		yes	Housing damage, PHP.
infrastructure_cost_php	numeric		yes	Infrastructure damage, PHP.
agriculture_cost_php	numeric		yes	Agricultural damage, PHP.
cities_municipalities_affected	bigint		yes	Distinct cities/municipalities with reported damage.
provinces_affected	bigint		yes	Distinct provinces with reported damage.
affected_area_population	bigint		yes	Population of the cities/municipalities that reported damage (denominator of the per-100k rates).
dead_per_100k	numeric		yes	Deaths per 100,000 population (Sendai target A); event: affected places, province: whole province.
missing_per_100k	numeric		yes	Missing persons per 100,000 population of the affected places.
affected_per_100k	numeric		yes	Persons affected per 100,000 population of the affected places (Sendai target B).
dead_per_100k_national	numeric		yes	Deaths per 100,000 of the national population.

Column	Type	Key	Null	Description
peak_persons_in_centers	bigint		yes	Highest daily total of persons in centers (sum across centers, max across days).
peak_families_in_centers	bigint		yes	Highest daily total of families in centers.
days_with_evacuees	bigint		yes	Days on which any center held evacuees.
centers_used	bigint		yes	Distinct evacuation centers used during the event.
over_capacity_center_days	bigint		yes	Center-days in which a center was over capacity.
peak_persons_outside_centers	bigint		yes	Highest daily total of persons displaced outside centers.
peak_persons_displaced	bigint		yes	Highest daily total of persons displaced, inside plus outside centers.
relief_value_php	numeric		yes	Relief value, PHP (standard quantity × unit cost).
relief_national_government_php	numeric		yes	Relief value provided by national government (DSWD), PHP.
relief_lgu_php	numeric		yes	Relief value provided by local government units, PHP.
relief_ngo_php	numeric		yes	Relief value provided by NGOs, PHP.
relief_international_php	numeric		yes	Relief value provided by international organizations (UN), PHP.
families_served	bigint		yes	Sum of beneficiary families over distributions (a family can be served more than once).
distributions	bigint		yes	Number of distinct distributions.
relief_php_per_affected_person	numeric		yes	Relief value divided by persons affected, PHP.
response_requests	bigint		yes	Number of response requests.
median_hours_to_dispatch	numeric		yes	Median hours from request to dispatch.
median_hours_to_arrive	numeric		yes	Median hours from dispatch to arrival.
completion_rate	numeric		yes	Share of requests completed, 0–1.
lifeline_disruptions	bigint		yes	Places × lifelines that went down during the event.
lifelines_still_down	bigint		yes	Disruptions not yet restored at the latest sitrep.
median_hours_to_restore	numeric		yes	Median hours from interruption to restoration.

mart.mart_impact_cube

Table 41 lists its columns.

Table 41: Columns of mart.mart_impact_cube

GROUP BY CUBE over year × region × hazard type: every subtotal, labelled by grouping_level. Materialized view, 160 rows. PK and FK mark the primary and foreign keys; Null says whether a value may be missing.

Column	Type	Key	Null	Description
grouping_level	text		yes	Which dimensions a CUBE row is broken down by: year, region, hazard, combinations, or grand_total.
event_year	integer		yes	Year the event started.
region_name	text		yes	Region name.
hazard_type	text		yes	Tropical Cyclone or Flood (southwest monsoon).
events	bigint		yes	Number of distinct events.
dead	numeric		yes	Deaths.
affected_persons	numeric		yes	Persons affected (NDRRMC).

Column	Type	Key	Null	Description
houses_totally_damaged	numeric		yes	Houses totally destroyed.
damage_cost_php	numeric		yes	Total damage cost, PHP.

mart.mart_lifeline_restoration

Table 42 lists its columns.

Table 42: Columns of mart.mart_lifeline_restoration

Per event × lifeline: places disrupted, restored, still down, and hours to restore. Materialized view, 137 rows. PK and FK mark the primary and foreign keys; Null says whether a value may be missing.

Column	Type	Key	Null	Description
event_code	text		yes	Event natural key, TC<year>-<LOCAL NAME> (e.g. TC2024-KRISTINE); 'All' in roll-up rows.
lifeline_code	text		yes	Lifeline code (natural key): ROAD, BRIDGE, POWER, WATER or COMMS.
lifeline_name	text		yes	Lifeline name, e.g. Road sections not passable.
lifeline_sector	text		yes	Transport or Utilities.
unit_of_measure	text		yes	What units_affected counts: road sections, bridges or barangays.
places_disrupted	bigint		yes	Cities/municipalities where the lifeline went down.
units_affected	bigint		yes	Road sections, bridges or barangays affected.
places_restored	bigint		yes	Of those, places where service was restored.
places_still_down	bigint		yes	Places still without the lifeline at the latest sitrep.
restored_within_72h	bigint		yes	Places where service came back within 72 hours.
share_restored_within_72h	numeric		yes	Share of disrupted places restored within 72 hours, 0–1.
median_hours_to_restore	numeric		yes	Median hours from interruption to restoration.
p90_hours_to_restore	numeric		yes	90th-percentile hours from interruption to restoration.

mart.mart_province_event_impact

Table 43 lists its columns.

Table 43: Columns of mart.mart_province_event_impact

Event × province impact next to relief delivered (coverage). Materialized view, 168 rows. PK and FK mark the primary and foreign keys; Null says whether a value may be missing.

Column	Type	Key	Null	Description
event_code	text		yes	Event natural key, TC<year>-<LOCAL NAME> (e.g. TC2024-KRISTINE); 'All' in roll-up rows.
event_name	text		yes	Local name with international name, e.g. Kristine (Trami).
event_year	integer		yes	Year the event started.
region_name	text		yes	Region name.
province_name	text		yes	Province in force on the fact's date (e.g. Maguindanao del Sur after Sep 2022).

Column	Type	Key	Null	Description
dead	numeric		yes	Deaths.
affected_persons	numeric		yes	Persons affected (NDRRMC).
affected_families	numeric		yes	Families affected (NDRRMC).
houses_totally_damaged	numeric		yes	Houses totally destroyed.
damage_cost_php	numeric		yes	Total damage cost, PHP.
province_population	bigint		yes	Population of the whole province at the time of the event.
dead_per_100k	numeric		yes	Deaths per 100,000 population (Sendai target A); event: affected places, province: whole province.
affected_per_100k	numeric		yes	Persons affected per 100,000 population of the affected places (Sendai target B).
relief_value_php	numeric		yes	Relief value, PHP (standard quantity × unit cost).
families_served	bigint		yes	Sum of beneficiary families over distributions (a family can be served more than once).
relief_php_per_affected_person	numeric		yes	Relief value divided by persons affected, PHP.
family_coverage_ratio	numeric		yes	Family servings divided by affected families.

mart.mart_relief_by_org_category

Table 44 lists its columns.

Table 44: Columns of mart.mart_relief_by_org_category

Relief delivered per event × organization × resource category. Materialized view, 851 rows. PK and FK mark the primary and foreign keys; Null says whether a value may be missing.

Column	Type	Key	Null	Description
event_code	text		yes	Event natural key, TC<year>-<LOCAL NAME> (e.g. TC2024-KRISTINE); 'All' in roll-up rows.
org_code	text		yes	Organization code (natural key).
org_name	text		yes	Organization name.
org_type	text		yes	National Government, NGO, International or LGU.
resource_category	text		yes	Food, Water, Non-Food, Medical or Shelter.
distributions	bigint		yes	Number of distinct distributions.
line_items	bigint		yes	Number of distribution lines.
relief_value_php	numeric		yes	Relief value, PHP (standard quantity × unit cost).
families_served	bigint		yes	Sum of beneficiary families over distributions (a family can be served more than once).

mart.mart_response_performance

Table 45 lists its columns.

Table 45: Columns of mart.mart_response_performance

Response timeliness with GROUPING SETS roll-ups; 'All' marks a rolled-up value. Materialized view, 671 rows. PK and FK mark the primary and foreign keys; Null says whether a value may be missing.

Column	Type	Key	Null	Description
event_code	text		yes	Event natural key, TC<year>-<LOCAL NAME> (e.g. TC2024-KRISTINE); 'All' in roll-up rows.

Column	Type	Key	Null	Description
response_type	text		yes	Response type name ('All' in roll-up rows).
priority	text		yes	P1 critical, P2 urgent, P3 routine ('All' in roll-up rows).
requests	bigint		yes	Number of requests.
median_hours_to_dispatch	numeric		yes	Median hours from request to dispatch.
p90_hours_to_dispatch	numeric		yes	90th-percentile hours from request to dispatch.
median_hours_to_arrive	numeric		yes	Median hours from dispatch to arrival.
median_hours_to_complete	numeric		yes	Median hours from request to completion.
completion_rate	numeric		yes	Share of requests completed, 0–1.
personnel_deployed	bigint		yes	Total personnel sent.

3.5 Supporting layers

Column-level detail for these layers is in the database itself; summarized here.

Table 46 summarizes them.

Table 46: Supporting objects in the raw, reference, staging, data-quality and governance layers
These feed or protect the star schema; users of the dashboards never read them directly.

Object	Kind	Rows	Columns
raw.dromic_displaced_outside	table	21,896	report_date, disaster_name, region, province, municipality, families_outside, persons_outside, _batch_id, _source_file, _row_number
raw.dromic_evacuation	table	31,241	report_date, disaster_name, region, province, municipality, barangay, ec_code, ec_name, ec_type, capacity_families, families_inside, persons_inside, _batch_id, _source_file, _row_number
raw.lgu_response_log	table	32,749	request_id, incident, province, lgu_name, request_type, priority, requested_at, dispatched_at, arrived_at, completed_at, responding_unit, personnel_count, _batch_id, _source_file, _row_number
raw.ndrrmc_damage_report	table	16,151	sitrep_no, report_date, incident_name, region, province, city_muni, damage_category, dead, injured, missing, affected_families, affected_persons, houses_partial, houses_total, cost_php, remarks, _batch_id, _source_file, _row_number
raw.ndrrmc_lifeline_status	table	2,677	incident_name, region, province, city_muni, lifeline, units_affected, interrupted_at, restored_at, _batch_id, _source_file, _row_number
raw.ngo_distribution	table	8,438	record, _batch_id, _source_file, _row_number

Object	Kind	Rows	Columns
raw.pagasa_event	table	39	local_name, international_name, category, max_wind_kph, max_signal, start_date, end_date, affected_areas, _batch_id, _source_file, _row_number
ref.location_alias	table	0	alias_name, alias_scope, psgc_code, match_method
ref.organization	table	13	org_code, org_name, org_type, parent_agency
ref.organization_alias	table	42	alias_name, org_code
ref.psgc	table	43,793	psgc_code, name, geographic_level, region_code, province_code, city_muni_code, city_class, income_class, urban_rural, population, population_census, publication, province_name
ref.psgc_changes	table	37	psgc_code, city_muni_name, attribute, old_value, new_value, effective_date, reference
ref.resource_item	table	10	item_code, item_name, category, standard_unit, unit_cost_php
ref.resource_item_alias	table	25	source_item_code, item_code, source_unit, to_standard_factor
meta.defect_manifest	table	19	source_system, defect, planted_count, seed, loaded_at
meta.load_batch	table	1,099	batch_id, source_system, source_file, row_count, started_at, finished_at, status, origin
meta.schema_migration	table	8	version, applied_at
staging.stg_dromic_displaced_outside	table	21,896	_batch_id, _source_file, _row_number, raw_report_date, report_date, disaster_name, province, raw_municipality, families_outside, persons_outside, psgc_code, location_match_method, event_key, dq_issues, dq_status
staging.stg_dromic_evacuation	table	31,241	_batch_id, _source_file, _row_number, raw_report_date, date_format, dominant_date_format, report_date, disaster_name, province, raw_municipality, barangay, raw_ec_code, ec_name, ec_type, capacity_families, families_inside, persons_inside, psgc_code, location_match_method, event_key, ec_code, dq_issues, dq_status
staging.stg_event_alias	table	124	event_key, alias, start_date, end_date

Object	Kind	Rows	Columns
staging.stg_lgu_response	table	32,749	_batch_id, _source_file, _row_number, request_id, incident, province, raw_lgu_name, raw_request_type, request_type, priority, requested_at, dispatched_at, arrived_at, completed_at, responding_unit, personnel_count, psgc_code, location_match_method, event_key, org_code, hours_to_dispatch, hours_to_arrive, hours_on_site, hours_to_complete, dq_issues, dq_status
staging.stg_location_resolution	table	2,969	raw_province, raw_name, psgc_code, official_name, match_method, edit_distance
staging.stg_ndrrmc_damage	table	16,151	_batch_id, _source_file, _row_number, copy_number, sitrep_no, report_date, incident_name, province, raw_city_muni, damage_category, dead, injured, missing, affected_families, affected_persons, houses_partial, houses_total, cost_php, remarks, raw_dead, raw_injured, raw_missing, raw_affected_families, raw_affected_persons, raw_houses_partial, raw_houses_total, raw_cost_php, psgc_code, location_match_method, event_key, dq_issues, dq_status
staging.stg_ndrrmc_lifeline	table	2,677	_batch_id, _source_file, _row_number, incident_name, province, raw_city_muni, lifeline, units_affected, interrupted_at, restored_at, raw_restored_at, psgc_code, location_match_method, event_key, dq_issues, dq_status
staging.stg_ngo_distribution_line	table	15,277	_batch_id, _source_file, _row_number, line_no, distribution_id, distributed_on, event_key, raw_event_ref, org_code, raw_organization, psgc_code, province, raw_municipality, location_match_method, beneficiary_families, item_code, source_item_code, source_unit, source_quantity, to_standard_factor, standard_unit, quantity_standard, value_php, header_issues, line_issues, dq_issues, dq_status

Object	Kind	Rows	Columns
staging.stg_pagasa_event	table	39	_batch_id, _source_file, _row_number, local_name, international_name, category, max_wind_kph, max_signal, start_date, end_date, affected_areas, event_key, copy_number, key_rank, dq_issues, dq_status
dq.dq_issue_log	table	20,507	source_system, rule_code, field_name, severity, _batch_id, _source_file, _row_number, line_no
dq.dq_quarantine	table	925	source_system, _source_file, _row_number, line_no, reject_reasons, all_issues, batch_source, batch_loaded_at
dq.dq_reconciliation	table	19	source_system, planted_defect, detected_by, planted_count, detected_count, reconciliation_status
dq.dq_rule_summary	table	45	source_system, rule_code, severity, action, description, issue_count, affected_rows, source_rows, pct_rows_affected
dq.dq_rules	table	45	source_system, rule_code, severity, action, description, planted_defect
secure.regional_event_impact	view	168	event_code, event_name, region_name, province_name, dead, affected_persons, houses_totally_damaged, damage_cost_php, relief_value_php, family_servings
secure.regional_response_summary	view	450	event_code, event_name, region_name, response_type, requests, median_hours_to_dispatch, completion_rate

4. ETL Design

4.1 Overview

The ETL is a set of **custom Python and SQL scripts**: Python for extraction and loading, and **dbt** for transformation, data-quality rules and tests. One command runs the whole pipeline:

```
.venv/Scripts/python ingestion/pipeline.py      # migrate → reference → psgc → generate → raw → dbt → export
```

Figure 8 shows the workflow from source files to dashboards.

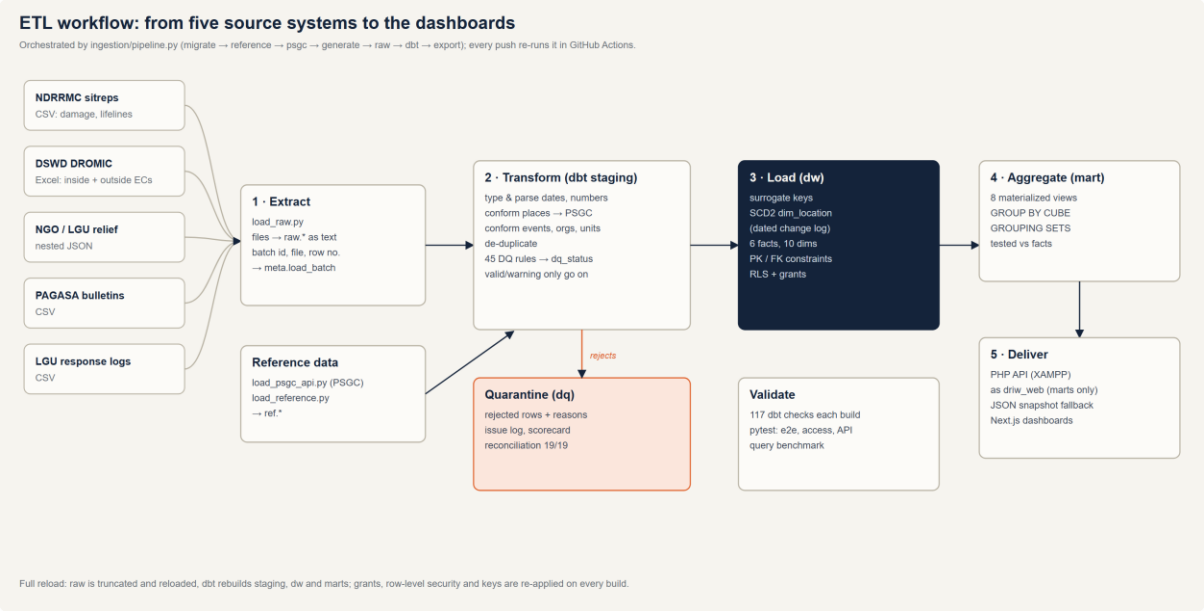


Figure 8: ETL workflow

The source systems land unchanged in the raw layer, are typed, conformed and quality-checked in staging, and are loaded into the star schema and marts that the dashboards and exports read.

4.2 Data sources

Table 47 lists every source with its format, volume and the defects it carries.

Table 47: Data sources with format, volume and planted defects

Defects are planted on purpose and counted, so the transformation can be proven correct (Chapter 5).

Source system	Format	Files	Rows landed	Typical defects (planted and counted)
NDRRMC situation reports	CSV	31 (one per event)	16,151	duplicate rows, blanks and "N/A", thousands separators, negative costs, swapped figures, misspelled towns

Source system	Format	Files	Rows landed	Typical defects (planted and counted)
NDRRMC lifeline status (roads, bridges, power, water, communications)	CSV	31	2,677	none planted; checked for unknown lifelines, unparseable times and restoration before interruption
DSWD DROMIC evacuation reports, sheet "Inside ECs"	Excel (.xlsx)	31	31,241	mixed date formats per field office, missing center codes, overcrowding, misspelled towns
DSWD DROMIC, sheet "Outside ECs" (families with relatives or friends)	same workbooks	31	21,896	none planted; same date, location and persons-vs-families checks
NGO, UN and LGU relief distributions	nested JSON	6 (one per year)	8,438 records	organization aliases, partner item codes and units, missing locations, unknown items
PAGASA bulletins	CSV	1	39	re-issued (duplicate) bulletins, local vs international names
LGU / MDRRMO response logs	CSV	1	32,749	non-standard request types, arrival before dispatch, open requests, misspelled towns
PSA PSGC geography	JSON (community API mirror of PSA codes; PSGC API, n.d.)	6	43,793 units	— (master data)
Hand-curated reference data	CSV	3	events, 13 organizations, 10 relief items	—

Mock vs real. Event names, dates and geography are real; operational figures are synthetic, produced by `ingestion/generate_sources.py` with seed 271 (deterministic). The generator records each defect it plants in `_defect_manifest.json` so the transformation can be proven against it.

4.3 Extract (raw layer)

- `load_raw.py` reads each file in its native format (csv module, openpyxl, json) and bulk-loads it with PostgreSQL `COPY` into `raw.<source>`.
- **Every business column is stored as text**, so values like `"1,234"`, `"N/A"` or `"23-Oct-2024"` are never rejected at load: nothing is lost before the quality rules run.
- Each file becomes a batch in `meta.load_batch`; each row keeps `_batch_id`, `_source_file` and `_row_number` (lineage).
- Strategy: **full reload** (truncate and reload), appropriate for a batch analytical warehouse of this size and simplest to make repeatable.

4.4 Transform (staging and data-quality layers, dbt)

Cleaning

- Parse numbers (strip thousands separators; `"N/A"`, `"-"`, `"TBD"` → null), dates in four formats (ISO, US, `DD-Mon-YYYY`, Excel serial numbers) and timestamps.
- Trim and normalize text; map non-standard codes (`RESQ` → `RESCUE`, `Medical` → `MEDEVAC`).
- Remove exact duplicates (sitrep rows, re-issued bulletins).
- Impute a missing evacuation-center code when the same place, name and barangay always carry one code.

Integration (conformance)

- **Places → PSGC** (`stg_location_resolution`): exact name, then normalized name, then fuzzy match (Levenshtein ≤ 2) within the reported province; ties are left for a steward. 0 false matches against ground truth across 63,347 rows.
- **Events**: local name, international name, `TY <NAME>` or code, matched within the event's time window (PAGASA reuses names).
- **Organizations and relief items**: alias tables in `ref`; partner quantities converted to a standard unit (e.g. 1 sack of rice = 50 kg).
- NGO JSON is flattened from one record per distribution to one row per item line.

Data-quality rules. 45 rules in `dq_rules` with severity *info* (corrected), *warning* (kept, flagged) or *reject* (quarantined). Every staging row gets `dq_issues` and `dq_status`; staging never drops a row. Details: Chapter 5.

Aggregation. Facts keep detail at their grain; aggregation happens in the marts (materialized views) with `GROUP BY CUBE` and `GROUPING SETS`, and in dashboard queries.

4.5 Load (warehouse and marts)

- **Dimensions** load first: surrogate keys by `row_number()`; `dim_location` built as **SCD Type 2** from `ref.psgc` plus a dated change log (`psgc_changes`).
- **Facts** load only *valid* and *warning* rows and look up each dimension key; locations are looked up **as of the fact's date** (SCD2). Damage keeps the latest valid sitrep per event × place × category (sitreps are cumulative).
- Open milestones (a request not yet completed) point to date key -1, so foreign keys are never null.

- After each model: primary/foreign keys, grants and row-level-security policies are re-applied (post-hooks).
- **Marts** refresh as materialized views; `export_marts.py` writes the dashboards' fallback snapshot.

4.6 Orchestration, scheduling and error handling

Table 48 summarizes how the pipeline is run, scheduled and protected against bad data.

Table 48: Orchestration, scheduling and error handling

Row-level errors never stop a load, while a failing step or test stops the build before anything is published.

Concern	Design
Orchestration	`ingestion/pipeline.py` runs the steps in order; any step can be run alone (`pipeline.py dbt export`).
Scheduling	Batch, on demand or by a scheduler (Windows Task Scheduler / cron) running `pipeline.py`; GitHub Actions runs the full pipeline on every push.
Idempotency	Full reload plus dbt's rebuild make every run produce the same warehouse from the same inputs (tested).
Row-level errors	Never stop the load: bad rows go to `dq.dq_quarantine` with their reasons and source file and row.
Step failures	`pipeline.py` stops at the first failing step; `dbt build` fails if any of its 76 data tests fail, so a bad build is never published.
Monitoring	`dq.dq_rule_summary` (scorecard), `dq.dq_reconciliation`, `meta.load_batch` (row counts and times per file).

4.7 Tools

Table 49 lists the tools used at each stage.

Table 49: Tools used by the ETL pipeline

All are open source or free, so the pipeline runs on an agency's own hardware.

Purpose	Tool
Extraction and loading	Python 3.13, psycopg 3 (`COPY`), openpyxl
Transformation, data quality, tests	dbt Core 1.12 with dbt-postgres
Warehouse	PostgreSQL 17
Orchestration and CI	`pipeline.py`, GitHub Actions
Delivery	PHP 8.1 API on XAMPP (Apache), Next.js dashboards

5. Data Quality Framework

How DRIW detects, corrects, flags and quarantines bad source data before it reaches the star schema, and how we prove the rules work.

5.1 Where data quality happens

```
raw.*  ──▶  staging.stg_*  ──────────────▶  dw.*  (valid + warning rows only)
 (text,      typed, conformed, every row
  as-is)     tagged with dq_issues/dq_status
                       │
                       ├──▶ dq.dq_issue_log       one row per issue (rule, field, source
file, row)
                       ├──▶ dq.dq_rule_summary    scorecard per rule
                       ├──▶ dq.dq_quarantine      rejected rows + reasons
                       └──▶ dq.dq_reconciliation  planted vs detected (synthetic-data
proof)
```

Staging models never drop rows. Each raw row keeps its lineage (`_batch_id`, `_source_file`, `_row_number`) and gets.

Table 50 defines the two columns.

Table 50: Data-quality columns added to every staging row
Together they tell the loader which rows may reach the warehouse, and record why the others did not.

Column	Meaning
`dq_issues`	Array of rule codes that fired, e.g. `{LOC_NAME_VARIANT, MISSING_VALUE:dead}`
`dq_status`	`valid` · `warning` · `rejected` · `duplicate`

5.2 Severity policy

Each rule has a severity, and Table 51 gives the action each severity triggers.

Table 51: Severity policy of the data-quality rules
The severity decides whether a problem is corrected, flagged or quarantined.

Severity	Action	Reaches the warehouse?	Example
info	Corrected automatically	Yes	`Naga City` → PSGC *City of Naga*; `1,234` → 1234; `R50` sacks → kg
warning	Kept and flagged	Yes	Evacuation center over capacity; required measure blank
reject	Quarantined in `dq.dq_quarantine`	No	Negative cost; arrival before dispatch; unknown item code

The full catalog (45 rules across 5 sources, including lifeline and outside-evacuation-center checks) is `dbt/seeds/dq_rules.csv`. A dbt test fails the build if a staging model emits

a rule that is not in the catalog, or if a row's `dq_status` disagrees with the severities of its issues.

5.3 Key techniques

Location conformance (PSGC). Every distinct (province, city/municipality) pair from all sources is resolved once in `stg_location_resolution`, always within the reported province:

1. **Exact** official name.
2. **Normalized**: case, spacing, `ñ`→`n`, `City of X` / `X City`, `Sto.`/`Sta.`.
3. **Fuzzy**: Levenshtein distance ≤ 2 on normalized and letters-only keys. The best candidate must be unique; ties (e.g. `Malniao` is equally close to *Malinao* and *Manito*) stay unmatched for a data steward instead of being guessed.

How "0 false matches" is validated. Two sources carry the true place code inside another field: every LGU request ID and every DROMIC evacuation-center code begins with the 7-digit PSGC prefix of the city or municipality it belongs to. That prefix is the ground truth. The dbt test `assert_location_matching_is_accurate` compares it with the place the matcher chose for **every** such row, not a sample: 32,748 LGU rows and 30,599 DROMIC rows, 63,347 in all. A row fails if the two disagree; none do. The test runs on every build, so a change to the matcher that caused a wrong match would fail the build.

Two limits apply. Rows from NDRRMC, NGO and the outside-evacuation-center reports carry no embedded code, so they are not checked directly; they go through the same matcher, which resolves each distinct (province, city/municipality) pair once for all sources. And the misspellings were planted by the generator, so real reports may contain kinds of errors it did not plant. Names the matcher cannot resolve with confidence are left unmatched for a data steward rather than guessed.

Event conformance. Sources refer to events by local name (*Kristine*), international name (*Trami*), `TY KRISTINE` or the key `TC2024-KRISTINE`. `stg_event_alias` lists every form; matches must also fall within the event's time window, because PAGASA reuses names.

Unit standardization. Partner item codes map to canonical items with a conversion factor (`R50`: 1 sack = 50 kg), so quantities and PHP values are comparable across organizations.

Imputation. A missing evacuation-center code is recovered when the same place + center name + barangay always carries a single code elsewhere; otherwise it's rejected.

De-duplication. Exact duplicate sitrep rows and re-issued PAGASA bulletins are marked `duplicate` and excluded.

5.4 Proof: reconciliation against planted defects

The synthetic generator records every defect it plants in `_defect_manifest.json` (loaded to `meta.defect_manifest`). `dq.dq_reconciliation` compares that with detected issues.

Table 52 gives the result.

Table 52: Reconciliation of planted and detected defects

Every planted defect type is found at exactly the planted count, and no row is rejected without a planted cause.

Result (seed 271)	
Defect types planted	19
Exactly reconciled	**19 / 19**
Rejected rows not explained by a planted defect	0

Load rates after data quality: NDRRMC 98.2%, DROMIC 99.6%, NGO lines 97.7%, LGU 98.8%; the outside-evacuation-center and lifeline reports load in full (100%).

A warn-level test (`warn_unplanted_rejections`) flags reject rules firing with no planted cause. During development it caught a real parsing bug: monsoon incidents named *"Southwest Monsoon (Habagat) Carina (Gaemi)"* were matched on the first parentheses instead of the last, wrongly rejecting 1,244 rows.

5.5 Running

```
.venv/Scripts/python ingestion/pipeline.py          # everything
.venv/Scripts/python ingestion/pipeline.py dbt      # rebuild staging + dq and run all
tests
```

Useful queries:

```
select * from dq.dq_reconciliation;                       -- planted vs detected
select * from dq.dq_rule_summary where issue_count > 0;   -- scorecard
select * from dq.dq_quarantine limit 50;                  -- what was rejected and why
```

6. Data Governance, Security and Privacy

Who may see what in DRIW, how it is enforced inside PostgreSQL, and how it is tested. Everything is enforced by the database itself, so it holds for any client: dashboards, SQL tools, BI software or scripts.

6.1 Roles

Access is granted to **group roles** that describe a job. People (or service accounts) are made members of a group; nobody is granted on objects directly.

Table 53 defines the four roles and what each may and may not read.

Table 53: Database roles and what each may read
Access follows least privilege: each role sees only what its job needs.

Role	Stakeholder (section 1.2)	Can read	Cannot read
`driw_executive`	Executive leadership	`mart.*` aggregates	warehouse detail, staging, raw, quarantine
`driw_analyst`	Data analysts	`dw.*` (all rows), `mart.*`, `secure.*`, DQ scorecards (`dq_rule_summary`, `dq_reconciliation`)	raw, staging, quarantine
`driw_regional`	Operational managers in a region	`dw.*` and `secure.*`, **only rows in their own region(s)**	marts (cross-region), other regions, the region mapping
`driw_steward`	Support staff / data engineers	`raw`, `staging`, `ref`, `meta`, all of `dq` incl. quarantine; may edit alias maps in `ref`	`dw`, `mart`

This follows **least privilege**: executives see conclusions, analysts see detail, regional officers see their own area, and stewards work on data quality without browsing the finished warehouse.

Demo personas used in tests and the presentation (NOLOGIN, used with `SET ROLE`): `driw_demo_executive`, `driw_demo_analyst`, `driw_demo_bicol`, `driw_demo_cagayan`, `driw_demo_steward`.

The web back-end's login

The PHP API (`web/php/api`, served by XAMPP) connects as `driw_web`, a LOGIN role whose only membership is `driw_executive`. A bug or an injection in the web tier can therefore reach the aggregated marts but never warehouse detail, raw data or quarantine (tested in `tests/test_php_api.py`). It also has a 10-second statement timeout and a 20-connection limit. Its password lives only in the git-ignored `.env` and in a PHP config outside

the web root (`C:\xampp\driw-config\config.php`), both written by `ingestion/setup_web_role.py`.

6.2 Row-level security (regional users)

`governance.region_access` maps a role to the region(s) it may see.

Table 54 shows the mapping for the two demonstration users.

Table 54: Region mapping used by row-level security

Each regional role is mapped to the region(s) whose rows it may see; a role with no mapping sees nothing.

role_name	region_code	region
driw_demo_bicol	0500000000	Bicol Region
driw_demo_cagayan	0200000000	Cagayan Valley

Each fact table has row-level security enabled with two policies (`dbt/macros/row_security.sql`):

```
create policy analyst_all_rows on dw.fact_damage_assessment
    for select to driw_analyst using (true);

create policy regional_own_region on dw.fact_damage_assessment
    for select to driw_regional
    using (location_key in (
        select l.location_key from dw.dim_location l
        where l.region_code in (select region_code from governance.my_regions)));
```

- **Default deny.** A regional user not yet mapped to a region sees zero rows, not everything.
- **The mapping stays private.** `governance.my_regions` is a view that shows each user only their own regions. Inside a view `current_user` is the querying user, while the view reads the mapping table with its owner's rights. A `SECURITY DEFINER` function was tried first and rejected: inside one, `current_user` is the function's owner, so no user ever matched (the governance tests caught this).
- **Secure views inherit it.** The views in `secure` (`regional_event_impact`, `regional_response_summary`) are `security_invoker`, so the same view shows Bicol to the Bicol officer, Cagayan Valley to the Cagayan officer, and everything to an analyst.
- **Marts are excluded on purpose.** Materialized views cannot carry row-level security, so regional users are not granted the marts; the secure views serve them instead.

Granting a new regional officer:

```
grant driw_regional to new_officer;
insert into governance.region_access (role_name, region_code) values ('new_officer',
'0800000000');
```

6.3 Surviving rebuilds

dbt rebuilds tables on every run, which would normally drop privileges. Grants are declared in `dbt_project.yml` (`+grants`) and row-level security is a post-hook on each fact, so both are re-applied on every build. The end-to-end tests run a second `dbt build` and check that policies and grants are still in place.

6.4 Auditing access

`governance.v_access_matrix` lists, for every role and every object, whether the role can actually read it (schema `USAGE` and object `SELECT`) and whether row-level security applies. It is the role table (Table 53) generated from the database itself:

```
select role_name, schema_name, count(*) filter (where can_select) as readable, count(*)
as objects
from governance.v_access_matrix group by 1, 2 order by 1, 2;
```

6.5 Privacy

The governing law is the **Data Privacy Act of 2012** (Republic Act No. 10173, 2012) and its implementing rules. DRIW is designed so that it holds no personal information at all.

Data classification

Table 55 classifies the data DRIW holds and how each class is handled.

Table 55: Data classification under the Data Privacy Act
Personal information is not collected at all; sensitive aggregates are limited by role and region.

Class	Examples in DRIW	Handling
Public	Event names and dates, PSGC geography, organization names	No restriction
Internal (aggregated operational)	Damage costs, relief values, response times	Role-based access (Executive, Analyst)
Sensitive (aggregated)	Deaths, injured, missing, persons in evacuation centers, by city/municipality	Analysts and the region's own officers only (row-level security); executives see marts only
Personal information	Names, addresses, contact details or health data of affected people or responders	**Not collected.** Never enters the warehouse.

Privacy principles applied

- **Data minimization.** Sources are reduced to counts at the lowest level the analysis needs (city/municipality, evacuation center, request). No evacuee rosters, victim lists or patient records are loaded; a response request carries a request id, type, timestamps and team size, never the requester's identity.

- **Aggregation and small counts.** Casualty figures are aggregated per city/municipality and event. Before any public release, counts below 5 should be suppressed or rounded to prevent re-identification in small barangays (a reporting rule for published outputs).
- **Purpose limitation.** Data is used only for disaster-response analysis and planning, as stated in the business case.
- **Least privilege and need to know.** Four roles; regional officers see only their region; the web back-end logs in as `driw_web`, which can read only aggregated marts.
- **Security of processing.** Credentials are never in source code (git-ignored `.env`, PHP config outside the web root); the API accepts only allow-listed, validated GET requests; all access is enforced inside PostgreSQL, so it holds for every client.
- **Retention.** Raw extracts are replaced on every load (full reload). Warehouse history is kept for the analysis period (2020 onwards); source files under `data/sources/` are regenerable and not retained in version control.
- **Transparency.** Every published figure is traceable to its source file and row (lineage), and every rejected row is kept with its reasons (quarantine).
- **Synthetic data.** In this implementation all operational figures are synthetic; no real person's data was processed at any stage.

If real data is onboarded: sign data-sharing agreements with each agency, register the processing with the National Privacy Commission where required, appoint a data protection officer, run a privacy impact assessment, and enable PostgreSQL connection and statement logging (`log_connections`, `pgaudit`) for an access audit trail.

6.6 Scope of the security evidence

The governance tests (Chapter 8) show that the access rules work as designed: each role reads what it should and is refused what it should not, regional users see only their region, and the web login reaches only the marts. That is evidence about the design, not a certification of production readiness. DRIW has not had a security audit or a penetration test. Before real agency data is loaded, it would also need encrypted connections (TLS), a password and account policy, audit logging (`pgaudit`), hardening of the XAMPP server, and an independent review.

6.7 Supabase

The same migrations run on Supabase. There, signed-in users connect as Supabase's `authenticated` role, so the regional mapping would key on the user's id (`auth.uid()`) or a JWT claim instead of `current_user`; the policies keep the same shape.

6.8 Standards alignment

This supports the emphasis of the UNDRR Data Strategy and Roadmap 2023–2027 (UNDRR, 2023) on data governance: defined roles and responsibilities (stewardship), controlled access to sensitive operational data (casualties by locality), and auditable, reproducible data handling.

7. Analytics: OLAP and Dashboards

7.1 OLAP approach (ROLAP)

DRIW implements OLAP relationally (ROLAP) in PostgreSQL rather than with a separate MOLAP cube engine. The star schema is the cube: facts hold the measures, dimensions hold the hierarchies, and SQL's OLAP operators produce the cube's cells.

Table 56 shows how each OLAP operation is expressed in SQL.

Table 56: OLAP operations and how DRIW performs them in SQL
Each operation is demonstrated by a sample query in Appendix A.

OLAP operation	How it is done	Example
Roll-up	`GROUP BY ROLLUP`, or grouping at a higher hierarchy level	Q02: damage by year with yearly subtotals
Drill-down	Group by a lower level of a hierarchy (region → province → city/municipality)	Q03
Slice	Fix one dimension value (`WHERE`)	Q07 over-capacity centers; Q09 life-saving requests
Dice	Filter on several dimensions at once	Q11 organization × resource category
Pivot	Turn dimension values into columns (`FILTER`)	Q04 cyclone vs flood per region
Cube	Every combination of dimensions with subtotals (`GROUP BY CUBE`)	Q05; `mart.mart_impact_cube`
Grouping sets	Chosen roll-ups in one pass	`mart.mart_response_performance`

Pre-aggregation. Eight materialized views in the `mart` schema hold the aggregates the dashboards need. They refresh on every build and are tested to add back up to the facts. Reading a mart is about 60–80× faster than aggregating the facts (see the Query Performance Test Report (Chapter 8)).

The full set of sample analytical queries, each tied to a business question, follows this chapter and is in Appendix A.

7.2 Dashboards and web application walkthrough

The web application is the warehouse's presentation layer. It is built with Next.js and served by Apache (XAMPP) at `http://localhost/driw/`. A PHP API (`/driw/api/`) reads the marts live from PostgreSQL as the read-only role `driw_web`; if the warehouse is unreachable, the pages fall back to a bundled snapshot of the same marts. This section shows every page and explains what each part of it means. All figures shown are synthetic (see the notice on every page).

Elements common to every page

Some elements appear on every page; Table 57 explains them.

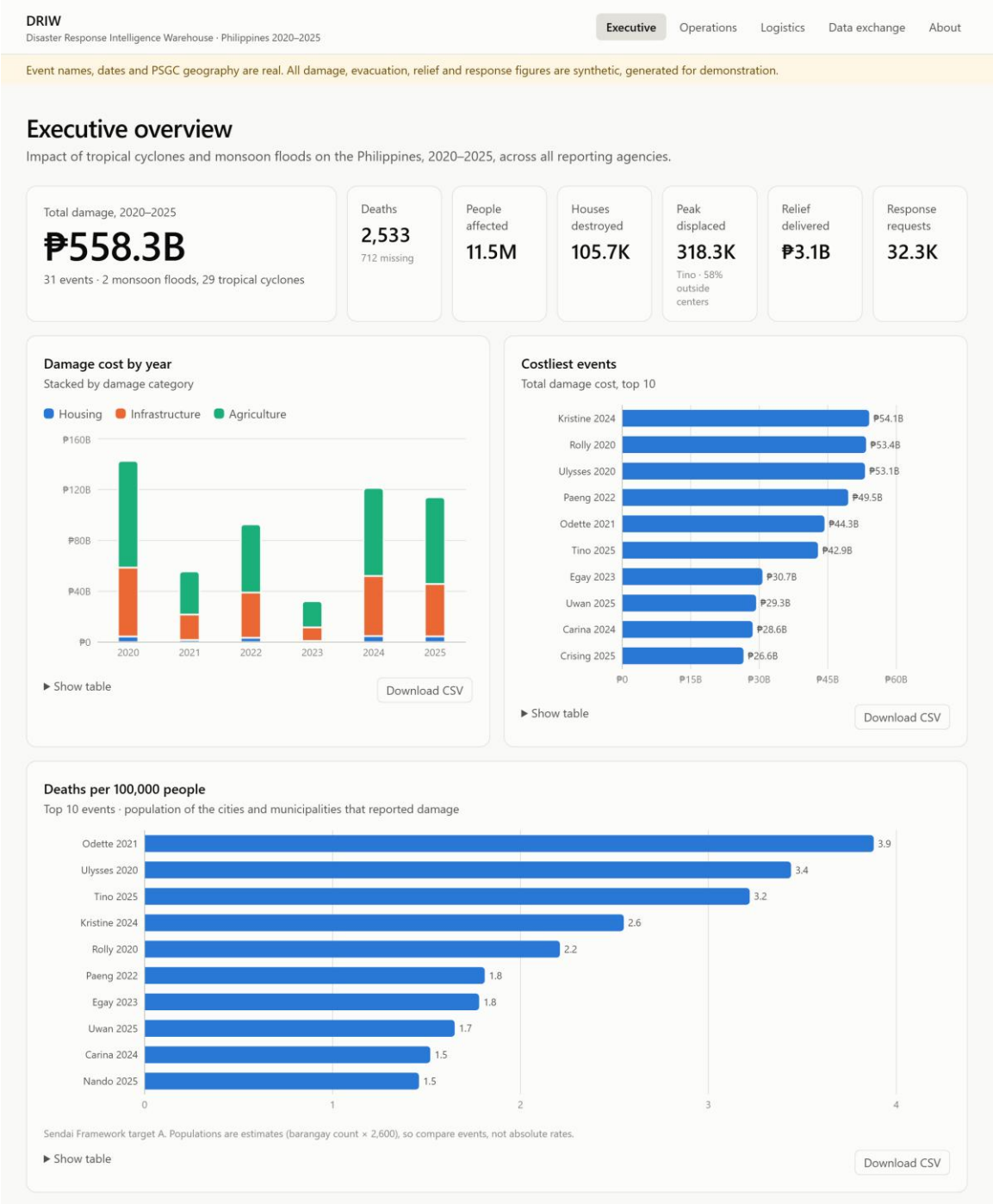
Table 57: Elements common to every dashboard page

They let a user filter, check and export what any chart shows.

Element	What it means
Navigation bar	Executive, Operations and Logistics (one per audience), Data exchange (templates, checked uploads and Excel export for LGUs and agencies, see Chapter 10) and About.
Yellow notice	States which data is real (event names, dates, PSGC geography) and which is synthetic (all operational figures), so no figure is mistaken for an official statistic.
Event selector	On Operations and Logistics. The choice is kept in the address (for example `?event=TC2024-KRISTINE`), so a view can be bookmarked, shared or reloaded. Selecting an event is an OLAP *slice*.
Show table	Every chart has a table with the exact numbers: the accessible, color-free way to read it.
Download CSV	Exports the rows behind the chart with raw, unformatted values; the file opens directly in Excel.
Footer	Says where the numbers came from: "Live from the PostgreSQL warehouse … as read-only role driw_web", or that the bundled snapshot is being shown.

Executive overview

Figure 9 shows the Executive dashboard.



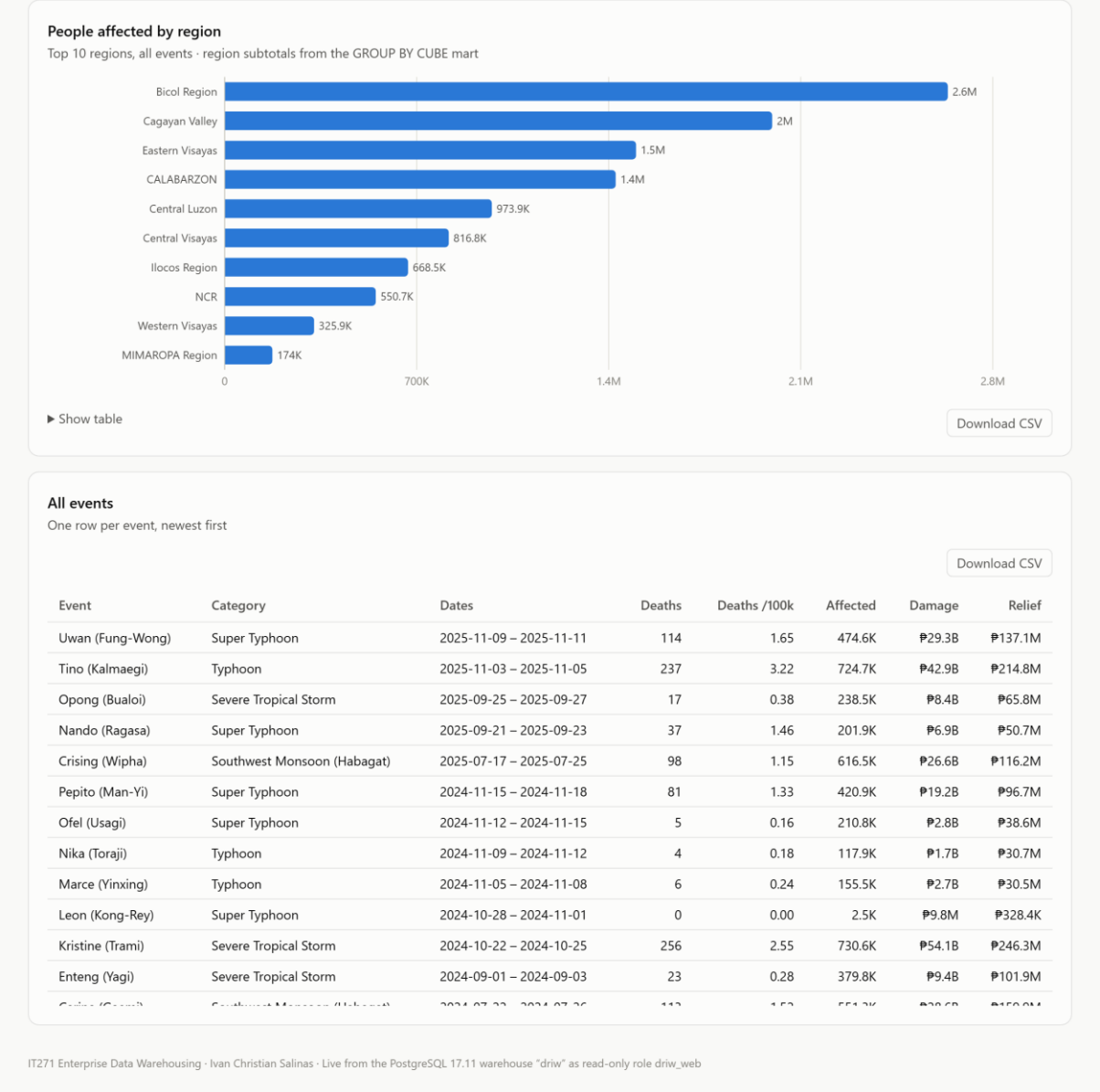


Figure 9: Executive dashboard

The six-year picture on one screen: total damage, headline tiles, damage by year, costliest events, people affected by region and deaths per 100,000 people.

Audience and purpose. Leadership (NDRRMC council, regional directors) needs the six-year picture on one screen: how bad it was, when, where, and how hard it hit relative to the population. Answers BQ1, BQ2, BQ4 and BQ11. Data comes from `mart_event_summary` (one row per event) and `mart_impact_cube` (the `GROUP BY CUBE` mart).

Table 58 explains each part of the screen in Figure 9.

Table 58: Parts of the Executive dashboard and how to read them
All figures are synthetic.

Part of the screen	What it shows and how to read it
Hero figure	**₱558.3B** total damage across **31 events** (29 tropical cyclones, 2 monsoon floods), the sum of the latest valid situation report per place and category.
Headline tiles	Deaths (2,533, with 712 missing), people affected (11.5M), houses destroyed (105.7K), relief delivered (₱3.1B) and response requests (32.3K) are sums over events. *Peak displaced* (318.3K, Tino 2025) is different: displacement is semi-additive, so it is the highest single-day total of people inside evacuation centers **plus** people staying with relatives or friends, not a sum of days. Its note says 58% of them were outside centers.
Damage cost by year	Stacked columns by damage category. 2020 was the costliest year (₱142.6B, four events) and 2023 the lightest (₱32.2B). Agriculture is the largest category every year (₱83.9B of 2020's total), then infrastructure; housing is the smallest.
Costliest events	The ten events ranked by damage: Kristine 2024 (₱54.1B), Rolly 2020 (₱53.4B) and Ulysses 2020 (₱53.1B) lead. Three of the top five struck in 2020.
People affected by region	Region subtotals read straight from the CUBE mart: Bicol Region (2.64M people, ₱145.4B), then Cagayan Valley (2.00M) and Eastern Visayas (1.50M).
Deaths per 100,000 people	The ten events with the highest death rate relative to the population of the places that reported damage (close-up in Figure 11).
All events table	One row per event, now with a *Deaths /100k* column; its CSV carries 38 columns per event, including the rates, displacement, relief by provider and lifeline figures.

What it means for decisions. Damage is concentrated in a few strong typhoons and in a few regions, Bicol above all. Agriculture absorbs the largest share of losses, so preparedness spending on crop protection and post-disaster agricultural recovery gives the most return.

Close-up: headline tiles

Figure 10 enlarges the headline tiles.



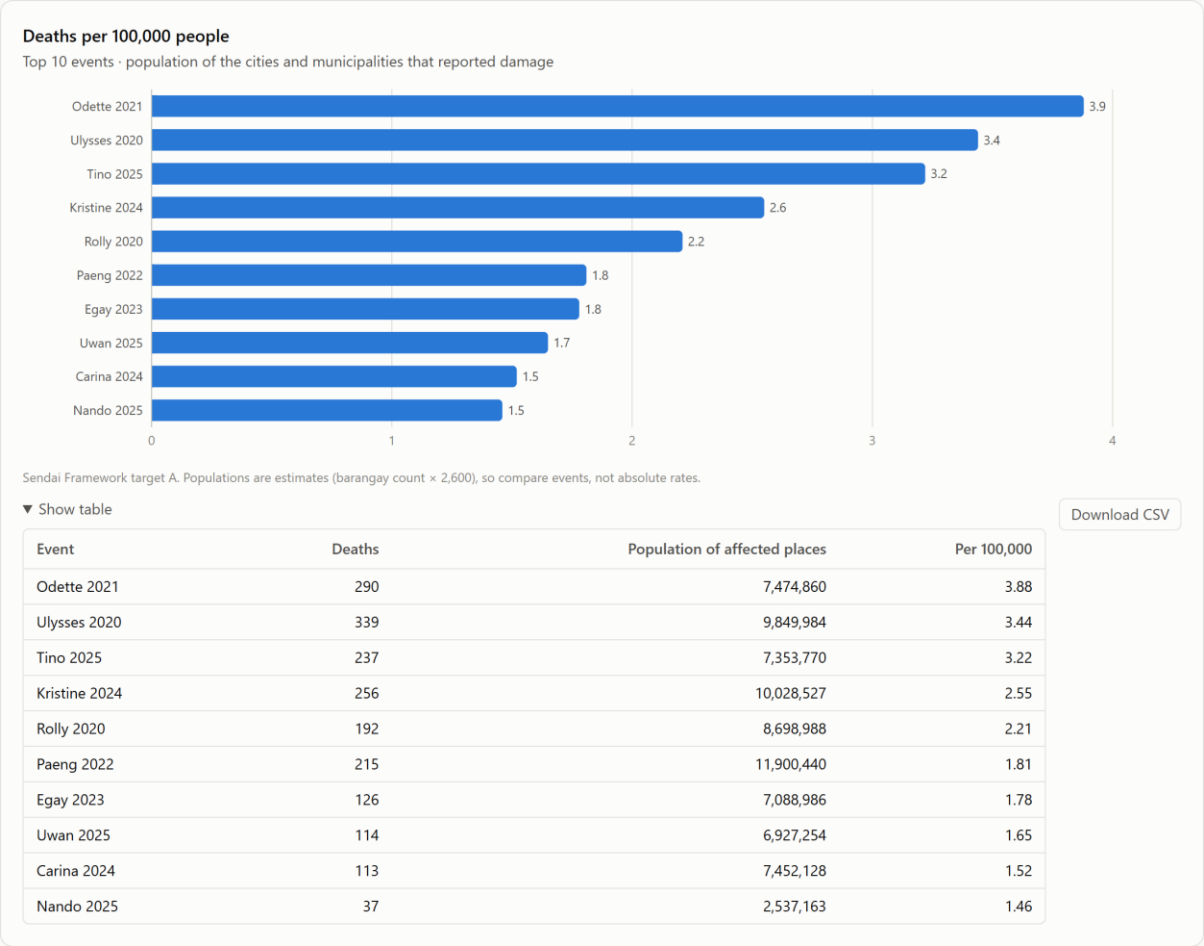
Figure 10: Close-up of the Executive headline tiles
The peak-displaced tile counts people inside and outside evacuation centers, more than twice the number in centers alone.

The tile row is the first thing leadership reads. The hero figure (₱558.3B) is the six-year cost; the tiles beside it count people, not pesos. The *Peak displaced* tile is the one to read carefully: **318.3K** people were displaced on Typhoon Tino's worst day in November 2025,

but only 132.5K of them were in evacuation centers. The other 185.8K (58%) stayed with relatives or friends. A dashboard that counted only evacuation centers would have shown less than half of the people who needed food packs, water and transport home.

Close-up: deaths per 100,000 people

Figure 11 enlarges the deaths-per-100,000 chart.



Event	Deaths	Population of affected places	Per 100,000
Odette 2021	290	7,474,860	3.88
Ulysses 2020	339	9,849,984	3.44
Tino 2025	237	7,353,770	3.22
Kristine 2024	256	10,028,527	2.55
Rolly 2020	192	8,698,988	2.21
Paeng 2022	215	11,900,440	1.81
Egay 2023	126	7,088,986	1.78
Uwan 2025	114	6,927,254	1.65
Carina 2024	113	7,452,128	1.52
Nando 2025	37	2,537,163	1.46

Figure 11: Close-up of deaths per 100,000 people

Ranking by death rate instead of death count shows which events hit hardest relative to the population in their path (Sendai Framework target A).

Raw death counts favour storms that crossed many densely populated places. This chart divides deaths by the population of the cities and municipalities that reported damage, the measure the **Sendai Framework for Disaster Risk Reduction** uses for its target A (mortality per 100,000; UNDRR, 2015, 2017). The table under the chart shows the three inputs: deaths, population of the affected places, and the rate.

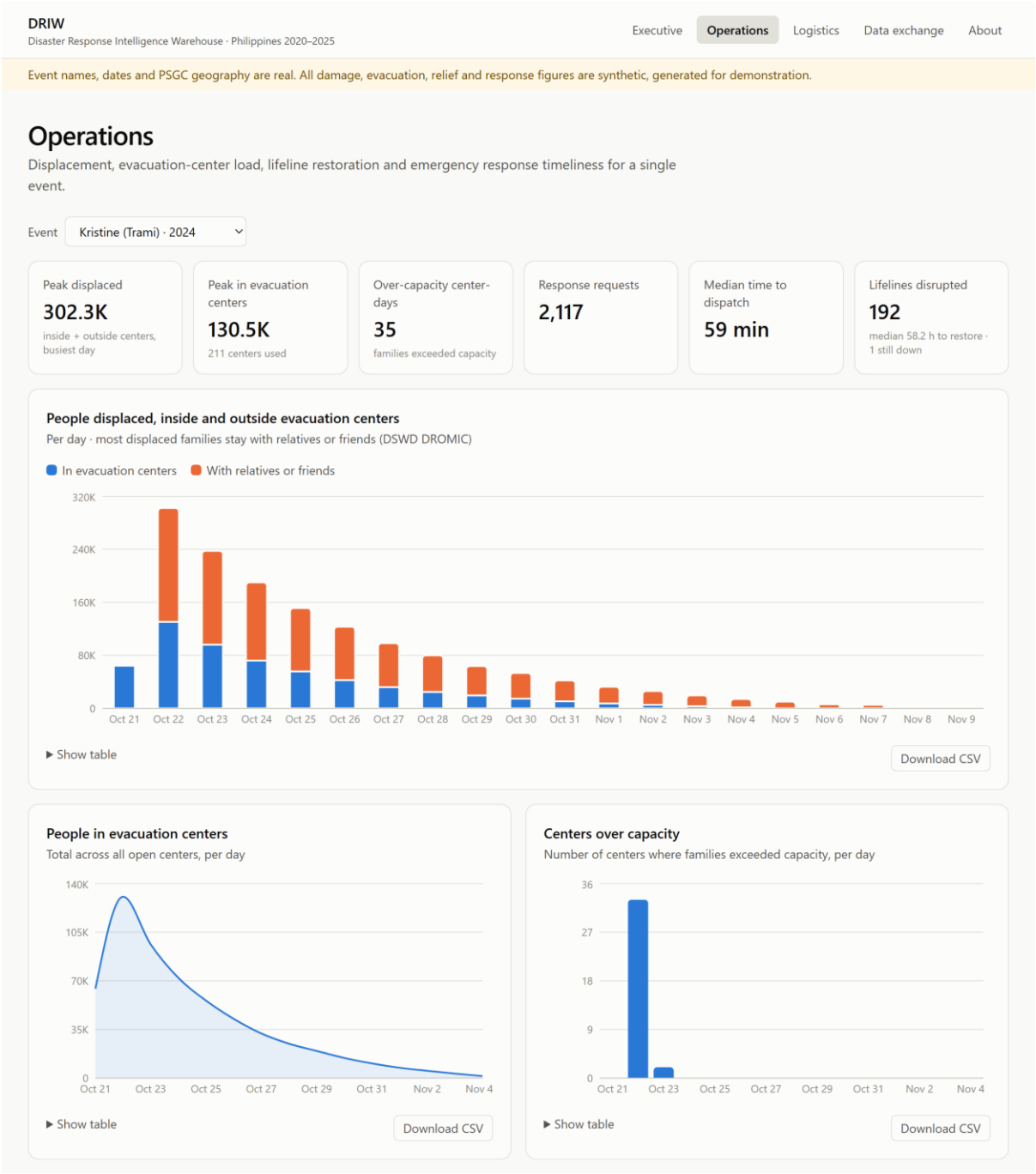
- **Odette 2021** has the highest rate (**3.88** deaths per 100,000) although Ulysses 2020 had more deaths (339 against 290): Odette killed a larger share of the people in its path.

- **Kristine 2024**, the costliest event (₱54.1B), is only fourth by death rate (2.55). Cost and mortality rank storms differently, so both views are needed.
- **Nando 2025** enters the top ten with just 37 deaths, because it struck a small population (2.5 million).
- Across the six years, 2,533 deaths equal **2.13 per 100,000** of the national population.

The population figures are estimates (barangay count × 2,600; see the note on the chart), so the rates are for comparing events with each other, not official statistics. With the PSA census file loaded, the same chart shows official rates without any code change.

Operations (one event)

Figure 12 shows the Operations dashboard for Typhoon Kristine.



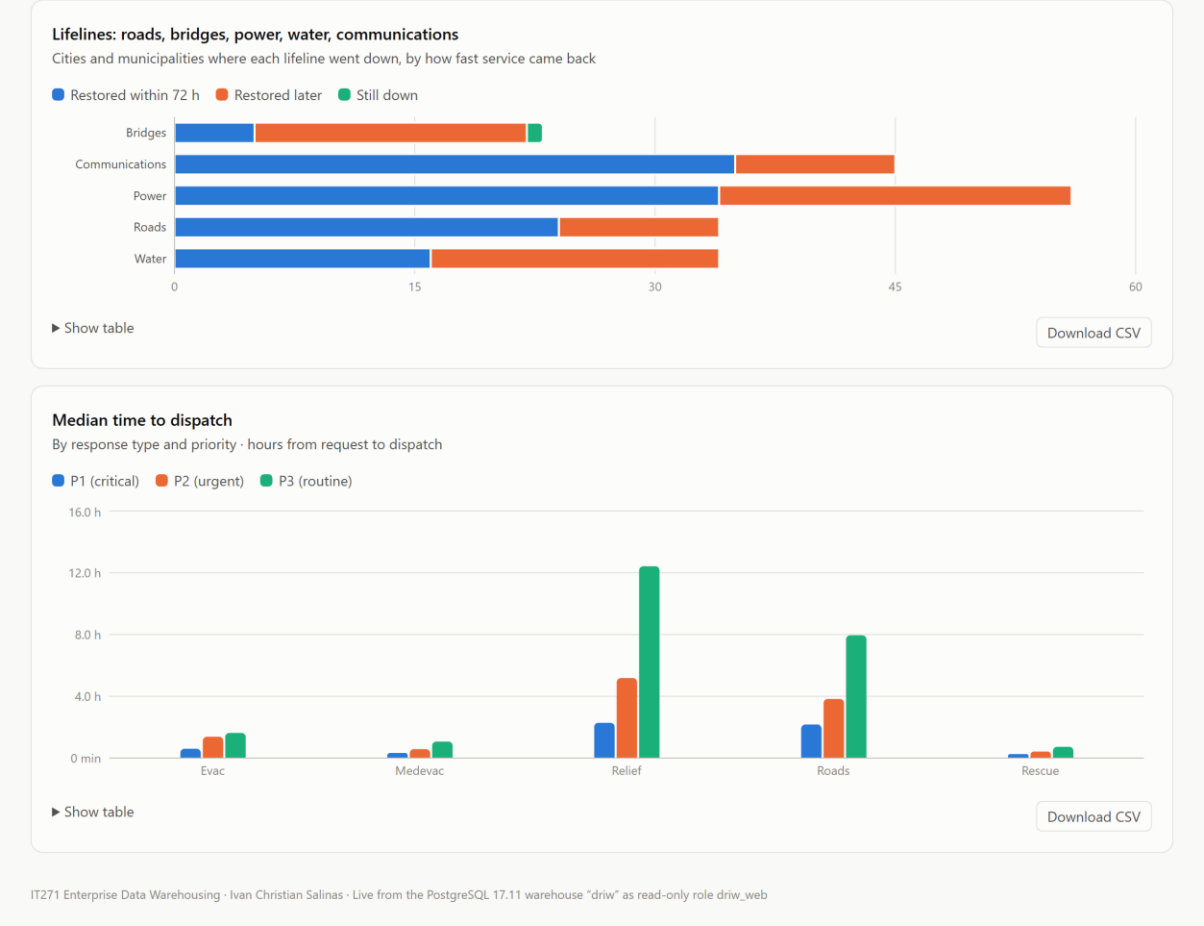


Figure 12: Operations dashboard for Typhoon Kristine (October 2024)

One event's displacement, overcrowding, lifeline restoration and dispatch times on one page.

Audience and purpose. Regional operations managers and incident commanders. For one event, it shows how many people were displaced (inside and outside evacuation centers), where centers were overcrowded, which roads, bridges, power, water and communication lines went down and how fast they came back, and how fast teams were dispatched. Answers BQ5, BQ6, BQ7 and BQ12. Shown here for Kristine (October 2024); data comes from `mart_event_summary`, `mart_displacement_daily`, `mart_evacuation_daily`, `mart_lifeline_restoration` and `mart_response_performance`.

Table 59 explains each part of the screen in Figure 12.

Table 59: Parts of the Operations dashboard and how to read them
The figures are for Typhoon Kristine.

Part of the screen	What it shows and how to read it
Headline tiles	Peak displaced **302.3K** (inside plus outside centers) on the busiest day; peak in evacuation centers **130.5K** in **211** centers; **35** center-days over capacity; **2,117** response requests; median **59 minutes** from request to dispatch; **192** lifeline disruptions, restored in a median **58.2 hours**, one still down.
People displaced, inside and outside evacuation centers	Stacked columns per day: evacuation centers (blue) and families staying with relatives or friends (orange). See Figure 14.
People in evacuation centers	Total evacuees across all open centers, per day. It rises from 64,326 on 21 October (pre-emptive evacuation) to the peak of 130,515 on landfall day (22 October), then decays over about two weeks as families go home.
Centers over capacity	Number of centers whose families exceeded capacity, per day: 33 centers on landfall day, 2 the day after, then none. Overcrowding is a one-to-two-day problem.
Lifelines	For each lifeline, the cities and municipalities where it went down, split by how fast service came back. See Figure 15.
Median time to dispatch	Grouped columns by response type and priority (P1 critical, P2 urgent, P3 routine). Life-saving work goes fastest: search-and-rescue P1 in 0.26 h (about 16 minutes), medical evacuation P1 in 0.33 h. Relief delivery and road clearing wait longest, especially when routine: 12.4 h and 7.9 h at P3.

What it means for decisions. Overflow centers must be ready *before* landfall, because nearly all overcrowding happens on that one day. Priority triage works for life-saving requests; the slow lanes are logistics (relief delivery, road clearing), where pre-positioned stock and equipment would cut waiting most.

Close-up: operations tiles

Figure 13 enlarges the headline tiles.

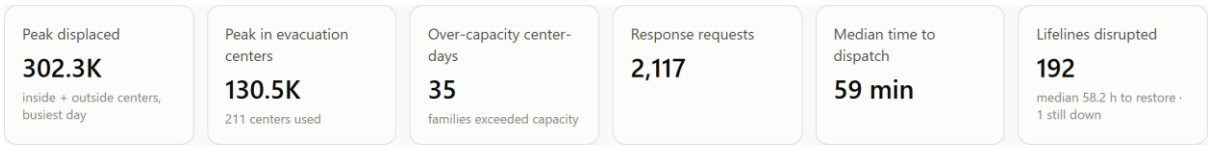


Figure 13: Close-up of the Operations headline tiles for Kristine
Six numbers summarize the size of the displacement, the overcrowding, the response and the lifeline disruption.

Six numbers summarise the operation. The first two together show the size of the displacement: 302.3K people at the peak, of whom 130.5K were in centers. The last tile, *Lifelines disrupted*, tells the incident commander how much of the area lost basic services

(192 city-and-lifeline combinations) and how long recovery took (median 58.2 hours, one bridge still impassable at the last report).

Close-up: people displaced inside and outside evacuation centers

Figure 14 enlarges the displacement chart.

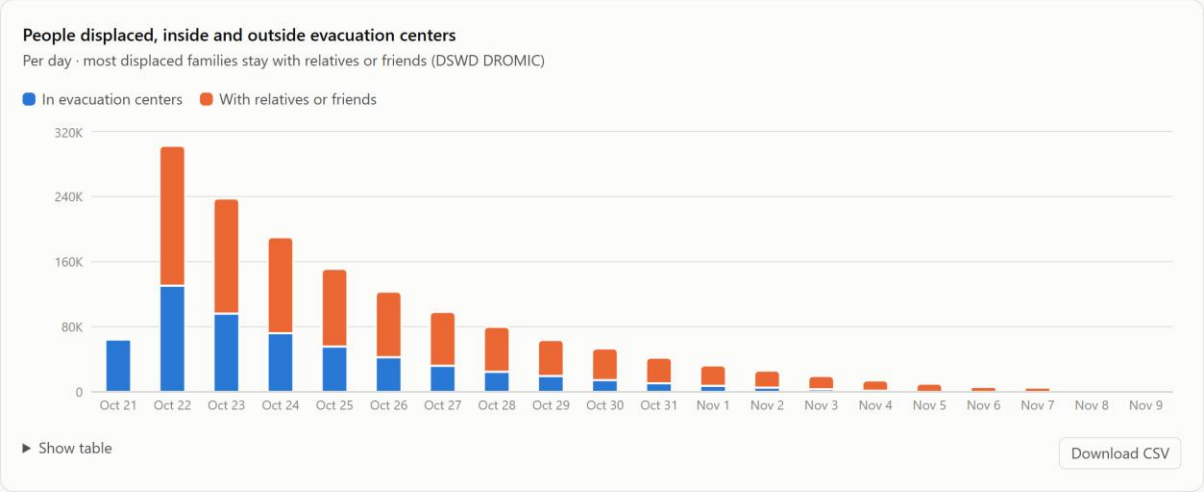


Figure 14: Close-up of daily displacement for Kristine

Blue is people in evacuation centers, orange people with relatives or friends; the orange part is the majority and lasts longer.

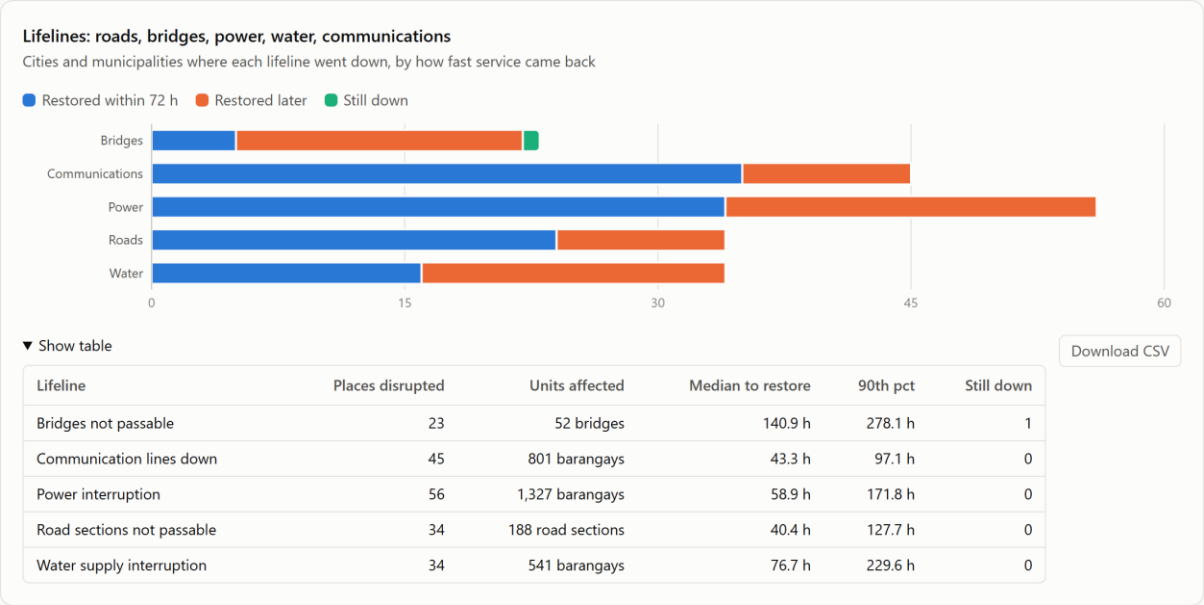
DSWD's DROMIC reports count displaced families in two places: inside evacuation centers, and outside them, staying with relatives or friends. The chart stacks both for every day of Kristine:

- **21 October** (the day before the event began) shows only blue: **64,326** people moved into centers as a pre-emptive evacuation.
- **22 October** (the first day of the event) is the peak: **302,348** displaced, **130,515** in centers and **171,833** (57%) outside.
- Afterwards the centers empty faster than homes of relatives: by 24 October 62% of the displaced were outside centers. The last evacuees left the centers on **4 November**, but families were still staying with relatives until **9 November**.

The difference matters for relief: families outside centers are harder to find and register, yet they are the majority. Across all 31 events, 61% of the people displaced on each event's peak day were outside centers. *Show table* gives the daily numbers and the share outside.

Close-up: lifelines (critical infrastructure and basic services)

Figure 15 enlarges the lifeline chart.



Lifeline	Places disrupted	Units affected	Median to restore	90th pct	Still down
Bridges not passable	23	52 bridges	140.9 h	278.1 h	1
Communication lines down	45	801 barangays	43.3 h	97.1 h	0
Power interruption	56	1,327 barangays	58.9 h	171.8 h	0
Road sections not passable	34	188 road sections	40.4 h	127.7 h	0
Water supply interruption	34	541 barangays	76.7 h	229.6 h	0

Figure 15: Close-up of lifeline restoration for Kristine

Each bar is one lifeline, split by how fast service came back; bridges and water take longest.

NDRRMC situation reports track the "lifelines" a community needs to function: roads and bridges (transport) and power, water and communications (utilities). Each bar is one lifeline; its length is the number of cities and municipalities where it went down, split into restored within 72 hours (blue), restored later (orange) and still down at the last report (green).

Table 60 gives the figures behind each bar of Figure 15.

Table 60: Lifeline disruption and restoration during Kristine

Read with Figure 15: the median gives typical recovery, and the last column what it means for operations.

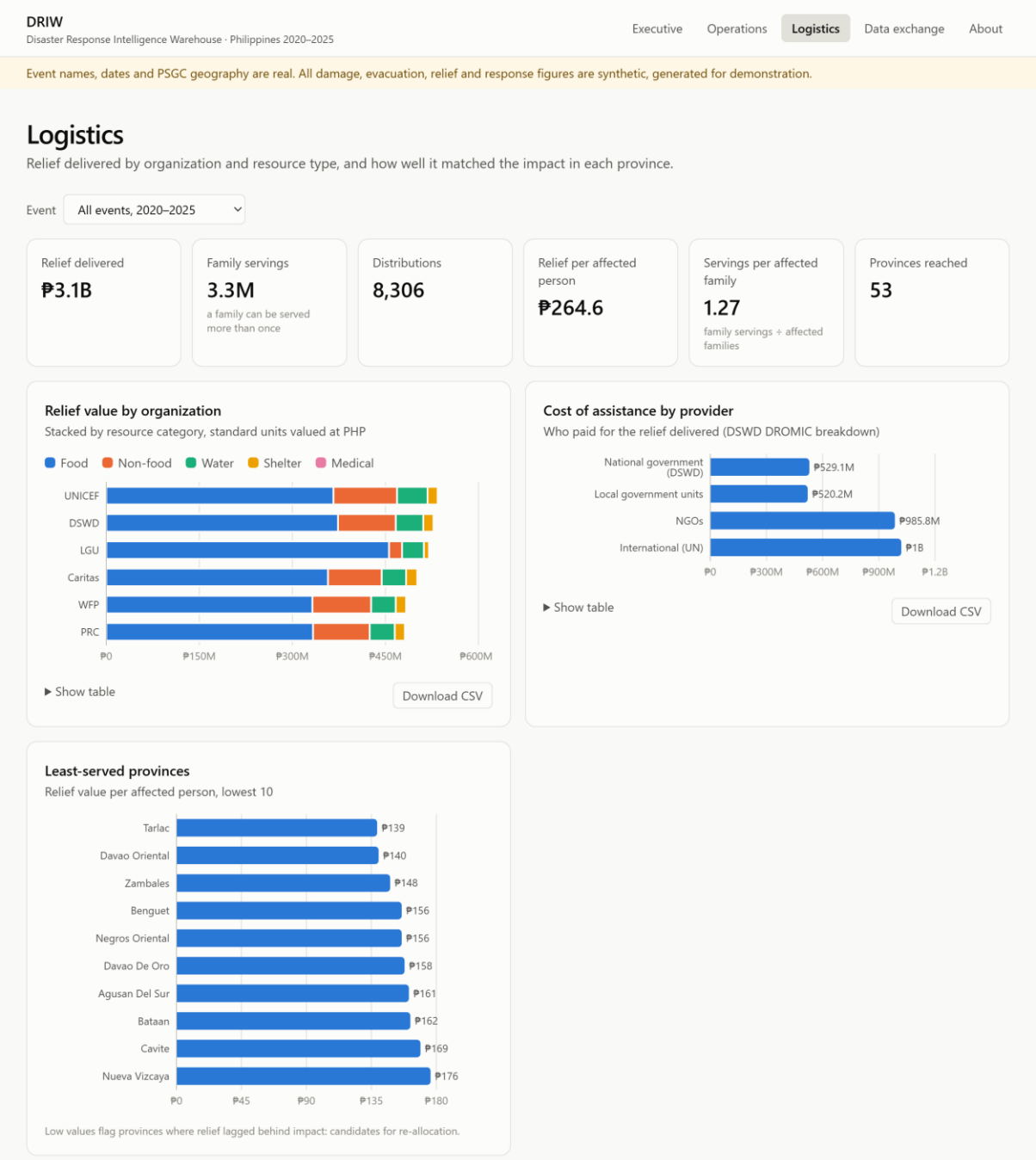
Lifeline (Kristine)	Places disrupted	Median to restore	Reading
Power	56 (1,327 barangays)	58.9 h	The most widespread outage; 34 of 56 places (61%) restored within three days.
Communications	45 (801 barangays)	43.3 h	Comes back fastest, usually with mobile network repairs.
Roads	34 (188 road sections)	40.4 h	Most cleared within three days, by DPWH and LGU teams.
Water	34 (541 barangays)	76.7 h	Slower: under half (16 of 34) back within three days, since pumps need power first.
Bridges	23 (52 bridges)	140.9 h	Slowest by far: fewer than a quarter reopened within three days, and one was still impassable.

The 90th-percentile column shows the worst cases (278 hours, almost 12 days, for bridges). For planning, the message is that bridges and water decide how long relief convoys

and evacuees are cut off, so spare bridge modules and water-treatment units should be pre-positioned in the areas most often hit.

Logistics (all events)

Figure 16 shows the Logistics dashboard for all events.



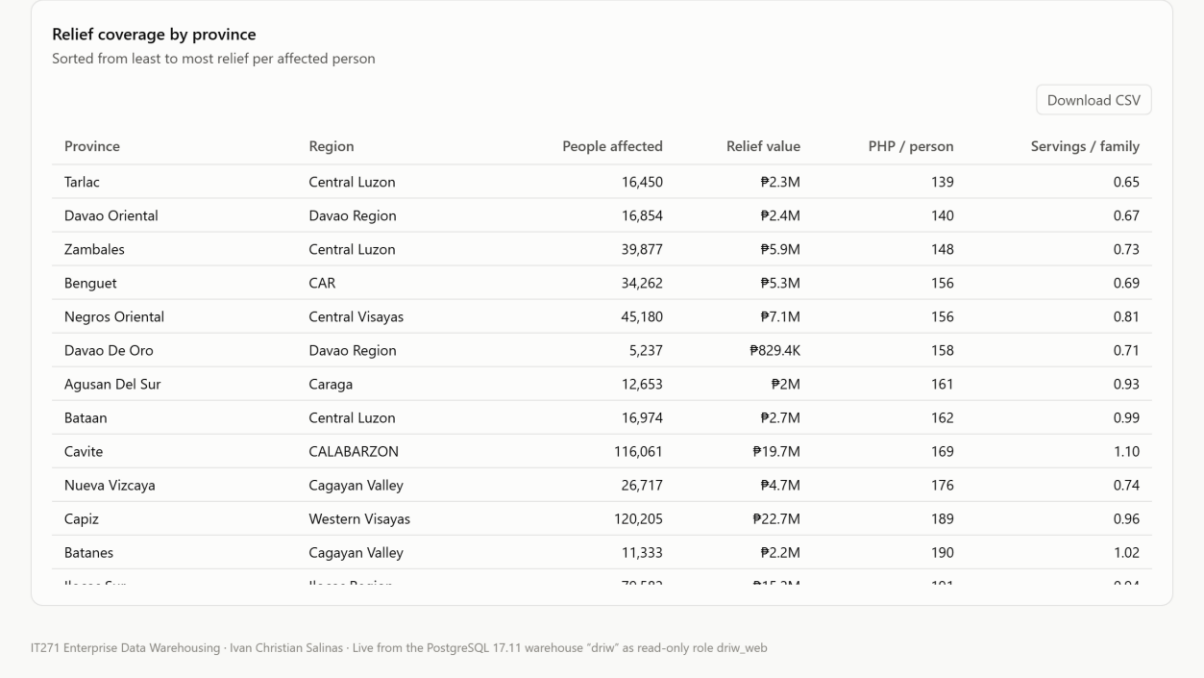


Figure 16: Logistics dashboard for all events

Relief by organization and resource category, cost of assistance by provider, and the provinces that received the least relief per affected person.

Audience and purpose. Relief and logistics officers of DSWD and partner NGOs. It shows who delivered what, and whether relief reached provinces in proportion to the impact. Answers BQ8 and BQ9 (including who paid for the relief). Data comes from `mart_relief_by_org_category`, `mart_province_event_impact` and `mart_event_summary`.

Table 61 explains each part of the screen in Figure 16.

Table 61: Parts of the Logistics dashboard and how to read them

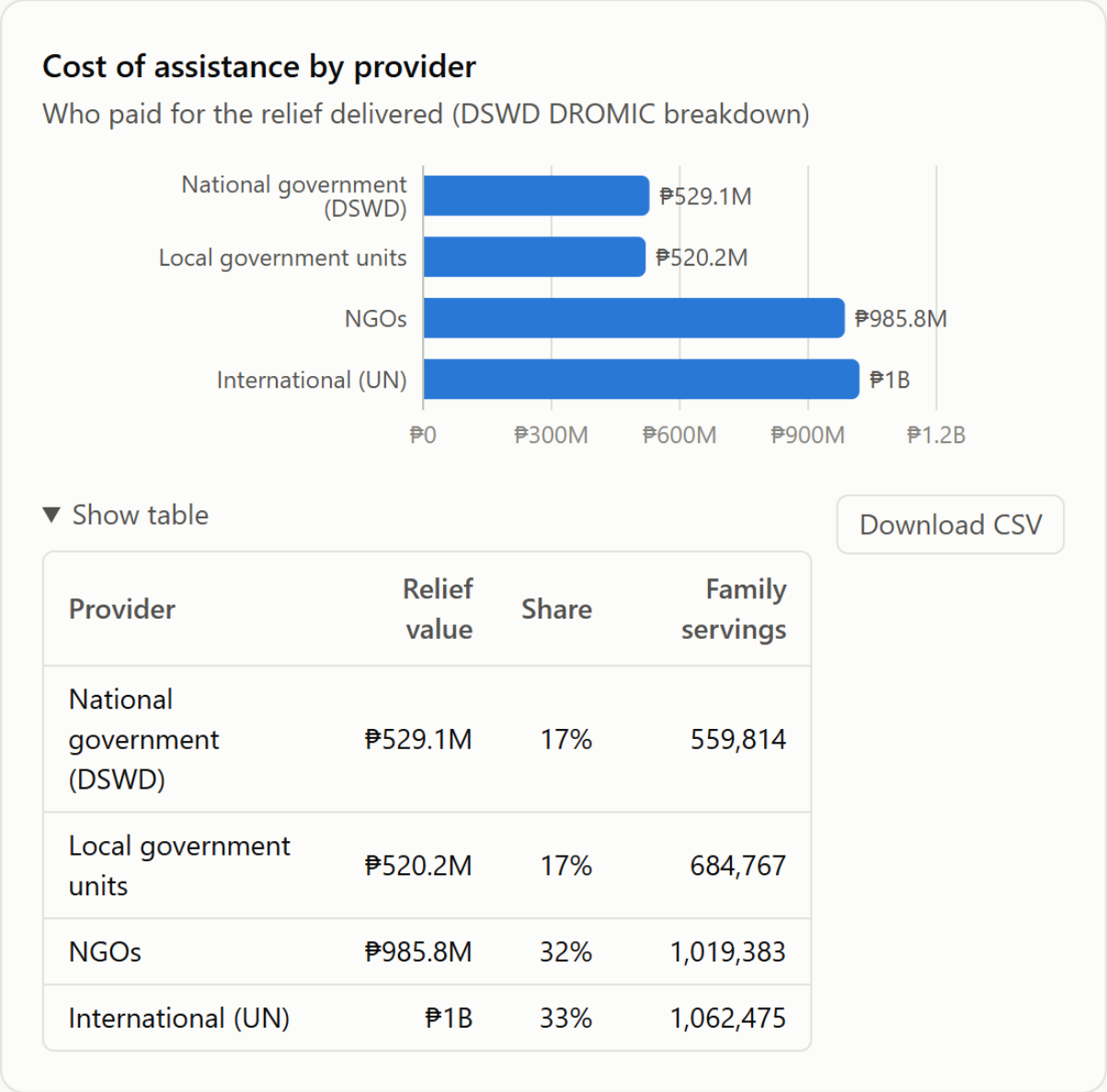
The figures cover all 31 events.

Part of the screen	What it shows and how to read it
Headline tiles	**₱3.1B** of relief in **8,306** distributions; **3.3M** family servings (a family served twice counts twice); **₱264.6** per affected person; **1.27** servings per affected family; **53** provinces reached.
Relief value by organization	One bar per organization, stacked by resource category in standard units valued at PHP. The six providers, including local government units (MDRRMO/CDRRMO/PDRRMO), each delivered between ₱483M and ₱535M. Food is 68–71% of every partner's value, and 88% for LGUs, which hand out family food packs first.
Cost of assistance by provider	Relief value by type of provider, as DSWD DROMIC reports it (close-up in Figure 17).
Least-served provinces	The ten provinces with the lowest relief value per affected person. Tarlac (₱138.8) and Davao Oriental (₱140.4) are lowest, about half the overall average of ₱264.6.
Relief coverage by province	Every province, sorted from least to most relief per person, with servings per affected family.

What it means for decisions. Relief is spread evenly across partners, but not across provinces: the bottom of the coverage ranking is where the next allocation should go first.

Close-up: cost of assistance by provider

Figure 17 enlarges the cost-of-assistance card.



Provider	Relief value	Share	Family servings
National government (DSWD)	₱529.1M	17%	559,814
Local government units	₱520.2M	17%	684,767
NGOs	₱985.8M	32%	1,019,383
International (UN)	₱1B	33%	1,062,475

Figure 17: Close-up of the cost of assistance by provider

Relief value and families reached by national government, LGUs, NGOs and international organizations, grouped as DROMIC reports them.

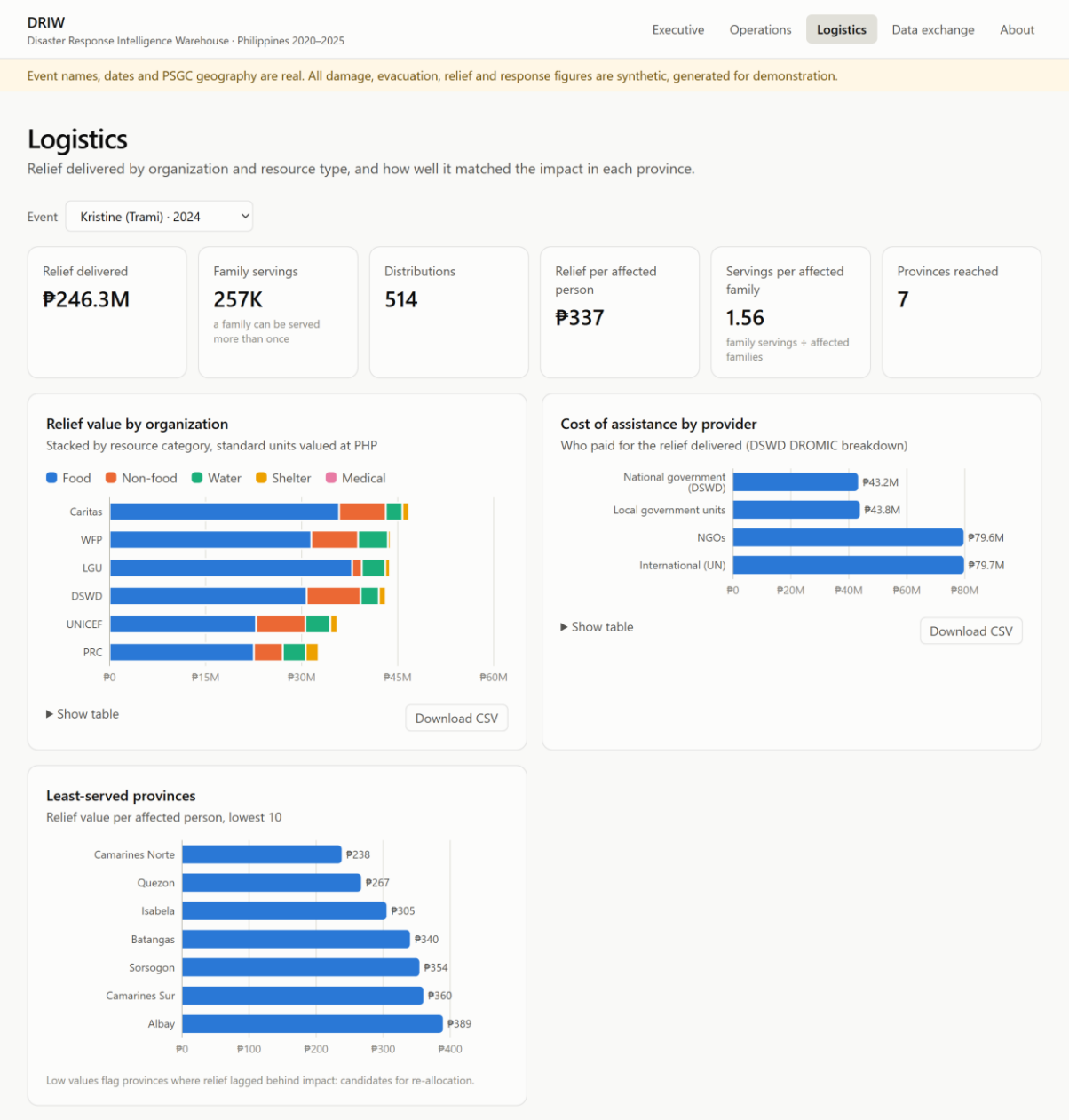
DROMIC reports always state who paid for the assistance: DSWD (national government), the local government units, NGOs, and other partners. This card groups every organization by that type and shows the value each type delivered and its share. Over all events, international organizations (UN agencies: WFP and UNICEF) delivered ₱1.0B

(33%), NGOs (Philippine Red Cross, Caritas) ₱985.8M (32%), national government (DSWD) ₱529.1M (17%) and LGUs ₱520.2M (17%). *Family servings* shows how many families each type reached.

Two readings matter for coordination. LGUs reach many families (684,767 servings) with a smaller budget, because they give mostly food packs in the first week. In real operations DSWD usually carries the largest share; the even split here comes from the synthetic generator choosing partners at random. The event selector shows the same split for one event: for Kristine, NGOs and international partners gave about ₱80M each, and DSWD and LGUs about ₱43M each.

Logistics filtered to one event

Figure 18 shows the same page filtered to one event.



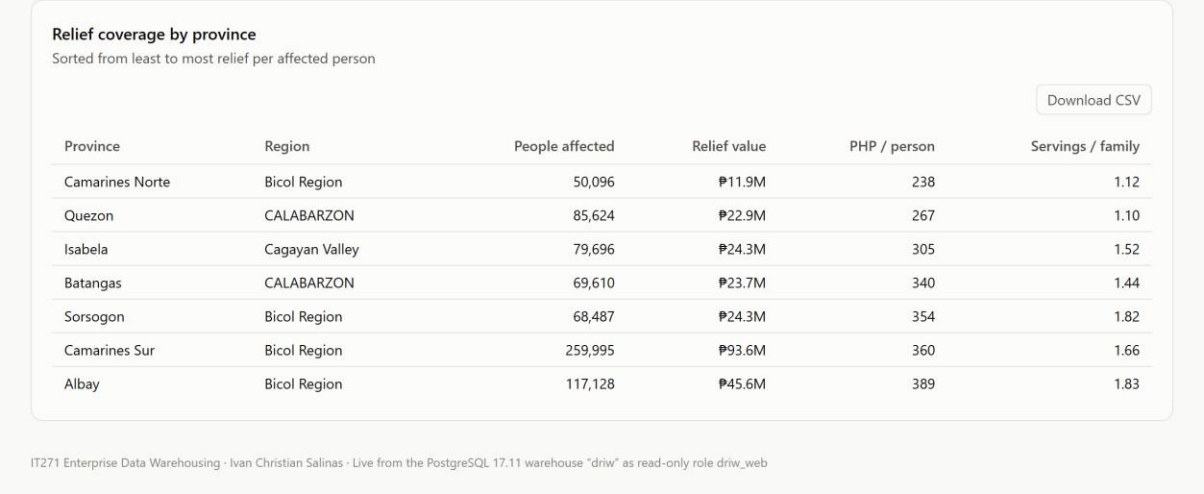


Figure 18: Logistics dashboard filtered to Typhoon Kristine

Every tile, chart and table re-computes for the selected event, which shows the gaps between provinces within one storm.

The same page sliced to Kristine through the event selector (`/logistics/?event=TC2024-KRISTINE`). Every tile, chart and table re-computes for that event: ₱246.3M in 514 distributions across 7 provinces, ₱337 per affected person. Within one storm the gap is sharper: **Camarines Norte** received ₱238 per person and 1.12 servings per affected family, while **Albay** received ₱389 and 1.83. A logistics officer reading this during the response would redirect the next convoys to Camarines Norte.

About the warehouse

Figure 19 shows the About page.

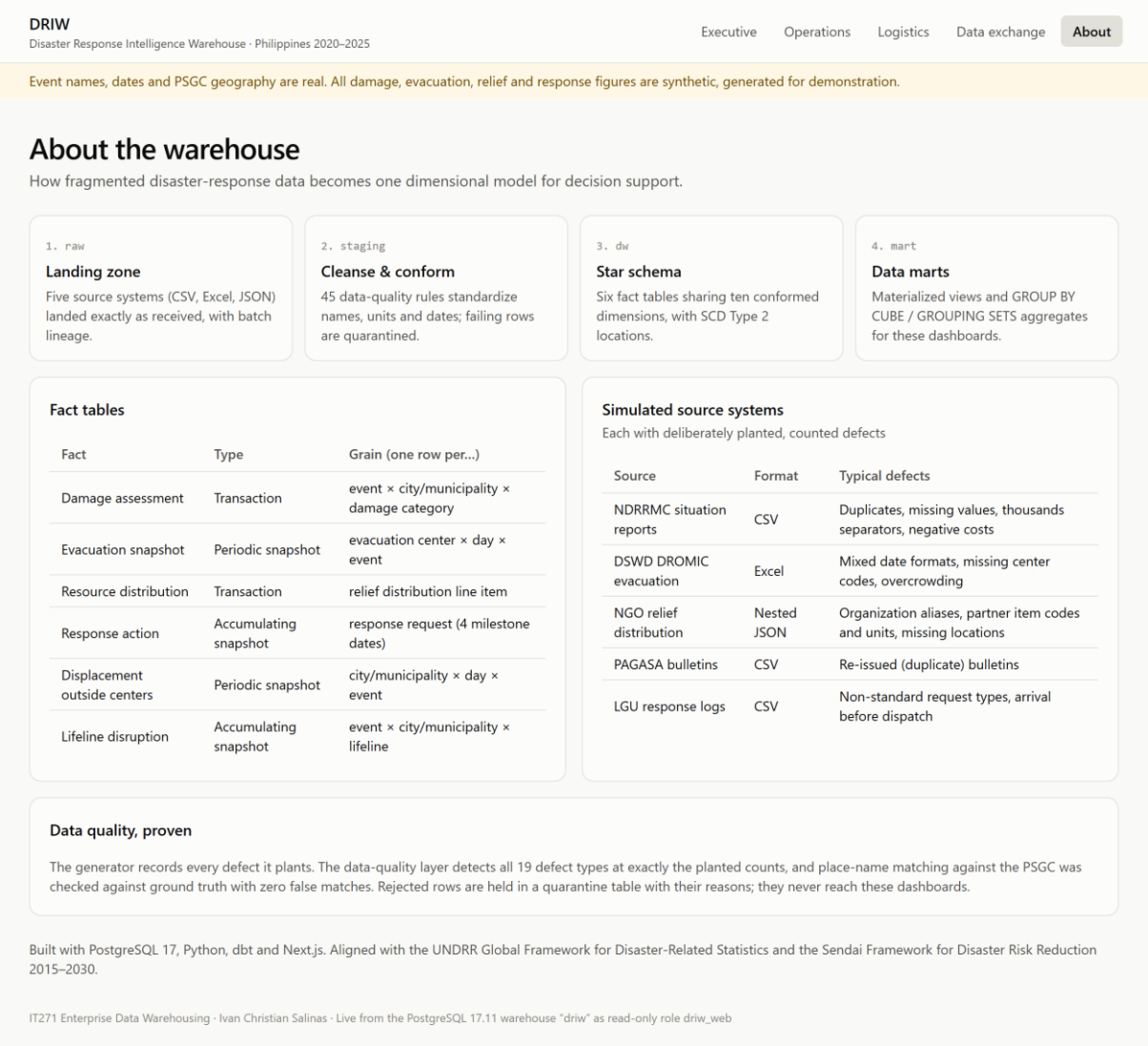


Figure 19: About page

A plain-language explanation of the layers, fact tables, sources and data-quality proof for non-technical readers.

A plain-language explanation of the system for any visitor, especially non-technical stakeholders: the four layers (`raw`, `staging`, `dw`, `mart`), the six fact tables and their grain, the five source systems with the defects each one carries, and the data-quality result (all 19 planted defect types detected exactly; zero false place-name matches). It answers "can I trust these numbers?" before anyone asks.

The PHP API (back-end)

Figure 20 shows the health check of the API.

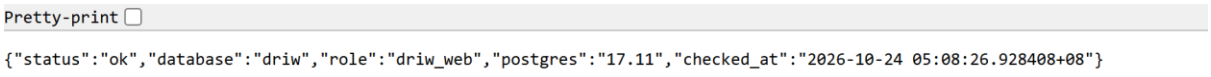


Figure 20: API health check

The PHP back-end confirms the database, its version, and that it connects as the read-only role `driw_web`.

Figure 21 shows one of its responses: the event summary for Kristine as JSON.

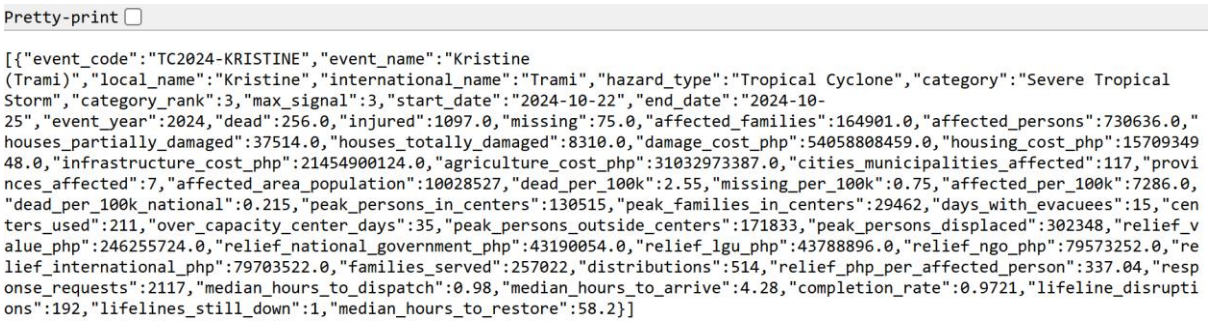


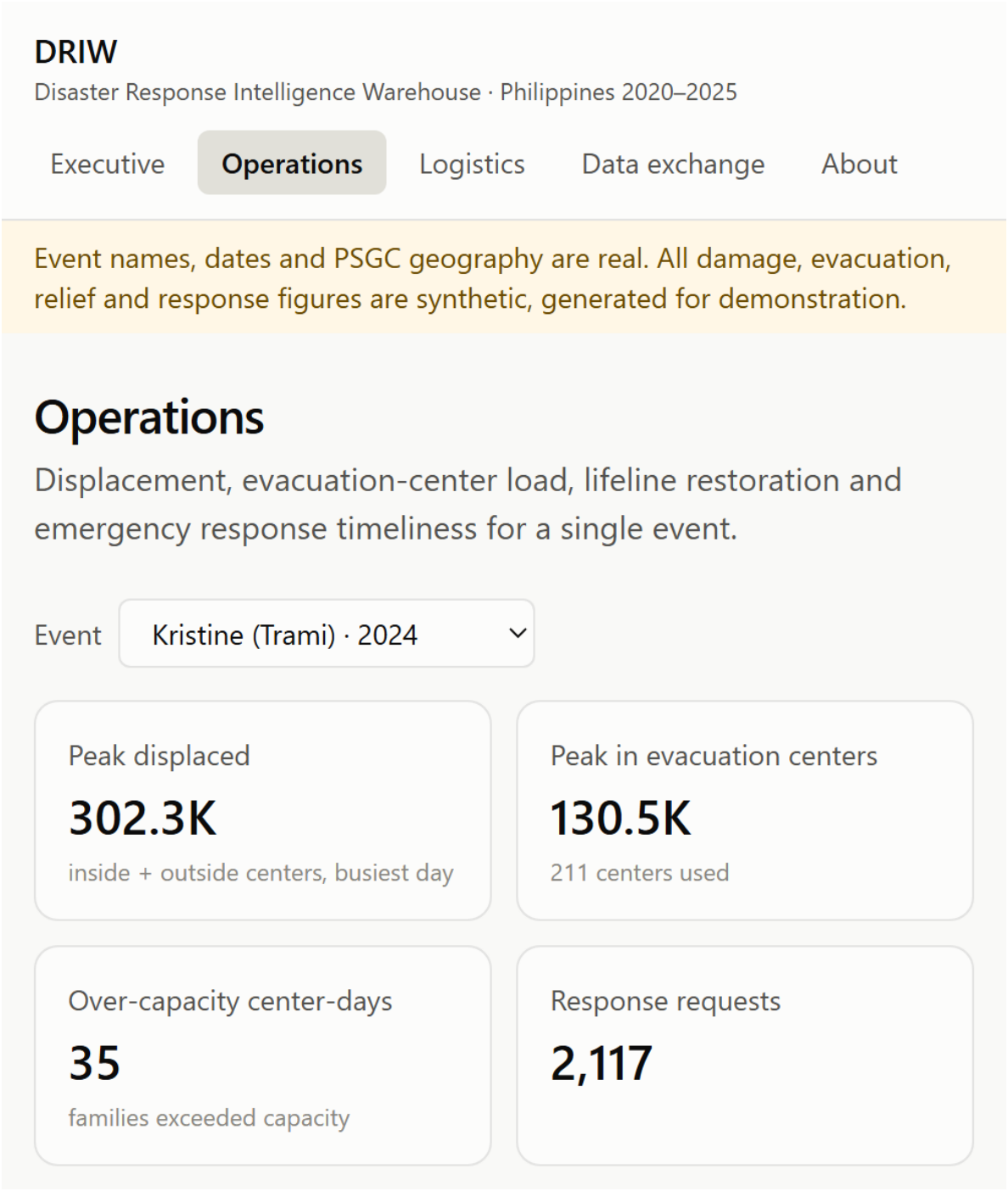
Figure 21: API response for the event summary mart

The same JSON the dashboards read, here for Kristine, available to any other system.

The dashboards are static pages; their data comes from two PHP endpoints. `health.php` (first image) confirms the connection: database `driw`, PostgreSQL 17.11, connected as `driw_web`. That role is a member of `driw_executive` only, so the web server can read the marts and nothing else (least privilege). `marts.php` (second image) returns one mart as JSON, here `mart_event_summary` for Kristine. Any other system can consume the same endpoint: an agency information system, or Excel and Power BI through a web query.

Narrow screens

Figure 22 shows the Operations page on a phone-sized screen.



Median time to dispatch

59 min

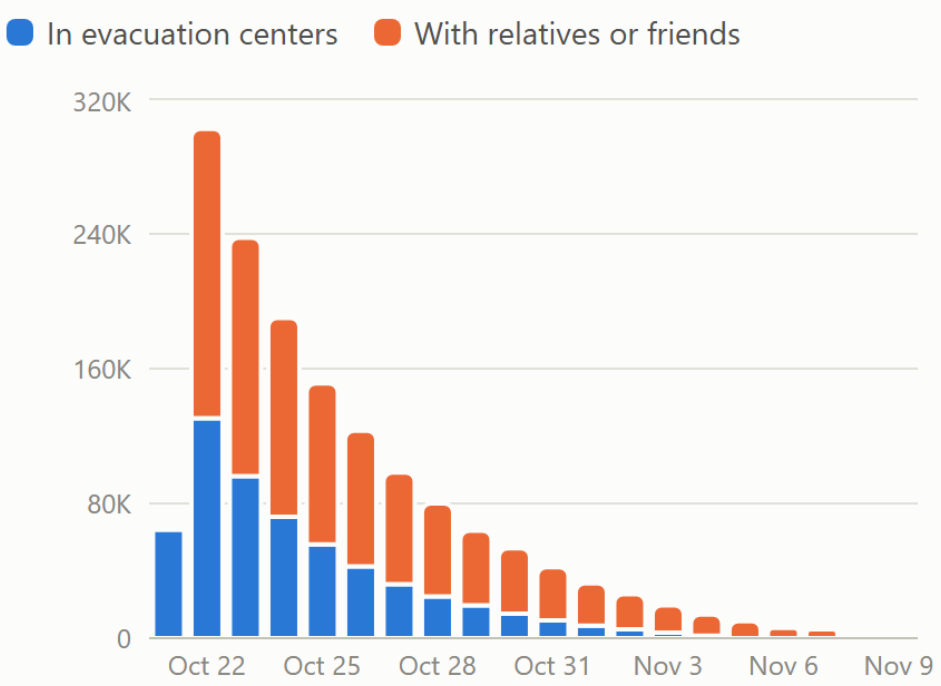
Lifelines disrupted

192

median 58.2 h to restore · 1 still down

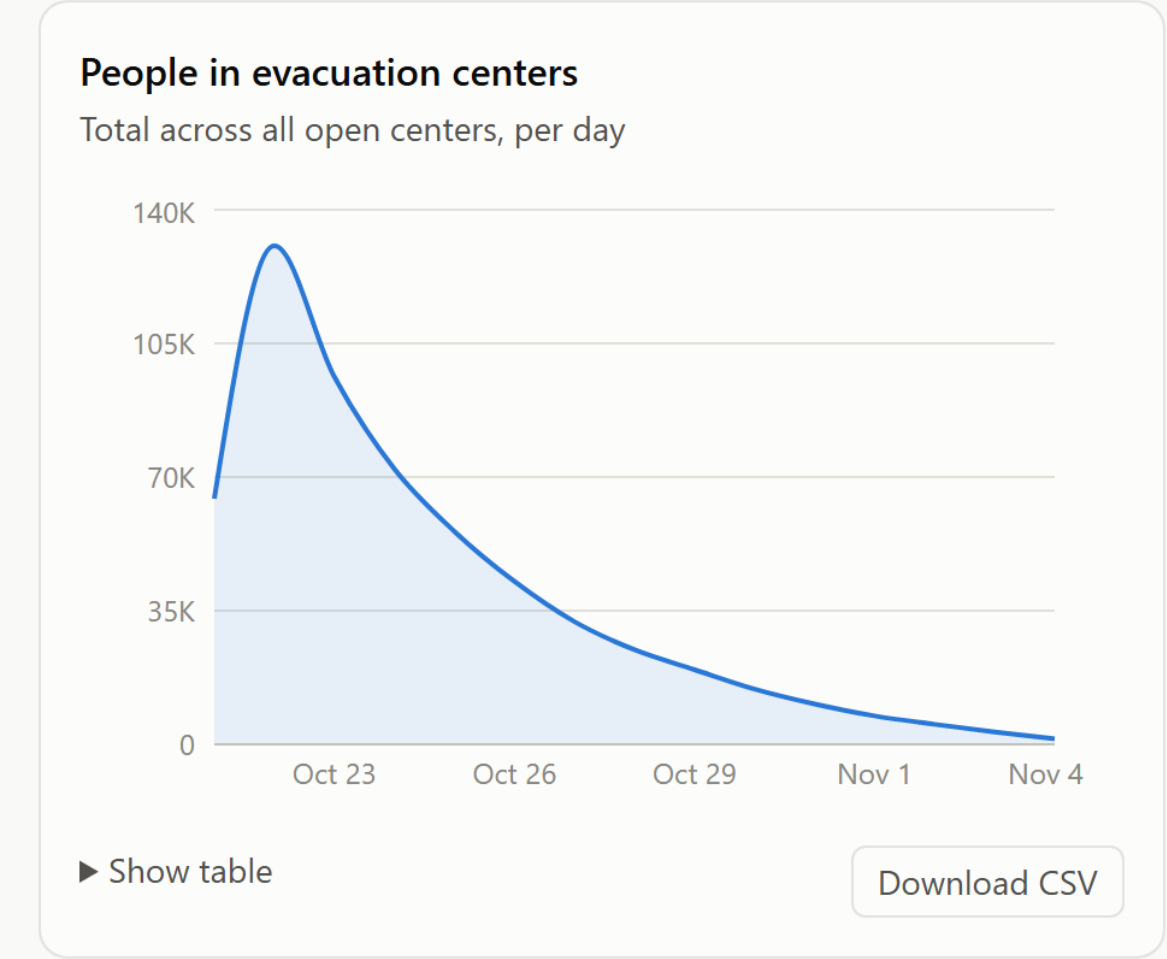
People displaced, inside and outside evacuation centers

Per day · most displaced families stay with relatives or friends (DSWD DROMIC)



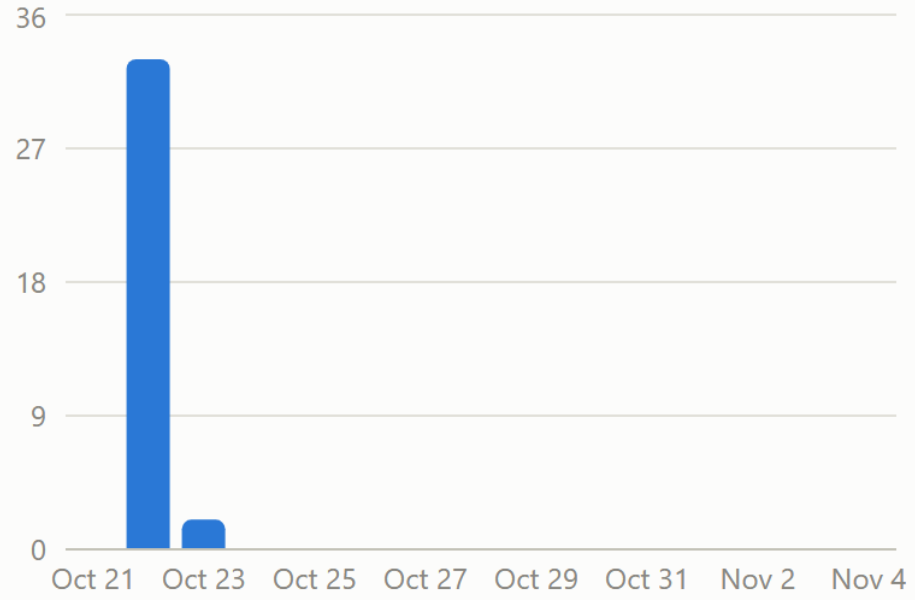
▶ Show table

Download CSV


People in evacuation centers
Total across all open centers, per day
140K
105K
70K
35K
0
Oct 23
Oct 26
Oct 29
Nov 1
Nov 4
▶ Show table
Download CSV

Centers over capacity

Number of centers where families exceeded capacity, per day

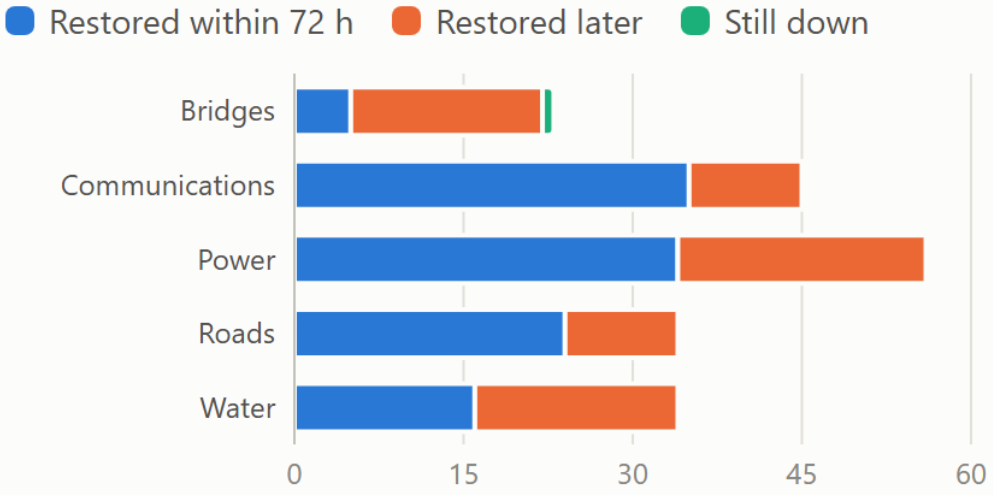


▶ Show table

Download CSV

Lifelines: roads, bridges, power, water, communications

Cities and municipalities where each lifeline went down, by how fast service came back



▶ Show table

Download CSV

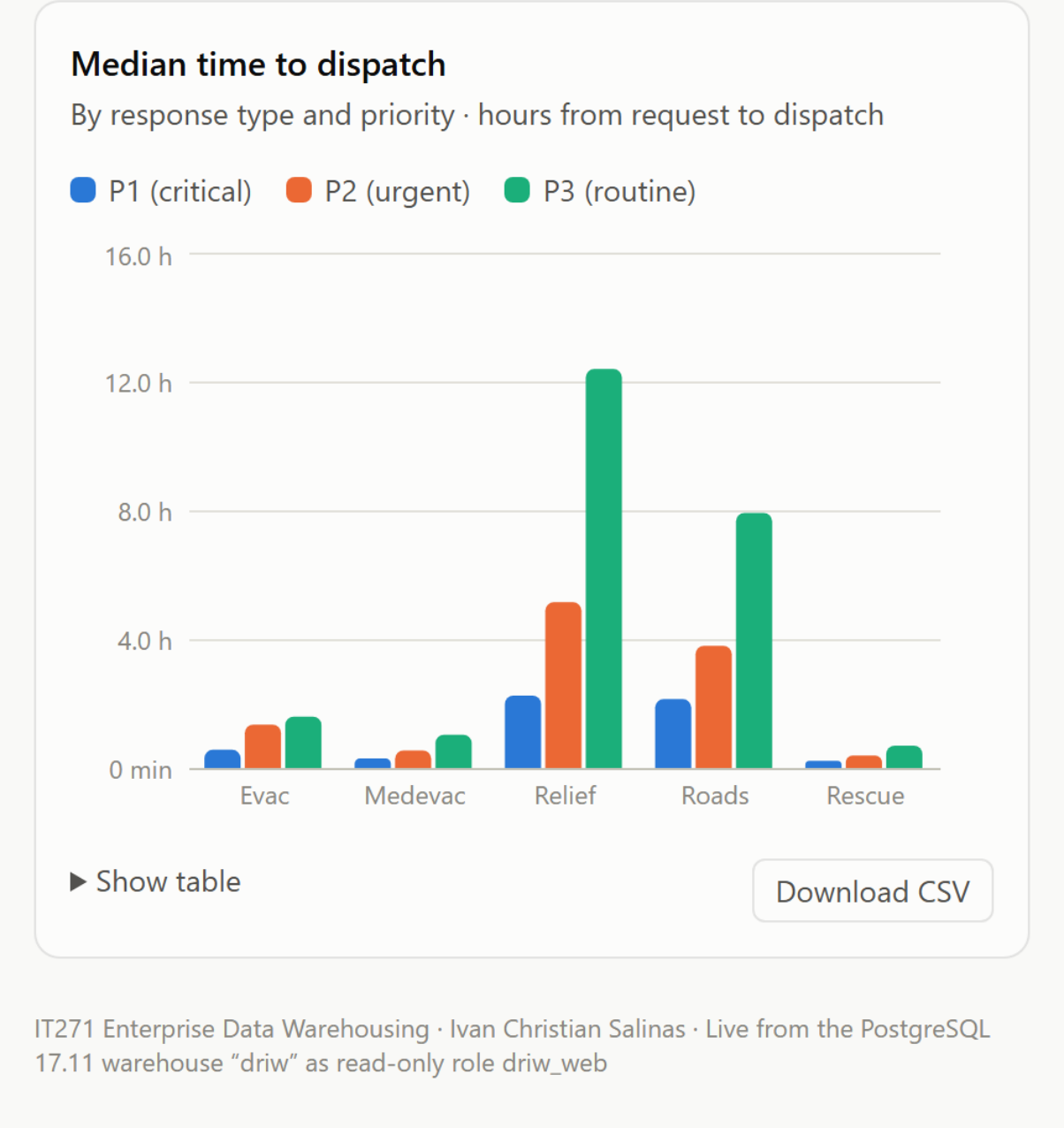


Figure 22: Operations page on a narrow screen

Tiles and charts reflow into one or two columns, so field staff can use the dashboards on a phone or tablet.

The same Operations page on a narrow screen. Tiles fall into two columns and charts stack in one column, with no sideways scrolling, so field staff can use the dashboards on a tablet or phone during operations.

7.3 Sample findings (synthetic data)

The findings in this section show the kind of answer each dashboard gives. Because the operational figures are synthetic, they demonstrate analytical capability, not facts about real disasters.

- **Kristine (2024)** was the costliest event at about ₱54 billion; total damage across the 31 events is ₱558.3 billion.
- **Bicol Region** had the most people affected (about 2.6 million), mainly from tropical cyclones.
- Evacuation peaks on landfall day and decays over about two weeks; overcrowding is concentrated in the first two days, which is when extra centers should open.
- **Relief delivery and road clearing** wait longest for dispatch, especially at routine priority; rescues go out fastest.
- **Tarlac** and **Davao Oriental** received the least relief per affected person (about ₱139–140), candidates for re-allocation.
- Most displaced people are **outside evacuation centers**: 61% on the peak days, and they stay displaced longer than people in centers.
- Relative to population, **Odette (2021)** was the deadliest event (3.88 deaths per 100,000), not the costliest one.
- **Bridges** take longest to restore (median 126 hours across events, only 20% within 72 hours); communications come back fastest (86% within 72 hours).

These figures come from synthetic operational data and illustrate the analysis, not real outcomes.

8. Testing and Validation

How DRIW was tested, and the results. Data seed 271, run on PostgreSQL 17.11.

8.1 Summary

Table 62 summarizes every test suite and its result.

Table 62: Test suites and their results

Every suite passes; together they cover completeness, correctness, consistency, model integrity, reproducibility, data exchange and security.

Suite	Checks	Result
dbt build (39 models, 2 seeds, 76 data tests)	117	**117 pass**
End-to-end tests (`tests/test_end_to_end.py`)	36	**36 pass**
Governance tests (`tests/test_governance.py`)	27	**27 pass**
PHP API tests (`tests/test_php_api.py`)	15	**15 pass** (skipped where PHP is not installed, e.g. CI)
Data exchange tests (`tests/test_intake.py`)	13	**13 pass**
Query performance (`ingestion/benchmark_queries.py`)	14 queries	**14 within the 1 s budget**, slowest about 34 ms
Dashboards (`npm run lint`, `npm run build`)	lint + type check + static build	**pass**

Run everything with `.venv/Scripts/python -m pytest tests`. It builds a brand-new database (`driw_e2e`) from nothing through the whole pipeline (migrations, reference data, PSGC, generation, raw load, dbt, export), runs every check, then rebuilds dbt a second time. Duration: about one and a half minutes. GitHub Actions runs the same suite and the dashboard build on every push (`.github/workflows/ci.yml`). Query timings are in the Query Performance Test Report (Chapter 8).

8.2 What is validated

1. Completeness: nothing is lost between layers

- Every raw row reaches staging (7 raw tables from 5 sources; NGO checked at item-line level).
- Every staged row has a status; every rejected row is in quarantine.
- Every valid/warning row reaches its fact table (dbt `assert_facts_reconcile_to_staging`).
- Every layer, fact and mart is non-empty.

2. Correctness: data quality is proven, not assumed

- **19/19 planted defect types detected at exactly the planted count** (`dq.dq_reconciliation`).
- Place-name matching checked against ground truth: **0 false matches** across ~63,000 rows.
- Every rejection traces to a planted defect (`warn_unplanted_rejections`).

3. Consistency: aggregates agree with detail

- Damage totals are equal in facts, the event mart and the CUBE grand total.
- Every mart adds back up to its facts (`assert_marts_reconcile_to_facts`).
- Exported JSON row counts equal mart row counts, and the dashboard data committed in `web/src/data/` is identical to a fresh build (fails if someone forgets to re-export).

4. Model integrity

- Primary and foreign keys on the star schema (10 PKs, 38 FKs).
- One row per declared grain in each fact.
- SCD Type 2: no overlapping or missing validity periods, exactly one current version; Typhoon Paeng (Oct 2022) rolls up to Maguindanao del Norte/del Sur.

5. Reproducibility

- The generator is deterministic: two runs with the same seed produce identical content.
- A fresh database built from scratch matches the working database row for row (111,763 fact rows, 1,548 Bicol damage rows in both).
- A second dbt build gives identical fact counts and keeps grants and row-level security.

6. Data exchange (see Chapter 10)

- Every Excel template has its instructions, the expected headings and drop-down lists.
- The Albay sample, in Albay's own layout, gets the expected verdict on every one of its 10 rows.
- Checking a file never changes the warehouse; a file missing required columns is refused whole.
- A submitted file loads with every warehouse check still passing: valid rows reach the facts, the rest are quarantined with reasons, and the synthetic-data reconciliation stays 19/19.
- Submitting the same file again replaces it instead of duplicating it.

- The Excel export has every mart, a data dictionary, frozen headings and filters.

7. Security (see Chapter 6)

- Executive reads marts only; denied on warehouse detail, staging, raw, quarantine.
- Analyst sees every fact row, marts and DQ scorecards; denied raw and quarantine.
- Regional officers (Bicol, Cagayan Valley) see exactly their region's rows in all six facts and in the secure views, and zero rows from other regions.
- An unmapped regional user sees zero rows (default deny).
- Regional users cannot read the mapping, grant themselves roles, or edit their mapping.
- Stewards work on raw, staging and quarantine, may fix alias maps, and are denied the warehouse.

8.3 Defects found by testing (and fixed)

Testing earned its keep. Each of these was caught by a test, not by inspection.

Table 63 lists the defects the tests found during development and how each was fixed.

Table 63: Defects found by testing, and their fixes

Each was caught by an automated test or a review, not by chance, which shows the tests do real work.

Found by	Problem	Fix
DQ reconciliation	Generator counted the placeholder "-" as a negative number	Count only real numbers as negative
Unplanted-rejection check	Monsoon incidents named "Southwest Monsoon (Habagat) Carina (Gaemi)" matched the first parentheses; 1,244 rows wrongly rejected	Match the last parentheses
Location accuracy test	Letter swaps inside "City of" (e.g. "Cityo f Legazpi") went unmatched	Letters-only fuzzy key
Governance tests	Regional users saw **zero** rows: a `SECURITY DEFINER` function sees its owner as `current_user`	Region lookup moved into a view
Governance tests	`SET ROLE` escalation test passed for the wrong reason (checked against the session user)	Test escalation via `GRANT` and mapping writes instead
Browser review	"Family coverage 101%" double-counted repeat distributions	Relabeled "servings per affected family"
Paper review	Logistics "Distributions" counted a distribution once per resource category (9,782 instead of 5,529)	Count from the event summary
Data exchange test	One impossible date ("10/32/2024") in a submitted file made `to_date` fail the whole staging model, which would block every load	Error-tolerant `meta.try_to_date` / `try_to_timestamp` and a range-checked integer cast; the row is flagged instead

Found by	Problem	Fix
Data exchange test	Rows that synthetic data never contains could break or silently drop out of a load (unknown responding unit, dispatch before request, unknown damage category, conflicting event dates)	Five new reject rules (36 → 41)
Browser test	Re-checking an already submitted file flagged all its rows as already reported	A resubmission now replaces its earlier version

8.4 Known limitations

- All operational figures are synthetic; magnitudes are calibrated to look plausible, not to match official NDRRMC totals.
- City population is estimated from barangay counts (the PSA workbook with census population could not be downloaded automatically).
- The PSGC source predates the 2022 Maguindanao split; the split is applied through the SCD2 change log.
- Seven misspelled place names are equally close to two real towns and stay unmatched by design, for a data steward to resolve.
- On XAMPP the dashboards read the marts live through the PHP API; elsewhere (a static host) they show the bundled snapshot.
- The Data exchange upload is limited to the server itself until the site has sign-in.

8.5 Query Performance Test Report

Generated by `ingestion/benchmark_queries.py` on 2026-10-25 03:30.

Method

- Every query in Appendix A runs once to warm the cache, then 5 times under `EXPLAIN (ANALYZE)`. Times are server-side planning + execution, in milliseconds.
- Acceptance criterion (NFR2, business case (Chapter 1)): every analytical query completes in under 1000 ms.
- Environment: PostgreSQL 17.11 with default settings on Windows 11, local laptop (12th Gen Intel(R) Core(TM) i5-12500H, 16 logical cores, 16 GB RAM); database and client on the same machine, no other load.
- Data volume (seed 271): fact_damage_assessment 8,806 rows, fact_evacuation_snapshot 31,110 rows, fact_resource_distribution 14,927 rows, fact_response_action 32,347 rows, fact_displacement_outside 21,896 rows, fact_lifeline_disruption 2,677 rows.

- Mart baseline: Q13 reads the event headline figures from `mart.mart_event_summary`; Q01 computes the same figures (ranking events by damage with their human toll) from `dw.fact_damage_assessment`. The ratio of their median times is the speed-up quoted below.
- Caveats: times are from one machine and include no network transfer; a warm cache favours repeated dashboard queries. A mart read takes well under 0.1 ms, so its time, and with it the speed-up, varies between runs (between about 60× and 80× in repeated runs); this report shows the latest run.

Results

Table 64 gives the timing of every query.

Table 64: Timing of the 14 sample analytical queries

Each query is tied to a business question and an OLAP operation; Median ms is the typical time over 5 runs and Slowest ms the worst run, compared with the 1000 ms budget.

Query	Business question	OLAP operation	Rows	Median ms	Slowest ms	Within budget
Q01	Costliest events and their human toll (BQ1)	Ranking (top-N)	10	3.60	3.64	Yes
Q02	Damage by year and category, with yearly subtotals (BQ2)	Roll-up (GROUP BY ROLLUP)	31	4.57	5.00	Yes
Q03	Drill-down: region -> province -> city/municipality for one event (BQ3)	Drill-down	15	1.30	1.54	Yes
Q04	Tropical cyclone vs monsoon flood impact per region (BQ4)	Pivot (FILTER) / dice	15	4.32	4.48	Yes
Q05	Every subtotal of impact by year x region x hazard (BQ2, BQ4)	Cube (GROUP BY CUBE)	160	11.34	12.11	Yes
Q06	Peak evacuees per event (semi-additive measure) (BQ5)	Aggregate over a snapshot	10	4.89	5.06	Yes
Q07	Evacuation centers most often over capacity (BQ6)	Slice (is_over_capacity)	10	17.54	17.90	Yes
Q08	Dispatch time by response type and priority (BQ7)	Accumulating snapshot analysis	15	30.93	31.59	Yes
Q09	Slowest-responding cities/municipalities for life-saving requests (BQ7)	Slice + ranking	10	31.80	33.19	Yes
Q10	Least relief per affected person, by province (BQ8)	Integration of two facts	10	7.88	7.98	Yes
Q11	Relief mix by organization and resource category (BQ9)	Dice (org x category)	29	7.46	7.58	Yes
Q12	Year-over-year change in damage cost (BQ2)	Window function (LAG)	6	2.42	2.54	Yes
Q13	Dashboard query served from a mart (BQ1, BQ5, BQ8)	Pre-aggregated ROLAP mart	10	0.05	0.07	Yes
Q14	What data was rejected, and why? (BQ10)	Data-quality audit	12	0.23	0.24	Yes

Findings

- **14 of 14 queries** meet the 1000 ms budget on every run; the slowest single run was Q09 at 33.2 ms.
- **Marts pay off:** the event headline read from `mart.mart_event_summary` (Q13, 0.05 ms) is about 78× faster than aggregating the same figures from the fact table (Q01, 3.60 ms). This is why the dashboards read marts, not facts.
- **Sequential scans are the right plan at this size.** With tens of thousands of rows per fact, PostgreSQL reads each table in a few milliseconds and chooses sequential scans over indexes. At national scale (millions of rows) the next step is B-tree indexes on the fact foreign keys most used in filters (`event_key`, `location_key`, date keys) and partitioning facts by event year; this benchmark is the baseline to measure that against.
- Plans per query (scan types used):
- Q01: Seq Scan
- Q02: Seq Scan
- Q03: Seq Scan
- Q04: Seq Scan
- Q05: Seq Scan
- Q06: Seq Scan
- Q07: Seq Scan
- Q08: Seq Scan
- Q09: Seq Scan
- Q10: Seq Scan
- Q11: Seq Scan
- Q12: Seq Scan
- Q13: Seq Scan
- Q14: Seq Scan

9. Implementation and Live Demo

9.1 Technology stack

Table 65 lists the technology used in each layer.

Table 65: Technology stack

Every component is open source or free and runs on one laptop or an agency server.

Layer	Technology
Data warehouse	PostgreSQL 17.11 (local; PostgreSQL Global Development Group, 2024)
ETL	Python 3.13 (extraction, loading), dbt Core 1.12 (transformation, data quality, tests; dbt Labs, n.d.)
OLAP	ROLAP in PostgreSQL: star schema, materialized views, `CUBE` / `ROLLUP` / `GROUPING SETS`
Back-end	PHP 8.1 on XAMPP (Apache) with `pdo_pgsql`
Dashboards	Next.js (static build) served by Apache
Version control and CI	GitHub (`sardinexszc/NEUSTIT271_EDW_Project`), GitHub Actions

9.2 Repository layout

Table 66 describes the folders of the repository.

Table 66: Repository layout

Each folder holds one part of the system; the documentation and its generators live with the code.

Folder	Contents
`ingestion/`	Python pipeline: migrations, reference loads, synthetic generator, raw loader, export, benchmark, web role
`supabase/migrations/`	SQL schema history (schemas, raw tables, reference tables, roles, governance)
`dbt/`	Staging, data-quality, warehouse and mart models; tests; rule catalog
`data/reference/`	Real reference data (events, PSGC-derived geography, organizations, relief items)
`web/`	Dashboards (Next.js) and the PHP API (`web/php/api`)
`tests/`	End-to-end, governance and API tests (pytest)
`docs/`	This documentation, diagrams and their generators

9.3 Build and run

```
# 1. Python environment and database connection (.env holds PG* settings)
python -m venv .venv
.venv/Scripts/pip install -r requirements.txt

# 2. Build the warehouse end to end (about 1 minute)
.venv/Scripts/python ingestion/pipeline.py

# 3. Validate
.venv/Scripts/python -m pytest tests

# 4. Excel templates and the Data exchange catalog (after the warehouse is built)
.venv/Scripts/python ingestion/intake.py templates

# 5. Web back-end login + PHP config (once), then publish to XAMPP
.venv/Scripts/python ingestion/setup_web_role.py
cd web && npm install && npm run deploy:xampp
```

XAMPP's `php.ini` needs `extension=pdo_pgsql` and `extension=pgsql` enabled, and Apache restarted.

9.4 Live demo script

1. **Warehouse.** In pgAdmin or psql, show the layers (`raw` → `staging` → `dw` → `mart`) and the ER diagram of a star (pgAdmin draws it from the real PK/FK constraints).
2. **Data quality.** `select * from dq.dq_reconciliation;` shows all 19 planted defect types detected at the exact count; `select * from dq.dq_quarantine limit 10;` shows rejected rows with reasons.
3. **OLAP.** Run Q02 (roll-up), Q03 (drill-down), Q05 (cube) and Q06 (semi-additive peak) from `docs/sample-queries.sql`.
4. **SCD Type 2.** `select * from dw.dim_location where city_muni_name = 'Parang' and psgc_code like '19038%';` shows Maguindanao before and Maguindanao del Norte after 18 Sep 2022.
5. **Security.** `begin; set local role driw_demo_bicol; select count(*) from dw.fact_damage_assessment; rollback;` returns only Bicol's rows; as `driw_demo_executive` the same query is refused.
6. **Dashboards.** Open `http://localhost/driw/`: Executive, then Operations for Kristine (`/driw/operations/?event=TC2024-KRISTINE`), then Logistics. Point out the footer ("Live from the PostgreSQL warehouse … as read-only role driw_web") and Download CSV.

7. **Data exchange.** Open `http://localhost/driw/exchange/`, download a template, then check `data/intake-samples/albay-pdrrmo-kristine-evacuation.xlsx` with the *Albay PDRRMO* layout: the report shows names matched to the PSGC, a recovered center code, overcrowding, and three rejected rows with reasons. Download the Excel workbook for Kristine.
8. **API.** `http://localhost/driw/api/health.php` and `http://localhost/driw/api/marts.php?name=mart_event_summary`.

10. Enterprise Positioning and Data Exchange

10.1 Why an enterprise data warehouse has no content management system

A content management system (CMS) is for creating and editing content: web pages, documents, records typed in by users. An enterprise data warehouse (EDW) has a different job. It **receives** data from the systems where data is entered, **integrates** it, **keeps its history**, and serves it for analysis. The three kinds of system sit in a chain.

Table 67 places the three kinds of system in a chain.

Table 67: Source systems, warehouse and business intelligence as a chain
Data is entered upstream, integrated in the warehouse and presented downstream; the warehouse itself is read-only.

Stage	Purpose	In disaster response
Source (operational) systems	Where data is entered and edited day to day	DROMIC workbooks, NDRRMC situation reports, LGU response logs, NGO distribution records, PAGASA bulletins
Enterprise data warehouse	Integrate, clean, historize and govern; read-only for its users	**DRIW**
Business intelligence	Present and explore	The DRIW dashboards, CSV and API exports

DRIW deliberately has no data-entry screens. Data can enter only through the controlled ETL pipeline, so every value is validated by the 45 data-quality rules, traceable to the file and row it came from, and reproducible by re-running the pipeline. A screen that let users edit warehouse figures would break all three guarantees. Where a CMS or portal belongs is upstream, as a way for an LGU to *submit* a file, which is what the Data exchange page in section 10.2 provides, or downstream, to publish reports. It is never the warehouse itself.

What makes DRIW an enterprise data warehouse

Inmon's classic definition (Inmon, 2005) calls a data warehouse subject-oriented, integrated, time-variant and non-volatile. DRIW meets each one.

Table 68 gives the evidence for each characteristic.

Table 68: Inmon's characteristics of a data warehouse, and the evidence in DRIW
DRIW meets all four.

Characteristic	Evidence in DRIW
Subject-oriented	Organized around six subjects of disaster response (damage, evacuation, displacement outside centers, relief, response, lifelines), not around any one agency's forms.
Integrated	Five sources in three formats conformed to one geography (PSGC codes), one event list (PAGASA names and aliases), one set of organizations and one set of relief units.
Time-variant	Six years of events (2020–2025); every fact is dated; locations keep their history (SCD Type 2, e.g. the 2022 Maguindanao split).
Non-volatile	Users can only read; loads happen through the pipeline; a full rebuild reproduces the same warehouse (a test enforces this).

"Enterprise" means the warehouse serves the whole organization, across agencies, from one governed model. DRIW has the capabilities that implies.

Table 69 lists those capabilities and where each is described.

Table 69: Enterprise capabilities and where DRIW provides them
Each capability points to the chapter that describes it.

Enterprise capability	Where it is in DRIW
Integration of many source systems	Five simulated agency systems (Chapter 4)
Conformed dimensions across subjects	Bus matrix: date, location and event shared by all six facts (Chapter 2)
Metadata and lineage	Data dictionary generated from the catalog; every row traces to its source batch (Chapter 3)
Data-quality management	Rule catalog, quarantine, reconciliation 19/19 (Chapter 5)
Security and governance	Four roles, row-level security by region, read-only web role (Chapter 6)
Serving many audiences	Executive, Operations and Logistics dashboards; CSV, Excel and API exports (Chapter 7, section 10.2)
Data exchange with LGUs and agencies	Excel templates, layout profiles, checked uploads, Excel export (section 10.2)
Reliability	208 automated checks (117 dbt, 91 pytest), run on every change (Chapter 8)
Room to grow	Standard PostgreSQL; the same migrations run on a cloud database such as Supabase

10.2 Data exchange with LGUs and national agencies

DRIW exchanges data with LGUs and national DRRM offices in Excel. An office downloads a template (or keeps its own layout), checks its report against the warehouse rules, submits it, and downloads the warehouse's data back as a formatted workbook. All of it happens on one page of the web application, **Data exchange** (`http://localhost/driw/exchange/`).

This is an exchange **mechanism**, not a live integration. No agency system is connected to DRIW: offices exchange files, and the mechanism was tested with sample files written in the layouts of two offices (Albay PDRRMO and a Quezon City incident log), not with files submitted by those offices. Connecting real systems would need data-sharing agreements, sign-in for LGU users and scheduled loads (section 10.4).

Table 70 describes each capability of the page.

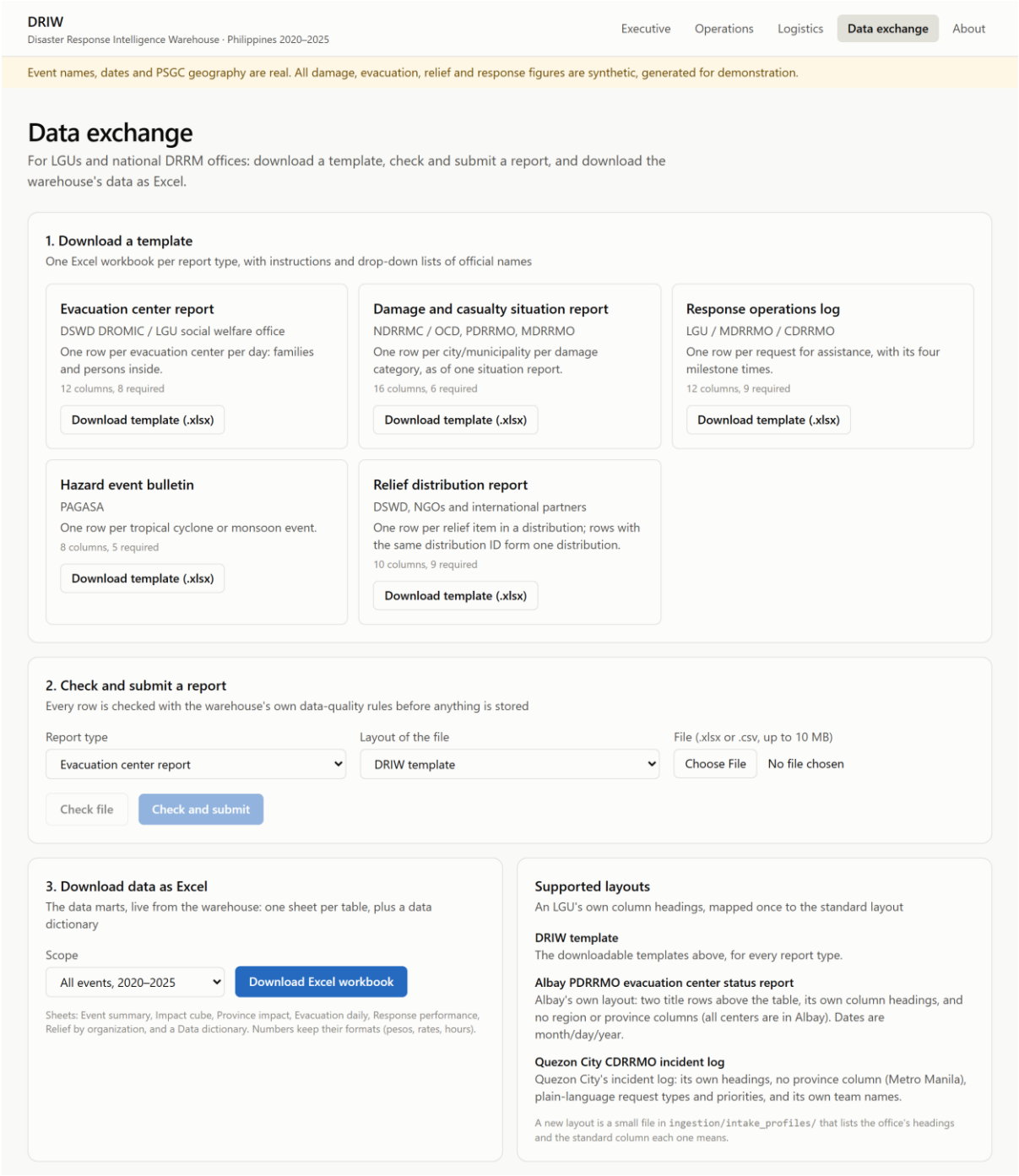
Table 70: Capabilities of the Data exchange page

An office downloads a template, checks and submits its report, and downloads the warehouse's data back as an Excel workbook.

Capability	How it works	Where
Excel templates	One workbook per report type (evacuation center report, damage and casualty sitrep, response operations log, hazard event bulletin, relief distribution report). Each has an *Instructions* sheet (every column, whether it is required, an example), a *Report* sheet with the headings, and drop-down lists of official names taken from the warehouse: provinces, cities and municipalities (PSGC), events, organizations, relief item codes and units.	`web/public/templates/`, generated by `ingestion/intake.py templates`
Layout profiles	An office whose report has its own headings, title rows, missing columns or its own codes is described once in a small file: which of its headings means which standard column, default values for columns it leaves out (e.g. province), and translations of its values (e.g. "Critical" → P1, "QC Rescue" → MDRRMO Rescue Team). Two examples ship: the Albay PDRRMO evacuation report and the Quezon City CDRRMO incident log.	`ingestion/intake_profiles/*.json`
Check	The upload is read with its layout, then checked in two stages. *Submission rules* catch problems that would break or distort a load: required values, unreadable or out-of-range dates, IDs repeated in the file or already in the warehouse, and evacuation reports already filed. Then the warehouse's **own** data-quality rules run on it: the rows are loaded into a separate sandbox database and the same dbt staging models run there, so the verdict is exactly what the warehouse would decide. Checking never changes the warehouse.	`ingestion/intake.py validate`, `web/php/api/intake.php`
Per-row report	Every row gets a status (accepted, corrected, warning, rejected, duplicate) and plain-language reasons: what was matched ("Legaspi City" → City of Legazpi), recovered, converted or translated by the layout, and why a row was rejected. The submitting office can download the report as CSV.	Data exchange page

Capability	How it works	Where
Submit	The file is checked again, saved in the standard layout under `data/intake/`, and the warehouse is reloaded with all its tests. Valid rows reach the fact tables; rejected rows go to quarantine with their reasons. Submitting a corrected file with the same name replaces the earlier version.	`ingestion/intake.py submit`
Excel export	All eight data marts as one workbook, live from the warehouse, for all events or one: an *About* sheet, one sheet per mart with styled, frozen headings, filters and number formats (pesos, percentages, hours), and a *Data dictionary* sheet describing every column.	`ingestion/export_xlsx.py`, `web/php/api/export.php`

Figure 23 shows the page.



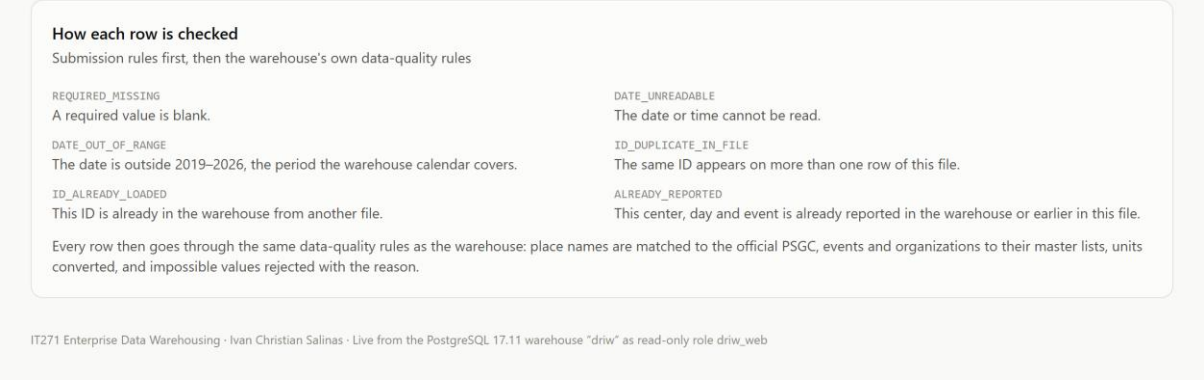


Figure 23: Data exchange page

Templates, the upload form, Excel export, the supported office layouts and the checking rules on one page.

Example: an LGU report in its own layout

The Albay PDRRMO keeps its evacuation report in its own workbook: two title rows, its own column headings, no region or province columns, and dates written month/day/year. With the Albay layout selected, DRIW reads it unchanged and returns this report.

Figure 24 shows the report DRIW returns.

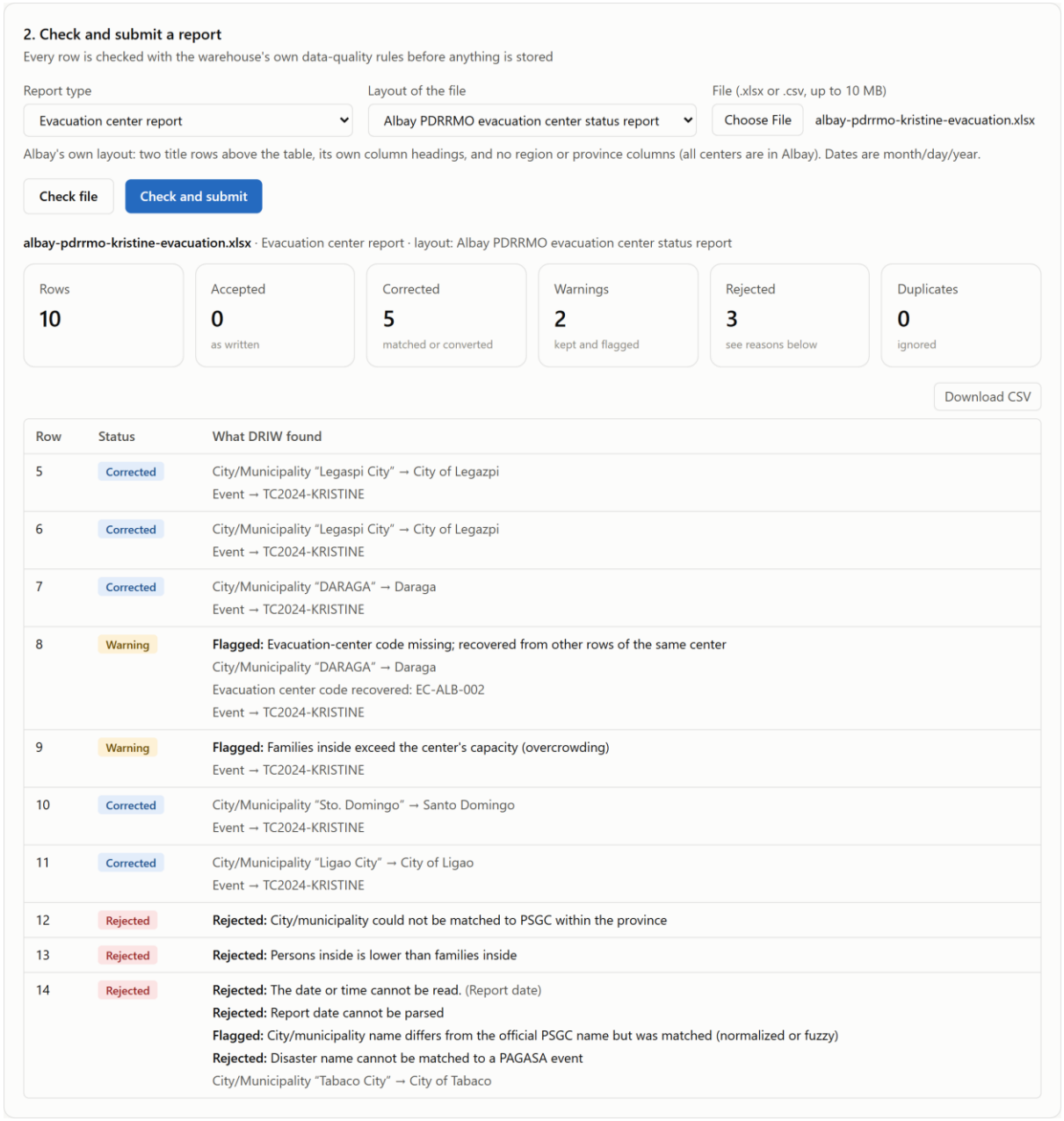


Figure 24: Check of the Albay PDRRMO evacuation report

A summary of the verdicts, then every row with its status and the reasons in plain language.

Table 71 explains the verdict on each row shown in Figure 24.

Table 71: Verdict on each row of the Albay report

Corrected and flagged rows still load; rejected rows go to quarantine with their reason.

Row	Verdict	Why
5–7, 10, 11	Corrected	Town names written differently from the PSGC were matched: "Legaspi City" → City of Legazpi, "DARAGA" → Daraga, "Sto. Domingo" → Santo Domingo, "Ligao City" → City of Ligao.
8	Warning	The center code was blank; DRIW recovered EC-ALB-002 from the same center's other row.
9	Warning	97 families in a center for 80: overcrowding, kept and flagged for operations.
12	Rejected	"Malniao" is equally close to two Albay towns (Malinao and Manito); DRIW does not guess.
13	Rejected	61 persons for 64 families: the counts are likely swapped.
14	Rejected	"10/32/2024" is not a date.

The Quezon City incident log goes through the same rules. Its plain-language request types, priorities and team names are translated by its layout. An unknown team ("Barangay Tanod"), a team recorded as arriving before it was dispatched, and an unclassified request are rejected, and a ticket number used twice is not stored. Submitted in testing, its five valid requests reached the response fact table, its three rejected rows went to quarantine, and every warehouse check still passed.

Safeguards

- **Same rules, separate sandbox.** Checking runs the warehouse's own staging models against a separate database, so a file can be checked any number of times without touching the warehouse.
- **A bad file cannot break a load.** Five rules were added for problems synthetic data never had but real files do (unknown responding unit, dispatch before request, completion before arrival, unknown damage category, an event redefined with different dates), and impossible dates or numbers now become flagged blanks instead of database errors.
- **The synthetic-data proof stays clean.** Submitted batches are marked `origin = 'intake'`; the planted-defect reconciliation (19/19) and the ground-truth matching test only consider the synthetic extracts.
- **Submitted data stays private.** Files in `data/intake/` are excluded from Git; uploads are stored outside the web root and deleted after checking; the PHP endpoint passes arguments without a shell and accepts only `.xlsx` and `.csv` files up to 10 MB.
- **Uploads only from the server itself** until the site has sign-in (configurable), because an open upload form must not be reachable from the network.

10.3 Other ways in and out

Table 72 lists the other channels.

Table 72: Other ways data enters and leaves DRIW

All of them pass through the same roles and checks as the dashboards.

Direction	Channel	Notes
In	Standard files placed in `data/sources/<type>/`	The synthetic extracts, or any file already in the standard layout; loaded by `pipeline.py raw dbt`
Out	Download CSV on every dashboard table	Raw values; opens directly in Excel
Out	PHP API (`/driw/api/marts.php`)	JSON for other systems, or Excel and Power BI through a web query
Out	Direct SQL by role	ODBC/JDBC from Excel, Power BI or Tableau; analysts see all rows, regional officers their region

10.4 Next steps

Table 73 lists the extensions that would come next.

Table 73: Next steps for data exchange

Each builds on mechanisms already in the warehouse, such as row-level security and layout profiles.

Extension	What it adds
Sign-in mapped to roles	LGU officers log in, upload from anywhere, and see only their region, using the row-level security already in the database
Submission tracking	A dashboard of which LGUs have reported for each event, and which are late or missing
Scheduled, incremental loads	Load new submissions on a schedule and add only what changed, instead of a full reload per submission
Layout profiles from the page	Let a steward map a new office's headings on screen instead of editing a file
Open-data publication	Aggregated, anonymized figures in the shape of UNDRR / Sendai Framework indicators

11. Conclusion and Recommendations

11.1 Summary

The project set out to give Philippine disaster managers one trusted view of an event that five agencies report separately. DRIW does this. It integrates NDRRMC situation reports and lifeline status, DSWD DROMIC evacuation reports (inside and outside evacuation centers), NGO, UN and LGU relief records, PAGASA bulletins and LGU response logs into one dimensional model. Every row is traceable to its source file, checked by 45 data-quality rules, and served to three audiences through dashboards, CSV and Excel exports and an API.

Table 74 matches each objective with what was achieved.

Table 74: Project objectives and what was achieved

Each objective of the business case (Chapter 1) is matched with the evidence that it was reached; the evidence is detailed in the chapter named.

Objective	Outcome	Evidence
Integrate the agencies' data in their native formats	113,191 source rows from 5 systems (CSV, Excel, JSON) in one model	Chapter 4
Conform places, events, organizations and units	All sources matched to PSGC codes with 0 false matches; event aliases, item units converted	Chapters 4–5
Prove data quality, not assume it	19 of 19 planted defect types detected at the exact count (9,995 defects)	Chapter 5
Model the business for analysis	Six star schemas, ten conformed dimensions, SCD Type 2 location history	Chapter 2
Answer the business questions quickly	14 of 14 sample queries under 1 second; eight marts for the dashboards	Chapters 7–8
Protect the data	Four roles, row-level security by region, read-only web login, no personal data	Chapter 6
Exchange data with LGUs and agencies	Excel templates, layout profiles, checked uploads, Excel export	Chapter 10
Rebuild and verify with one command	117 dbt checks and 91 automated tests pass on every build	Chapter 8

11.2 What the analysis demonstrates

The results fall into two kinds, and they should be read differently. **Demonstrated results** are properties of the system, measured by tests: the integration, the data-quality proof, the access rules and the query speed in section 11.1. **Illustrative findings** come from the synthetic figures: they show that the warehouse can answer all twelve business questions, but they are not findings about real Philippine disasters. On the synthetic data the dashboards expose patterns such as these:

- **Cost and mortality rank events differently.** The costliest event is not the deadliest per 100,000 people, so leadership needs both views (BQ1, BQ11).
- **Most displaced people are outside evacuation centers.** Counting only centers would miss more than half of the people who need relief (BQ5).
- **Overcrowding is a landfall-day problem**, and dispatch is slowest for logistics requests, not rescues (BQ6, BQ7).
- **Relief is uneven across provinces**, even within one storm, and the least-served provinces are visible at a glance (BQ8, BQ9).
- **Bridges and water take longest to restore**, and they decide how long communities stay cut off (BQ12).

11.3 Limitations

The operational figures are synthetic, because agency data is not public; population is estimated from barangay counts until the PSA census file is loaded; loads are full batch reloads; and uploads are limited to the server until the site has sign-in. The Known limitations section of Chapter 8 lists them in full. None of them requires a change to the design: replacing the generator with real extracts changes only the input files.

11.4 Recommendations

For the agencies using DRIW

1. Adopt the PSGC code as the required place identifier in every report form. Most data-quality work in this project is spent matching place names that a code would make unnecessary.
2. Report displacement outside evacuation centers and lifeline status on the same schedule as evacuation figures, so total displacement and recovery can be tracked daily.
3. Open overflow evacuation centers and pre-position relief before landfall, and pre-position bridge modules and water-treatment units in the areas most often hit.
4. Use the least-served-provinces ranking to direct the next relief allocation during a response.

For the next version of the system

1. Load the PSA census population so the per-100,000 rates become official figures.
2. Add sign-in mapped to the database roles, so LGU officers can upload from anywhere and see only their region.

3. Move from full reloads to scheduled, incremental loads as volumes grow, with indexes on the most filtered fact keys and partitioning by event year.
4. Publish aggregated, anonymized indicators in the shape of the Sendai Framework targets.

DRIW shows that a governed, tested enterprise data warehouse can turn five agencies' reports into one consistent picture of a disaster, and that its correctness can be demonstrated, not just claimed.

References

References follow APA 7th edition style, in alphabetical order. Laws are cited by their number and year in the text, for example (Republic Act No. 10173, 2012).

dbt Labs. (n.d.). *dbt documentation*. https://docs.getdbt.com/

Department of Social Welfare and Development [DSWD]. (n.d.). *Disaster Response Operations Monitoring and Information Center (DROMIC) reports*. https://dromic.dswd.gov.ph/

Inmon, W. H. (2005). *Building the data warehouse* (4th ed.). Wiley.

Kimball, R., & Ross, M. (2013). *The data warehouse toolkit: The definitive guide to dimensional modeling* (3rd ed.). Wiley.

National Disaster Risk Reduction and Management Council [NDRRMC]. (n.d.). *Situation reports*. https://ndrrmc.gov.ph/

Philippine Atmospheric, Geophysical and Astronomical Services Administration [PAGASA]. (n.d.). *Tropical cyclone information*. https://www.pagasa.dost.gov.ph/

Philippine Statistics Authority. (n.d.). *Philippine Standard Geographic Code (PSGC)*. https://psa.gov.ph/classification/psgc

PostgreSQL Global Development Group. (2024). *PostgreSQL 17 documentation*. https://www.postgresql.org/docs/17/

PSGC API. (n.d.). *Philippine Standard Geographic Code API* [Community mirror of PSA codes]. https://psgc.gitlab.io/api/

Republic Act No. 10121. (2010). *Philippine Disaster Risk Reduction and Management Act of 2010*. Congress of the Philippines.

Republic Act No. 10173. (2012). *Data Privacy Act of 2012*. Congress of the Philippines.

Republic Act No. 11550. (2021). *An act dividing the province of Maguindanao into the provinces of Maguindanao del Norte and Maguindanao del Sur*. Congress of the Philippines.

United Nations Economic and Social Commission for Asia and the Pacific [ESCAP]. (2018). *Disaster-related statistics framework (DRSF)*. Expert Group on Disaster-related Statistics in Asia and the Pacific.

United Nations Office for Disaster Risk Reduction [UNDRR]. (2015). *Sendai Framework for Disaster Risk Reduction 2015–2030*. United Nations. https://www.undrr.org/publication/sendai-framework-disaster-risk-reduction-2015-2030

United Nations Office for Disaster Risk Reduction [UNDRR]. (2017). *Technical guidance for monitoring and reporting on progress in achieving the global targets of the Sendai Framework for Disaster Risk Reduction*. United Nations.

United Nations Office for Disaster Risk Reduction [UNDRR]. (2023). *UNDRR data strategy and roadmap 2023–2027*. United Nations. https://www.undrr.org/data-strategy-and-roadmap-2023-2027

Appendix A. Sample Analytical Queries

Each query answers a key business question (section 1.3) and demonstrates an OLAP operation. Timings are in the Query Performance Test Report (Chapter 8). Source: docs/sample-queries.sql.

Q01. Costliest events and their human toll (BQ1)

OLAP operation: Ranking (top-N)

```
select e.event_name,
       e.category,
       sum(f.cost_php)          as damage_cost_php,
       sum(f.dead)              as dead,
       sum(f.affected_persons)  as affected_persons
from dw.fact_damage_assessment f
join dw.dim_event e using (event_key)
group by e.event_name, e.category
order by damage_cost_php desc
limit 10;
```

Q02. Damage by year and category, with yearly subtotals (BQ2)

OLAP operation: Roll-up (GROUP BY ROLLUP)

```
select coalesce(e.event_year::text, 'All years')            as event_year,
       coalesce(dc.damage_category, 'All categories')       as damage_category,
       sum(f.cost_php)                                      as damage_cost_php
from dw.fact_damage_assessment f
join dw.dim_event e using (event_key)
join dw.dim_damage_category dc using (damage_category_key)
group by rollup (e.event_year, dc.damage_category)
order by e.event_year nulls last, dc.damage_category nulls last;
```

Q03. Drill-down: region -> province -> city/municipality for one event (BQ3)

OLAP operation: Drill-down

```
select l.region_name,
       l.province_name,
       l.city_muni_name,
       sum(f.affected_persons) as affected_persons,
       sum(f.cost_php)         as damage_cost_php
from dw.fact_damage_assessment f
join dw.dim_location l using (location_key)
join dw.dim_event e using (event_key)
where e.event_code = 'TC2024-KRISTINE'
group by l.region_name, l.province_name, l.city_muni_name
order by affected_persons desc
limit 15;
```

Q04. Tropical cyclone vs monsoon flood impact per region (BQ4)

OLAP operation: Pivot (FILTER) / dice

```
select l.region_name,
       sum(f.affected_persons) filter (where e.hazard_type = 'Tropical Cyclone')
           as cyclone_affected,
       sum(f.affected_persons) filter (where e.hazard_type = 'Flood')
           as flood_affected,
       sum(f.affected_persons) as total_affected
from dw.fact_damage_assessment f
join dw.dim_event e using (event_key)
join dw.dim_location l using (location_key)
group by l.region_name
order by total_affected desc;
```

Q05. Every subtotal of impact by year x region x hazard (BQ2, BQ4)

OLAP operation: Cube (GROUP BY CUBE)

```
select e.event_year, l.region_name, e.hazard_type,
       grouping(e.event_year, l.region_name, e.hazard_type) as grouping_id,
       sum(f.dead)            as dead,
       sum(f.affected_persons) as affected_persons,
       sum(f.cost_php)        as damage_cost_php
from dw.fact_damage_assessment f
join dw.dim_event e using (event_key)
join dw.dim_location l using (location_key)
group by cube (e.event_year, l.region_name, e.hazard_type);
```

Q06. Peak evacuees per event (semi-additive measure) (BQ5)

OLAP operation: Aggregate over a snapshot

```
select e.event_name,
       max(daily.persons) as peak_persons_in_centers,
       count(*)           as days_with_evacuees
from (
    -- sum across centers per day
    select event_key, snapshot_date_key, sum(persons_inside) as persons
    from dw.fact_evacuation_snapshot
    group by event_key, snapshot_date_key
) daily
join dw.dim_event e using (event_key)
group by e.event_name   -- then MAX over days, never SUM
order by peak_persons_in_centers desc
limit 10;
```

Q07. Evacuation centers most often over capacity (BQ6)

OLAP operation: Slice (is_over_capacity)

```
select c.ec_name, c.ec_type, l.province_name,
       count(*) filter (where f.is_over_capacity) as over_capacity_days,
       max(f.families_inside)                     as max_families,
       c.capacity_families
from dw.fact_evacuation_snapshot f
join dw.dim_evacuation_center c using (evacuation_center_key)
join dw.dim_location l using (location_key)
group by c.ec_name, c.ec_type, l.province_name, c.capacity_families
having count(*) filter (where f.is_over_capacity) > 0
order by over_capacity_days desc, max_families desc
limit 10;
```

Q08. Dispatch time by response type and priority (BQ7)

OLAP operation: Accumulating snapshot analysis

```
select rt.response_type_name,
       f.priority,
       count(*) as requests,
       round(percentile_cont(0.5) within group (order by f.hours_to_dispatch)::numeric,
2)
           as median_hours_to_dispatch,
       round(percentile_cont(0.9) within group (order by f.hours_to_dispatch)::numeric,
2)
           as p90_hours_to_dispatch,
       round(100.0 * avg(f.is_completed::int), 1) as pct_completed
from dw.fact_response_action f
join dw.dim_response_type rt using (response_type_key)
group by rt.response_type_name, f.priority
order by rt.response_type_name, f.priority;
```

Q09. Slowest-responding cities/municipalities for life-saving requests (BQ7)

OLAP operation: Slice + ranking

```
select l.province_name, l.city_muni_name,
       count(*) as life_saving_requests,
       round(percentile_cont(0.5) within group (order by f.hours_to_dispatch)::numeric,
2)
           as median_hours_to_dispatch
from dw.fact_response_action f
join dw.dim_response_type rt using (response_type_key)
join dw.dim_location l using (location_key)
where rt.response_cluster = 'Life-saving'
group by l.province_name, l.city_muni_name
having count(*) >= 20
order by median_hours_to_dispatch desc
limit 10;
```

Q10. Least relief per affected person, by province (BQ8)

OLAP operation: Integration of two facts

```
with impact as (
    select l.province_name, sum(f.affected_persons) as affected
    from dw.fact_damage_assessment f join dw.dim_location l using (location_key)
    group by l.province_name
), relief as (
    select l.province_name, sum(f.value_php) as relief_php
    from dw.fact_resource_distribution f join dw.dim_location l using (location_key)
    group by l.province_name
)
select i.province_name, i.affected, coalesce(r.relief_php, 0) as relief_php,
       round(coalesce(r.relief_php, 0) / nullif(i.affected, 0), 2) as
relief_php_per_person
from impact i left join relief r using (province_name)
order by relief_php_per_person
limit 10;
```

Q11. Relief mix by organization and resource category (BQ9)

OLAP operation: Dice (org x category)

```
select o.org_name, r.category,
       sum(f.quantity)  as quantity_standard_units,
       sum(f.value_php) as relief_value_php
from dw.fact_resource_distribution f
join dw.dim_organization o using (organization_key)
join dw.dim_resource r using (resource_key)
group by o.org_name, r.category
order by o.org_name, relief_value_php desc;
```

Q12. Year-over-year change in damage cost (BQ2)

OLAP operation: Window function (LAG)

```
select event_year,
       damage_cost_php,
       round(100.0 * (damage_cost_php - lag(damage_cost_php) over (order by event_year))
             / nullif(lag(damage_cost_php) over (order by event_year), 0), 1)
           as pct_change_vs_prior_year
from (
    select e.event_year, sum(f.cost_php) as damage_cost_php
    from dw.fact_damage_assessment f join dw.dim_event e using (event_key)
    group by e.event_year
) yearly
order by event_year;
```

Q13. Dashboard query served from a mart (BQ1, BQ5, BQ8)

OLAP operation: Pre-aggregated ROLAP mart

```
select event_name, damage_cost_php, peak_persons_in_centers,
relief_php_per_affected_person
from mart.mart_event_summary
order by damage_cost_php desc
limit 10;
```

Q14. What data was rejected, and why? (BQ10)

OLAP operation: Data-quality audit

```
select source_system, reject_reasons, count(*) as rows_quarantined
from dq.dq_quarantine
group by source_system, reject_reasons
order by rows_quarantined desc;
```